BİLGİ KONTROL LİSTESİ

System Design Master

Temellerden üretime sistem tasarımında bilmen gerekenler



216 kavram · 19 teknik şema · 127 kaynak bağlantısı

Ratip Uysal | 6 Kasım 2026

Tıklanabilir içindekiler, aranabilir metin ve geniş şema sayfalarıyla okuma sürümü.

İçindekiler

Bir başlığa veya sayfa numarasına dokunarak bölüme gidebilirsin. Her sayfanın altındaki İçindekiler bağlantısı bu dizine döner. 001-216 numaraları yalnız kavramlara aittir; bölüm veya okuma sırası değildir.

Temeller 6

Gereksinimler ve tasarım kararları . . . 6

Teknik keşif ve seçenekleri sınamak . . . 7

Ürün davranışı ve kabul koşulları . . . 9

Bilgisayar ve çalışma zamanı . . . 10

İnternet ve istek yaşam döngüsü . . . 12

Web arayüzünün üretilmesi . . . 14

İstemci platformu ve erişilebilirlik . . . 15

API ve sözleşme tasarımı . . . 18

Veri modelleme ve sahiplik . . . 22

Uygulama mimarisi . . . 25

Üretimde çalışan sistemler 28

Depolama ve veritabanı seçimi . . . 28

Transaction ve eşzamanlı veri erişimi . . . 30

Önbellekleme . . . 33

Ölçekleme ve trafik yönetimi . . . 36

Mesajlaşma ve arka plan işleri . . . 40

Dayanıklılık ve hata yönetimi . . . 43

Güvenlik ve mahremiyet . . . 47

Gözlemlenebilirlik ve hizmet hedefleri . . . 50

Operasyon örnekleri: arızayı yönetmek . . . 52
Teknik teşhis araçlarını okuyabilmek . . . 53
Ürün analitiği ve deneyler . . . 56
Test ve doğrulama . . . 59
Dağıtım ve altyapı . . . 60
Geliştirmeden kullanıcıya ulaşma zinciri . . . 62

İleri sistem tasarımı **64**

Dağıtık sistem garantileri . . . 64
Çok bölgeli çalışma ve felaket kurtarma . . . 71
Arama ve veri işleme . . . 74
Gerçek zamanlı ve çevrimdışı sistemler . . . 75
Yapay zekâ kullanan sistemler . . . 78
Yapay zekâ çözümünü seçmek . . . 80
AI çıktı sözleşmeleri ve araç entegrasyonları . . . 82
Çok kiracılı ürünler ve maliyet . . . 83

Performans ve hız **85**

Ölçüm ve teşhis . . . 85
Görseller ve fontlar . . . 86
JavaScript ve üçüncü taraflar . . . 86
Veritabanı ve API . . . 87
Algılanan hız ve etkileşim . . . 88
Sunucu ve dağıtım . . . 89

Ek mimari ve ürün kararları **91**

Üretimde veri güvenliği ve karar sınırları 95

Bütüncül sistem örnekleri 101

Örnek sistem: ödeme ve sipariş . . . 101

Örnek sistem: arka planda rapor hazırlama . . . 103

Örnek sistem: kaynaklı AI destek asistanı . . . 104

Sayısal kapasite ve maliyet örneği 106

Trafikten sunucu sayısına . . . 106

Veriden depolama ve çıkış trafiğine . . . 106

Varsayımsal birim fiyatlarla maliyet . . . 107

Terim sözlüğü 108

Kaynakça ve önemli bağlantılar 111

HTTP ve API sözleşmeleri . . . 111

Teşhis araçları ve SQL . . . 111

Geliştirme ve yayın . . . 111

Analitik ve güvenilirlik . . . 111

Dağıtık sistemler ve güvenlik . . . 112

Yapay zekâ sistemleri . . . 112

Önceki kişisel çalışmadan örnekler . . . 112

Teknik diyagram örnekleri . . . 112

Bu genişletmenin başvuru bağlantıları . . . 112

İstemci platformları ve AI sözleşmeleri için kaynaklar . . . 113

Ek kararlar için birincil kaynaklar . . . 113

Üretim kararları için birincil kaynaklar . . . 114

Rehberi nasıl kullanmalı

Metin ve şemalar yapay zekâ desteğiyle hazırlandı. Şemalar sadeleştirilmiş öğretim modelleridir. Ürün senaryoları temsili, sayısal örneklerdeki trafik ve fiyatlar varsayımdır.

Sistem tasarımında bilmen gereken kavramlar, kararlar ve hata durumları. Temellerden ileri dağıtık sistemlere uzanan bir bilgi kontrol listesi; web, mobil, backend ve yapay zekâ kullanan ürünleri kapsar.

Her madde kısa açıklama, günlük bir ürün senaryosu ve kullanım koşuluyla açılır. Örnekler temsili senaryolardır; sayısal örnekler hesaplamayı göstermek içindir. İlgili konulardaki sık hata ve karar soruları, kavramları gerçek ürün kararlarına bağlar.

Bir kutuyu işaretlemek için konunun ne olduğunu, hangi sorunu çözdüğünü, hangi koşulda kullanılacağını ve hangi bedelleri getirdiğini açıklayabilmelisin. Başlıklar bilgi alanlarını gösterir; bir takvim veya zorunlu ders sırası oluşturmaz. Her tekniği her üründe kullanmak gerekmez.

28 Ekim 2026 tarihinde paylaştığın performans listesi korunarak kapsam genişletildi. Performans maddelerindeki koşula bağlı öneriler, uygulanacakları bağlamla netleştirildi.

Sayfada yolunu bul

Temel: Çoğu ürün kararında bilmen gereken bilgi. **İhtiyaca bağlı:** Belirli bir trafik, platform veya iş gereksiniminde devreye girer. **İleri:** Dağıtık koordinasyon ve işletim bedeli yüksek konular. Etiketler önem ve kullanım bağlamını gösterir; bir ders sırası değildir.

Aradığın cevap	İlgili başlık
Kullanıcı ne istiyor; hangi sınırlarla tasarlıyoruz?	Temeller
İstek, API, veri ve arayüz nasıl birleşiyor?	İnternet ve istek yaşam döngüsü; Web arayüzünün üretilmesi; Uygulama mimarisi
Trafik artınca veya bir bağımlılık bozulunca ne oluyor?	Üretimde çalışan sistemler
Birden fazla servis ve bölge nasıl tutarlı çalışıyor?	İleri sistem tasarımı
Bu uygulama neden yavaş?	Performans ve hız
Parçalar gerçek bir senaryoda nasıl birleşiyor?	Bütüncül sistem örnekleri
Kaç sunucu ve ne kadar bütçe gerekir?	Sayısal kapasite ve maliyet örneği
Bu kısaltma ne demek?	Terim sözlüğü

Temeller

Gereksinimler ve tasarım kararları

001 Fonksiyonel gereksinimler ve kalite hedefleri: Sistemin ne yapacağı ile hız, güvenilirlik, güvenlik ve maliyet beklentilerini ayırmak.

Temel

Açıklama: İşlev, kullanıcının yapabildiği iştir; kalite hedefi bunun ne kadar hızlı ve güvenilir gerçekleştiğidir.

Örnek: Yemek uygulamasında sipariş verebilmek işlevdir; ödeme ekranının yoğun saatte de açılması kalite beklentisidir.

Ne zaman kullanılır: Bir ürüne başlarken ve hangi sorunun öncelikli olduğuna karar verirken.

002 Kısıtlar ve varsayımlar: Kullanıcı sayısı, trafik, veri hacmi, ekip kapasitesi, bütçe ve dış bağımlılıkları belirlemek.

Temel

Açıklama: Tasarıma yön veren sınırları ve henüz ölçülmemiş kabulleri görünür kılarsın.

Örnek: İki kişilik ekibin işlettiği küçük bir randevu uygulamasıyla milyonlarca ziyaretçili bilet sitesi aynı altyapıya ihtiyaç duymaz.

Ne zaman kullanılır: Teknoloji ve hizmet seçmeden önce; trafik veya ekip değiştiğinde yeniden.

003 İş kuralları ve değişmezler: Çift tahsilat yapılmaması, stokun eksiye düşmemesi gibi her durumda korunacak doğruluk kuralları.

Temel

Açıklama: Değişmez, başka her şey aksasa da bozulmaması gereken iş kuralıdır.

Örnek: Son konser koltuğunu aynı anda isteyen iki kişiden yalnızca biri satın alabilmelidir.

Ne zaman kullanılır: Ödeme, stok, rezervasyon ve kullanıcı hakkı gibi hatası geri dönüşü zor alanlarda.

004 Kapasite hesabı: Ortalama ve tepe istek/saniye, eşzamanlı kullanıcı, veri büyümesi, bant genişliği ve kapasite payı.

Temel

Açıklama: Beklenen işi sayıya çevirip sistemin taşıması gereken yükü tahmin edersin. Ortalama değer, ani yığılmayı tek başına göstermez.

Örnek: Bir konserin biletleri açıldığında bir günlük ziyaret birkaç dakikaya sıkışabilir.

Ne zaman kullanılır: Sunucu ve veritabanı kapasitesi seçerken, kampanya öncesinde ve büyüme maliyetini hesaplarken.

005 Ödünleşimler: Hız, tutarlılık, kullanılabilirlik, maliyet ve geliştirme kolaylığı arasında karar vermek.

Temel

Açıklama: Her seçimin kazandırdığı ve zorlaştırdığı bir şey vardır; hedefe göre hangisini kabul edeceğini belirlersin.

Örnek: Ürün yorumları biraz gecikmeli görünebilir; ödeme sonucu için aynı esneklik kabul edilmeyebilir.

Ne zaman kullanılır: Birden fazla makul çözüm arasında seçim yaparken.

006 Tasarımı anlatmak: Sistem sınırı, bileşen ve sıra diyagramları; veri akışı; mimari karar kaydı ve reddedilen alternatiflerin gerekçesi.

Temel

Açıklama: Diyagramlar parçaları ve iletişimi, karar kaydı ise seçimin nedenini açıklar.

Örnek: Sipariş akışında ödeme başarısız olursa stok rezervasyonunu kimin kaldıracağı sıra diyagramında görünür.

Ne zaman kullanılır: İşi başka geliştiriciye devrederken, mimariyi incelerken ve aylar sonra kararı yeniden değerlendirirken.

Teknik keşif ve seçenekleri sınamak

007 Problem ile çözüm önerisini ayırmak: Kullanıcı sonucunu, mevcut kanıtı ve teknik hipotezi ayrı tanımlamak.

Temel

Açıklama: Bir teknoloji isteği bazen sorunun erken seçilmiş çözümüdür. Önce kullanıcının hangi işinin aksadığını ve bunu gösteren bulguyu yazarsın.

Örnek: “Bildirim sağlayıcısını değiştirelim” talebinin arkasında, işlerin kendi kuyruğunda uzun süre beklemesi olabilir.

Ne zaman kullanılır: Yeni altyapı, sağlayıcı veya mimari değişikliği önerildiğinde.

Sık hata: Bir ekibin tahminini ölçülmüş kök neden gibi sunmak.

Karar sorusu: Hangi bulgu sorunun önerdiğimiz bileşenden kaynaklandığını gösteriyor?

008 PoC ve spike: Kararı değiştirecek belirsizliği küçük ve sınırlı bir denemeyle sınamak.

Temel

Açıklama: PoC bir yaklaşımın işe yarayıp yaramayacağını, spike ise belirli bir teknik sorunun cevabını araştırır. Amaç, sınır ve karar ölçütü baştan açık olmalıdır.

Örnek: Bildirim sağlayıcısına tek mesaj göndermek yerine temsilî yükte gecikme, kota ve tekrar davranışını ölçersin.

Ne zaman kullanılır: Belirsizlik yüksekken ve yanlış seçimin geliştirme veya geçiş maliyeti büyükken.

Sık hata: Güzel çalışan bir demoyu üretim kapasitesi ve güvenilirlik kanıtı saymak.

Karar sorusu: Hangi sonuç bu seçeneği elememize veya seçmemize neden olacak?

009 Seçenek karşılaştırması: Geliştirme, işletim, geçiş ve çıkış maliyetini birlikte değerlendirmek.

Temel

Açıklama: İlk kurulum kadar bakım, gözlemleme, veri taşıma ve sağlayıcıdan ayrılma maliyeti de kararı etkiler. Tahminlerin dayandığı varsayımları görünür tutarsın.

Örnek: Ucuz mesaj servisi, ihtiyaç duyulan teslim raporlarını vermediği için destek ekibine daha fazla iş çıkarabilir.

Ne zaman kullanılır: Build versus buy, sağlayıcı seçimi veya önemli bir yeniden tasarım sırasında.

Sık hata: Yalnız aylık servis fiyatını karşılaştırmak.

Karar sorusu: Bu çözümü bir yıl işletmenin ve gerektiğinde bırakmanın maliyeti nedir?

010 Karar kaydı ve yeniden değerlendirme: Seçimi, alternatifleri ve kararı değiştirecek kanıtı saklamak.

Temel

Açıklama: Bir kararın hangi koşullarda doğru olduğunu yazarsın; belirsizliklerin sahibi ve yeniden bakma tetikleyicisi olur. Böylece ekip aynı tartışmayı bağlamsız tekrarlamaz.

Örnek: Modüler monolit seçimini, bağımsız ekip ve dağıtım ihtiyacı belirginleşirse yeniden değerlendirmek üzere kaydedersin.

Ne zaman kullanılır: Uzun süre etkili olacak teknik veya sağlayıcı kararlarında.

Sık hata: Varsayımları zamanla kesin gerçek kabul etmek.

Karar sorusu: Hangi yeni veri geldiğinde bu kararı yeniden açmalıyız?

Ürün davranışı ve kabul koşulları

011 Doğrulanabilir kabul koşulları: Başlangıç durumu, olay, beklenen veri değişimi ve kullanıcı sonucunu açık yazmak.

Temel

Açıklama: Bir gereksinim, doğru çalışıp çalışmadığı gözlenebildiğinde değerlendirilebilir. Hız gibi hedefler ölçüm noktası, cihaz, ağ ve yük koşuluyla anlam kazanır.

Örnek: “Hızlı giriş” yerine, belirlenen cihaz ve ağda girişten kullanılabilir profil ekranına kadar geçen süreyi hedeflersin.

Ne zaman kullanılır: Gereksinim yazarken, teknik tasarım değerlendirirken veya teslimatı kabul ederken.

Sık hata: “Çalışmalı” ve “hızlı olmalı” gibi yoruma açık ifadelerle yetinmek.

Karar sorusu: Bu davranışın gerçekleştiğini hangi gözlemle anlayacağız?

012 Sonuç ve hata matrisi: Başarı, boş veri, erişim reddi, kısmi başarı ve bilinmeyen sonucu ayırmak.

Temel

Açıklama: Her sistem sonucu için ekrandaki durum ve güvenli sonraki eylem tanımlanır. HTTP kodu, işin sonucu ve kullanıcıya gösterilen metin birlikte değerlendirilir.

Örnek: Kampanya listesinde 200 ve boş dizi boş ekran açıklaması gerektirir; 403 erişim reddidir; ağ kopması ise yeniden deneme yolu gerektirebilir.

Ne zaman kullanılır: API sözleşmesi, ekran durumları ve entegrasyon hataları tasarlanırken.

Sık hata: Tüm sonuçları tek bir genel hata mesajına veya sonsuz yüklemeye çevirmek.

Karar sorusu: Kullanıcı her durumda ne olduğunu anlayıp güvenli bir sonraki adım atabiliyor mu?

013 Destek ve uzlaşma yolu: Belirsiz veya yarım kalan işlemin sahibini ve toparlanma yöntemini tanımlamak.

Temel

Açıklama: Bazı işlemler kullanıcı ekranından çözülemez. İş referansı, durum sorgusu, yeniden kontrol ve gerektiğinde insan desteği aynı akışın parçasıdır.

Örnek: Ödeme alınmış ama paket hakkı açılmamışsa sipariş ve sağlayıcı referanslarıyla uzlaşma işi sonucu doğrular; destek bu kaydı bulabilir.

Ne zaman kullanılır: Ödeme, rezervasyon, veri aktarımı ve dış sağlayıcıya bağlı kritik işlemlerde.

Sık hata: Kullanıcıya tekrar tekrar denemesini söyleyip yarım işlemleri sahipsiz bırakmak.

Karar sorusu: İşlem takılırsa kim, hangi referansla, hangi sonuca kadar takip edecek?

Bilgisayar ve çalışma zamanı

014 CPU, bellek, disk ve ağ: Bir işin işlemciye mi, belleğe mi, giriş çıkışa mı takıldığını ayırt etmek.

Temel

Açıklama: CPU hesaplar, bellek çalışan veriyi tutar, disk kalıcı saklar, ağ başka makinelere erişir. Birindeki yavaşlık diğerini büyüterek çözülmeyebilir.

Örnek: Video dönüştürmek işlemciyi; uzaktaki dosyayı indirmek ağı meşgul eder.

Ne zaman kullanılır: Uygulama yavaşladığında hangi kaynağı iyileştireceğini bulmak için.

015 Process, thread ve event loop: İşlerin nasıl yürütüldüğü; blocking ve nonblocking işlemler.

Temel

Açıklama: Process ayrı bir çalışan programdır; thread onun içindeki yürütme yoludur. Event loop bekleyen işleri sırayla ilerletir; uzun bir engelleyici iş diğerlerini durdurabilir.

Örnek: Tek görevlinin olduğu kafede görevli kahvenin demlenmesini beklerken yeni sipariş alabilir; bütün süreyi bekleyerek geçirirse sıra büyür.

Ne zaman kullanılır: Sunucuda uzun hesaplar veya çok sayıda ağ isteği çalıştırırken.

016 Eşzamanlılık ve paralellik: Birden çok işin ilerlemesi ile aynı anda çalışması arasındaki fark.

Temel

Açıklama: Eşzamanlılık, işleri birbirini bekletmeden ilerletmektir. Paralellik, işleri aynı anda farklı işlem kaynaklarında yürütmektir.

Örnek: Bir aşçı fırındaki yemek pişerken salata hazırlar; iki aşçı ise iki yemeği aynı anda hazırlar.

Ne zaman kullanılır: Ağ beklemesi ile yoğun hesaplama için farklı yürütme yöntemleri seçerken.

017 Veri yapıları ve karmaşıklık: Dizi, hash map, ağaç, heap ve kuyruk; zaman ve bellek maliyetleri.

Temel

Açıklama: Veriyi nasıl tuttuğun, arama ve güncelleme işinin ne kadar büyüyeceğini belirler. Hash map anahtardan bulmayı, kuyruk sırayla işlemeyi kolaylaştırır.

Örnek: Müşteriyi numarasından bulmak için her seferinde tüm müşteri listesini taramak yerine anahtarlı bir yapı kullanırsın.

Ne zaman kullanılır: Veri büyüdüğünde arama, sıralama veya bellek kullanımı sorun yaratıyorsa.

018 Kaynak sınırları: Bağlantı havuzu, dosya tanımlayıcıları, thread havuzu, bellek sızıntısı ve garbage collection.

Temel

Açıklama: Bağlantı ve çalışan iş sayısı sınırsız değildir. Havuz bunları yeniden kullanır; sızıntı ise iş bitse de kaynağı serbest bırakmaz.

Örnek: Her sayfa açılışında yeni veritabanı bağlantısı açıp kapatmayan uygulama bir süre sonra yeni kullanıcı kabul edemez.

Ne zaman kullanılır: Yoğunlukta hata artarken, uygulama zamanla yavaşlarken veya bellek sürekli büyürken.

019 Serileştirme ve zaman: JSON ve ikili formatlar; karakter kodlaması; saat dilimi, UTC ve süre ölçümünde monotonik saat.

Temel

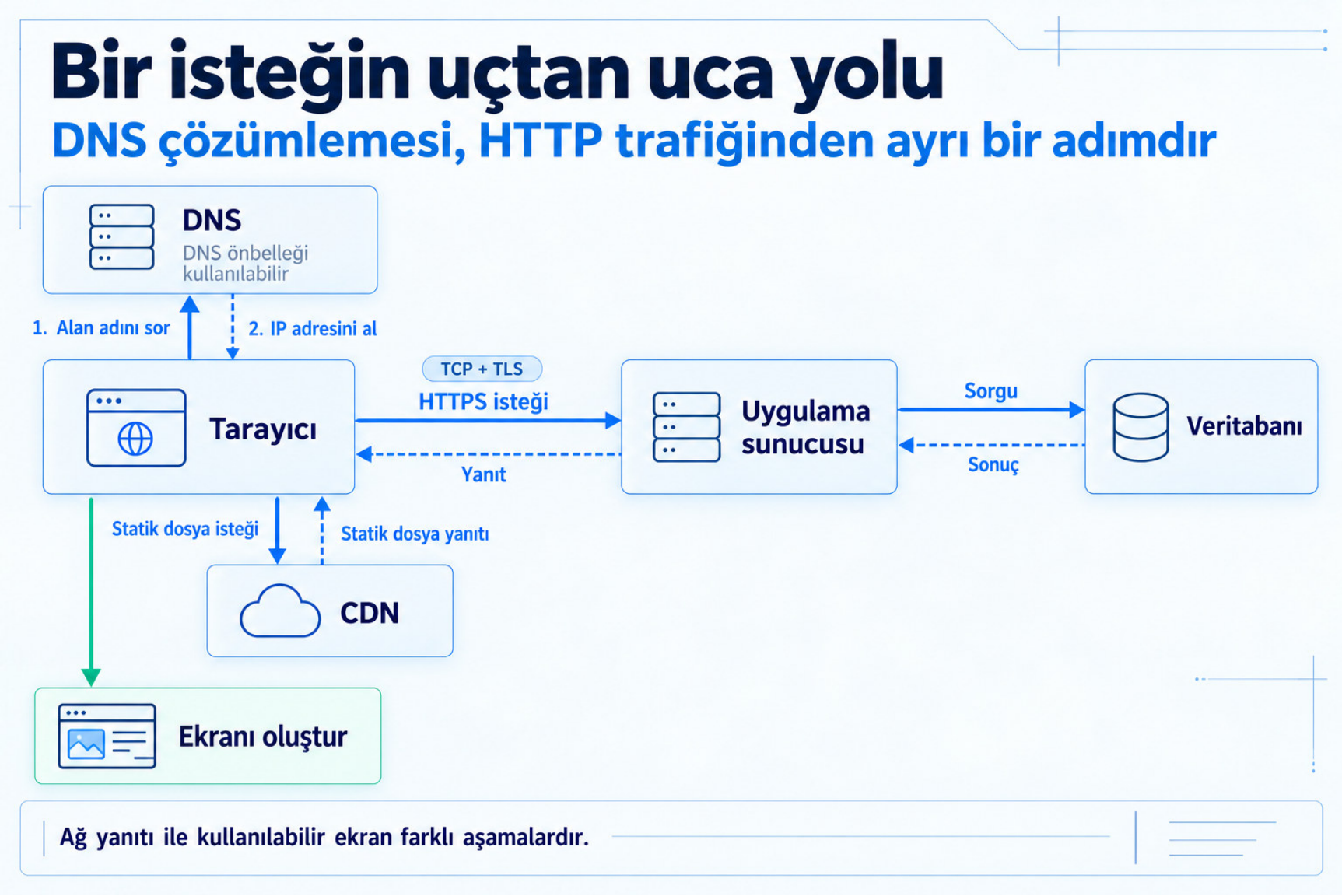
Açıklama: Serileştirme, veriyi taşınabilir biçime çevirmektir. Zamanı saklarken olayın anını ve kullanıcının saat dilimini; süre ölçerken saat ayarlarından etkilenmeyen sayacı düşünürsün.

Örnek: İstanbul'da oluşturulan toplantı Londra'daki kişiye kendi yerel saatinde gösterilir.

Ne zaman kullanılır: API, dosya aktarımı, uluslararası tarih gösterimi ve zaman aşımı ölçümü yaparken.

İnternet ve istek yaşam döngüsü

Şema 01 | Uçtan uca istek



Şemayı oku: Tarayıcı alan adını DNS ile çözer; ardından sunucuya bağlantı kurup HTTPS isteği gönderir. DNS, HTTP verisinin geçtiği bir ara sunucu değildir. Statik dosya CDN'den, dinamik veri uygulama sunucusundan gelebilir; yanıtı kullanılabilir ekrana dönüştürmek tarayıcının ayrı işidir. Yapay zekâ (Imagegen) ile üretilmiştir.

Sınır: DNS sonucu ve bağlantı yeniden kullanılabilir; her istekte baştan kurulmaları gerekmez. CDN ve yük dengeleyici gibi ara katmanlar bu sade şemada gösterilmedi.

020 Uçtan uca istek: DNS çözümlemesinden bağlantı kurulmasına, sunucuya, veritabanına ve yanıtın ekranda görünmesine kadar akış.

Temel

Açıklama: Bir ekranın açılması birçok küçük adımdan oluşur. Toplam gecikmeyi bu adımlara ayırarak sorunun yerini bulursun.

Örnek: Ürün ekranı yavaşsa sebep ürün sorgusu, fotoğraf indirmesi veya alan adının çözülmesi olabilir.

Ne zaman kullanılır: Kullanıcı 'uygulama yavaş' dediğinde incelemeye nereden başlayacağını belirlemek için.

021 IP, port, TCP, UDP ve QUIC: Bağlantı, güvenilir aktarım, paket kaybı ve protokol seçiminin etkileri.

Temel

Açıklama: IP makine adresini, port o makinedeki hizmeti belirtir. TCP sıralı güvenilir aktarım sağlar; UDP bunu kendisi sağlamaz; QUIC UDP üzerinde güvenilir akışlar ve başka özellikler ekler.

Örnek: Dosya indirmede eksiksiz aktarım, canlı ses iletişiminde geciken verinin değeri önemlidir.

Ne zaman kullanılır: Gerçek zamanlı iletişim, dosya aktarımı veya bağlantı sorunları tasarlarken.

022 HTTP ve HTTPS: Metotlar, durum kodları, başlıklar, TLS ve sertifika doğrulaması.

Temel

Açıklama: HTTP istek ve yanıt dilidir; HTTPS buna şifreli ve sunucunun kimliğini doğrulayan bağlantı ekler. Durum kodları sonucu anlatır.

Örnek: Mağazada ürün istemek GET, sipariş oluşturmak POST olabilir; 404 bulunamadığını, 401 kimlik doğrulama gerektiğini belirtir.

Ne zaman kullanılır: Web veya mobil istemciyi sunucuyla konuşturduğunda.

023 HTTP sürümleri ve bağlantı kullanımı: Keep-alive, multiplexing ve bağlantı kurulum maliyeti.

İhtiyaca bağlı

Açıklama: Bağlantıyı tekrar kullanmak her dosya için yeniden bağlantı kurma işini azaltır. Multiplexing bir bağlantıda birden çok aktarımı birlikte yürütür.

Örnek: Bir haber sayfasındaki yazı, stil ve görseller için gereksiz tekrar bağlantıları mobil ağda ek bekleme yaratır.

Ne zaman kullanılır: Çok sayıda küçük dosya veya API çağrısı olan uygulamalarda ağ gecikmesini incelerken.

024 Proxy, reverse proxy, yük dengeleyici ve CDN: İstek yolundaki farklı görevleri.

Temel

Açıklama: Proxy istemci adına, reverse proxy sunucu önünde çalışır. Yük dengeleyici işleri sunuculara böler; CDN içeriğin kopyalarını kullanıcıya yakın noktadan sunar.

Örnek: Mağazanın fotoğrafları yakındaki CDN'den gelirken sipariş isteği çalışan uygulama sunucularından birine yönlendirilir.

Ne zaman kullanılır: Trafik dağıtmak, sunucuları korumak veya dosya aktarımını hızlandırmak istediğinde.

025 Tarayıcı güvenlik sınırları: Origin, CORS, cookie ve same-origin policy; CORS'un kimlik doğrulama yerine geçmemesi.

Temel

Açıklama: Tarayıcı farklı siteler arasındaki erişimi sınırlar. CORS hangi origin'in yanıtı okuyabileceğini belirler; sunucu ayrıca kimlik ve yetki kontrolü yapmalıdır.

Örnek: Panelin ayrı bir API alan adına erişmesine izin verirsin ama bu izin başkasının siparişini okumaya yetki vermez.

Ne zaman kullanılır: Frontend ile API farklı adreslerdeyse veya cookie ile oturum yönetiyorsan.

Başvuru: MDN HTTP genel bakış.

Web arayüzünün üretilmesi

026 CSR: Arayüzü tarayıcıda JavaScript ile oluşturmak.

Temel

Açıklama: İstemci kodu çalıştırır, veriyi alır ve görünümü üretir. İlk açılış ve etkileşim maliyeti cihazın gücüne bağlıdır. Web rendering seçenekleri.

Örnek: Oturum sonrası yoğun etkileşimli bir yönetim paneli istemcide ekranlar arasında geçiş yapar.

Ne zaman kullanılır: Etkileşim ağırlıklı alanlarda; yalnız metin sunan bir sayfa için büyük istemci paketi gereksiz olabilir.

Sık hata: API hızlıysa ekranın da hızlı açılacağını varsaymak.

Karar sorusu: Yavaş telefonda ilk kullanılabilir ekranı hangi istemci işi geciktiriyor?

027 SSR: HTML'i istek sırasında sunucuda üretmek.

Temel

Açıklama: İlk içerik sunucudan gelir; dinamik üretim sunucu zamanı tüketir. HTML hazır olsa da etkileşim için istemci kodu gerekebilir.

Örnek: Kişiye göre değişen bir ürün sayfası sunucuda hazırlanır; kullanıcıya özel içerik ortak cache'e konmaz.

Ne zaman kullanılır: İlk içerik ve istek anındaki veri önemliyse; statik içerikte her istekte üretim gereksiz olabilir.

Sık hata: SSR'nin otomatik olarak az JavaScript veya hızlı etkileşim garantilediğini sanmak.

Karar sorusu: HTML üretme süresi, cache politikası ve kullanıcıya özel alanlar nasıl ayrılacak?

028 Statik üretim: Sayfaları önceden oluşturup dosya olarak sunmak.

Temel

Açıklama: SSG çıktısı her istekte yeniden üretilmez. İçerik değişince yeniden üretim ve cache yenileme yolu gerekir.

Örnek: Bir rehber sitesinin bölümleri önceden HTML olarak üretilip CDN'den sunulur.

Ne zaman kullanılır: İçerik ortak ve önceden üretilebiliyorsa; her kullanıcıya özel anlık veri için tek başına yeterli değildir.

Sık hata: Yeni içerik kaydedildiğinde eski dosyanın da hemen değiştiğini varsaymak.

Karar sorusu: Değişiklik ne zaman görünür ve başarısız yeniden üretimde ne sunulur?

029 Hydration ve seçici etkileşim: Hazır HTML'e istemci davranışı bağlamak.

Temel

Açıklama: Sunucudan gelen görünümü etkileşimli hale getirmek için istemci kodu çalışır. Yalnız gerekli alanları etkileşimli yapmak bu işi azaltabilir. React hydration.

Örnek: Uzun bir yazıda metin statik kalır; yalnız arama ve yorum alanları istemci davranışı yükler.

Ne zaman kullanılır: SSR veya statik HTML'e etkileşim eklerken; etkileşimsiz içerikte gereksiz istemci kodundan kaçınmak için.

Sık hata: Ekranda görünen düğmenin hemen çalışabildiğini varsaymak.

Karar sorusu: İlk tıklama ne zaman işleniyor; sunucu ve istemci görünümü uyuşmazsa ne oluyor?

İstemci platformu ve erişilebilirlik

030 Platform ve SDK seçimi: Native, cross-platform ve web yaklaşımının cihaz özellikleri, bakım ve bağımlılıklara etkisi.

İhtiyaca bağlı

Açıklama: Ortak kod geliştirmeyi kolaylaştırabilir; cihazın özelliklerine erişim, performans ve bakım yükü platforma göre değişir. SDK, uygulamaya eklediğin başka bir yazılım bileşenidir; boyut, izin, veri paylaşımı ve güncelleme sorumluluğu getirebilir.

Örnek: Kamera ile belge tarayan bir uygulama için yalnız ekranları paylaşmak yetmez; kamera davranışı ve cihazda işleme gereksinimi de değerlendirilir.

Ne zaman kullanılır: Yeni istemci platformu veya üçüncü taraf SDK seçerken.

Sık hata: Ortak kod oranını toplam bakım maliyetiyle aynı saymak.

Karar sorusu: Hangi cihaz yeteneğine ihtiyacımız var; SDK hangi veriyi işliyor ve güncelleme sorumluluğu kimde?

031 Deep link ve giriş sonrası niyeti korumak: Bağlantıdan doğru ekrana, gerektiğinde giriş üzerinden ulaşmak.

Temel

Açıklama: Bir bağlantı uygulamanın belirli içeriğini açar. Uygulamanın yüklü olmaması, oturumun kapalı olması, hedefin silinmesi veya eski sürüm ayrı sonuçlardır. Bağlantı nesneye erişim yetkisi vermez.

Örnek: Kampanya bağlantısına basan kullanıcı giriş yaptıktan sonra kampanyaya döner; kampanya bitmişse açıklama görür.

Ne zaman kullanılır: Bildirim, e-posta, paylaşım veya QR ile uygulama içeriğine yönlendirirken.

Sık hata: Ana ekrana düşmeyi her bağlantı için başarılı sonuç kabul etmek.

Karar sorusu: Hedef yoksa veya kullanıcı yetkisizse hangi güvenli çıkış gösterilecek?

032 Push bildirimi, izin ve teslim ayrımı: Gönderim isteği, cihaza ulaşma ve kullanıcının açması farklı olaylardır.

Temel

Açıklama: Bildirim servisine kabul edilmek, kullanıcının bildirimi gördüğünü kanıtlamaz. İzin reddi, geç teslim ve eski istemci için ürün davranışı gerekir.

Örnek: Sipariş sonucu yalnız push'a bırakılmaz; kullanıcı uygulamayı açtığında sunucudan güncel durumunu da sorgular.

Ne zaman kullanılır: Durum değişikliği veya kampanya bildirimi tasarlarken.

Sık hata: Bildirim iznini vermeyen kullanıcıya işlem sonucuna ulaşma yolu bırakmamak.

Karar sorusu: Bildirim hiç gelmezse kullanıcı sonucu nereden öğrenir?

033 Erişilebilirlik ve platform davranışı: Ekran okuyucu, büyük yazı, klavye/odak ve azaltılmış hareket gereksinimleri.

Temel

Açıklama: Bir akış yalnız standart yazı boyutu ve dokunmayla değil, farklı giriş ve algılama biçimleriyle de tamamlanabilmelidir. Hata ve yükleme durumları da bu kapsamdadır.

Örnek: Ödeme formundaki hata yalnız kırmızı kenarlıkla anlatılmaz; alanla ilişkili metinle açıklanır ve erişilebilir biçimde bildirilir.

Ne zaman kullanılır: Bileşen, form, gezinme ve hareketli arayüz tasarlarken; kabul koşullarını yazarken.

Sık hata: Otomatik taramadan geçen sayfayı tam erişilebilir saymak.

Karar sorusu: Ana iş ekran okuyucu, büyük yazı ve klavye ile tamamlanabiliyor mu?

API ve sözleşme tasarımı

Şema 02 | API sözleşmesi



Şemayı oku: Bir API sözleşmesi durum kodunu, yanıt gövdesini ve istemcinin göstereceği durumu birlikte tanımlar. Boş liste, erişim reddi ve kota aşımı farklı davranışlar gerektirir; örnekteki kontrol sırası, gerçek sunucunun tüm iç adımlarını göstermez. Yapay zekâ (Imagegen) ile üretilmiştir.

034 REST, RPC, gRPC ve GraphQL: Kaynak, operasyon ve sorgu modelleri; istemci ihtiyacına göre seçim.

İhtiyaca bağlı

Açıklama: REST kaynakları, RPC yapılacak işlemi, GraphQL istenen veri alanlarını öne çıkarır. gRPC sözleşmeli RPC için kullanılan bir teknolojidir.

Örnek: Ürün sayfası yalnız ad ve fiyat isterken yönetim paneli stok ve tedarikçi bilgilerini de isteyebilir.

Ne zaman kullanılır: İstemci ihtiyaçlarını, ekip araçlarını ve iletişim maliyetini karşılaştırarak API biçimi seçerken.

035 Sözleşme: İstek ve yanıt şemaları, doğrulama, hata modeli ve makine tarafından okunabilir API tanımı.

Temel

Açıklama: Sözleşme, iki tarafın ne gönderip ne alacağını yazılı hale getirir; hatalar da bunun parçasıdır.

Örnek: Mobil uygulama fiyatı sayı beklerken sunucu metin gönderirse ekran bozulabilir; şema bu uyuşmazlığı yakalatır.

Ne zaman kullanılır: Frontend, mobil ve backend ayrı geliştirildiğinde ya da dışarıya API açtığında.

Sık hata: 200 dönen API'nin her durumda doğru veri verdiğini sanmak; boş listede dizi yerine null dönmesi istemciyi bozabilir.

Karar sorusu: Başarı, boş veri ve 429 yanıtında gövde, başlık ve beklenen ekran davranışı sözleşmede açık mı?

036 İdempotency ve tekrar deneme: Aynı isteğin yeniden gelmesinin yan etkileri; idempotency key'in kapsamı ve saklama süresi.

Temel

Açıklama: Aynı işi tekrar istemek, aynı yan etkiyi tekrar üretmemelidir. İdempotency key tekrarların aynı iş olduğunu tanımaya yardım eder.

Örnek: Ödeme düğmesine iki kez basıldığında aynı işlem anahtarıyla yalnız bir tahsilat yapılır.

Ne zaman kullanılır: Ödeme, sipariş ve rezervasyon gibi yeniden denenen isteklerin çift işlem yaratabileceği yerlerde.

Sık hata: Aynı satın alma niyeti için her denemede yeni anahtar üretmek; K1 ve K2 iki ayrı işlem sayılabilir.

Karar sorusu: Yanıt kaybolursa aynı niyeti nasıl tanıyacağız; anahtarın süresi dolduğunda hangi kalıcı iş kaydını sorgulayacağız?

037 Liste API'leri: Filtreleme, sıralama, offset ve cursor sayfalama; kararlı sıralama ve eşzamanlı değişiklikler.

Temel

Açıklama: Büyük listeyi parçalara bölersin. Offset kaç kaydı atlayacağını, cursor kaldığın konumu belirtir; eşit sıralama değerleri için ek benzersiz anahtar gerekir.

Örnek: Yeni ilanlar eklenirken cursor kullanımı kullanıcının sonraki sayfada kayıtları atlamasını veya tekrar görmesini azaltabilir.

Ne zaman kullanılır: Büyüyen, sık değişen veya sonsuz kaydırmayla gösterilen listelerde.

038 Evrim: Geriye uyumluluk, sürümleme, alan kaldırma ve eski mobil istemcilerin desteklenmesi.

Temel

Açıklama: Sunucuyu değiştirirken eski uygulamaların beklediği davranışı korursun; her kullanıcı hemen güncelleme yapmaz.

Örnek: Telefonundaki eski sürüm 'name' alanını bekliyorsa bu alanı bir gecede kaldırmak giriş ekranını bozabilir.

Ne zaman kullanılır: API'ye alan eklerken, alan değiştirirken veya eski mobil sürümleri desteklerken.

Sık hata: Yeni bir durum veya alanı yalnız son istemci sürümüyle sınayıp eski mobil sürümlerin davranışını atlamak.

Karar sorusu: Eski istemci yeni yanıtı görünce ne yapacak; geçişin güvenli olduğunu hangi sürümlerle doğruladık?

039 Dış entegrasyonlar: Webhook imzası, tekrar teslim, sırasız olay, kota, timeout ve üçüncü taraf kesintileri.

İhtiyaca bağlı

Açıklama: Başka hizmetten gelen bildirim gecikebilir, tekrarlanabilir veya farklı sırada gelebilir. İmza, bildirimin beklenen kaynaktan geldiğini doğrulamaya yardım eder.

Örnek: Ödeme sağlayıcısı 'ödendi' bildirimini iki kez gönderse de yalnız bir sipariş hazırlanır.

Ne zaman kullanılır: Ödeme, kargo, e-posta veya başka bir sağlayıcıyı sisteme bağlarken.

Karar tablosu: offset mi cursor mı?

Açıklama: Pagination, sonucun bir kısmını döndürür. Offset kaç kaydı atlayacağını; cursor hangi sıralama anahtarından devam edeceğini belirtir. Her iki yöntemde de kararlı bir sıralama gerekir.

Seçenek	Gerçek hayat örneği ve ne zaman kullanılır?	Bedeli / ne zaman gereksiz?
Offset + limit	Küçük ve az değişen yönetim listesinde doğrudan 7. sayfaya gitmek	Büyük offset pahalılaşabilir. Araya kayıt girerse sayfalar arasında tekrar veya atlama olabilir.

Seçenek	Gerçek hayat örneği ve ne zaman kullanılır?	Bedeli / ne zaman gereksiz?
Cursor / keyset	Sürekli yeni içerik gelen akışta son görülen kayıttan devam etmek	Rastgele sayfaya atlamak zordur. Küçük bir listede bu karmaşıklık gerekmeyebilir.

Somut örnek: Yeni siparişler için sıralama created_at DESC, id DESC olsun. Son görülen ikiliyi cursor'da taşı; sonraki sorgu bu ikiliden küçük kayıtları, aynı filtre ve yetki kapsamında getirir. Eşit tarihlerde id bağlayıcıdır; uygun bileşik indeks sorgu planıyla doğrulanır.

Sık hata: Cursor'ın tek başına işlem boyunca değişmeyen bir veri fotoğrafı sağladığını sanmak. Sıralama alanı değişirse keyset'te de hareket olabilir. **Karar sorusu:** Rastgele sayfa numarası mı, kesintisiz devam mı gerekiyor; filtre değişince cursor geçersiz kılınıyor mu?

Veri modelleme ve sahiplik

Şema 03 | Sipariş veri modeli



Şemayı oku: Bir sipariş tek satın alma niyetini korur; her ödeme denemesi kendi sonucu ve sağlayıcı referansıyla izlenir. Bu tek paket örneğinde siparişe sıfır veya bir hak bağlanır. Bilinmeyen ödeme sonucu kesin başarısızlık sayılmaz. Yapay zekâ (Imagegen) ile üretilmiştir.

040 Varlıklar ve ilişkiler: Kimlik, birincil ve yabancı anahtarlar; bire bir, bire çok ve çoka çok ilişkiler.

Temel

Açıklama: Varlık sakladığın şeydir; ilişki şeylerin birbirine bağlanmasıdır. Anahtarlar bu kayıtları karıştırmadan tanımlar.

Örnek: Bir müşterinin birçok siparişi, bir siparişin birçok ürünü olabilir; sipariş kalemi aradaki ilişkiyi taşır.

Ne zaman kullanılır: Veritabanı şeması ve API veri yapıları tasarlarken.

Sık hata: İkinci ödeme denemesinde ilk denemenin sağlayıcı referansını ezmek; geç gelen sonucu doğru kayda bağlayamamak.

Karar sorusu: Tek satın alma niyeti, her ödeme denemesi ve verilen paket hakkı ayrı kimliklerle izlenebiliyor mu?

041 Veri sahipliği: Hangi bileşenin hangi verinin asıl kaynağı olduğu ve kimlerin değiştirebildiği.

Temel

Açıklama: Aynı bilginin farklı kopyaları olabilir ama hangi kaydın yetkili kaynak olduğu açık olmalıdır.

Örnek: Kargo durumu ekranda ve arama indeksinde görünür; asıl güncelleme kargo kayıtlarını yöneten bileşenden gelir.

Ne zaman kullanılır: Servisler veya cache'ler aynı veriyi tutmaya başladığında.

042 Normalizasyon ve denormalizasyon: Veri tekrarının okuma hızı, güncelleme karmaşıklığı ve doğrulukla ilişkisi.

Temel

Açıklama: Normalizasyon aynı bilgiyi gereksiz tekrar etmeden saklar. Denormalizasyon okuma kolaylığı için bazı kopyalar oluşturur; onları güncel tutmak gerekir.

Örnek: Müşteri adresini tek yerde tutarsın; geçmiş siparişte ise teslimat anındaki adresin ayrı kopyasını koruyabilirsin.

Ne zaman kullanılır: Hızlı okumayla güncelleme doğruluğu arasında denge kurarken.

043 Kısıtlar: Unique, not null, check ve referans bütünlüğü; yalnızca uygulama kontrolüne güvenmenin sınırları.

Temel

Açıklama: Veritabanı kısıtları, uygulama hata yapsa bile belirli yanlış kayıtları reddeder.

Örnek: İki eşzamanlı kayıt isteği aynı kullanıcı adını almaya çalışır; unique kısıtı yalnız birine izin verir.

Ne zaman kullanılır: Benzersizlik, zorunlu alan veya kayıtlar arası bağlılık her zaman korunmalıysa.

044 Durum makineleri: Sipariş, ödeme ve arka plan işi gibi kayıtların izin verilen geçişleri.

Temel

Açıklama: Durum makinesi bir kaydın hangi aşamadan hangisine geçebileceğini tanımlar.

Örnek: Sipariş 'hazırlanıyor'dan 'kargoda'ya geçebilir; iptal edilmiş siparişi yanlışlıkla 'teslim edildi' yapmazsın.

Ne zaman kullanılır: Ödeme, teslimat, onay ve arka plan işi gibi çok aşamalı akışlarda.

Sık hata: Timeout'u kesin başarısızlık sayıp “Yeniden öde” göstermek; ilk ödeme de tamamlanırsa çift işlem oluşabilir.

Karar sorusu: Sonucu bilinmeyen durumda hangi kayıt korunacak, nasıl sorgulanacak ve kullanıcı hangi güvenli eylemi yapabilecek?

045 Veri anlamı ve yaşam döngüsü: Para hassasiyeti, birimler, null anlamı, kimlik üretimi, silme, arşiv ve saklama süresi.

Temel

Açıklama: Bir değerin anlamı, biçimi kadar önemlidir. Bilinmeyen tutar sıfır değildir; silinmiş kayıt ile arşivlenmiş kayıt aynı değildir.

Örnek: Fiyatı para birimiyle birlikte saklar, parasal hesap için uygun kesin sayı türü kullanırsın.

Ne zaman kullanılır: Finansal tutar, ölçüm, tarih veya silme gerektiren veri tasarlarken.

Sık hata: Kampanya bitişini cihaz, sunucu ve raporda farklı saat dilimleriyle yorumlamak; sıfır indirimi bilinmeyen indirimle karıştırmak.

Karar sorusu: Para birimi, zaman bölgesi, dahil veya hariç bitiş anı ve null anlamı tüm tüketicilerde aynı mı?

Uygulama mimarisi

Şema 04 | Monolit ve mikroservisler



Şemayı oku: Modüler monolitte iş sınırları tek dağıtım içinde korunabilir. Mikroservisler bağımsız dağıtım sağlayabilir; ağ çağrıları, kısmi arıza ve ayrı işletim sorumluluğu ekler. İki taraf arasında herkes için geçerli bir üstünlük sırası yoktur. Yapay zekâ (Imagegen) ile üretilmiştir.

046 Monolit, modüler monolit ve mikroservisler: Dağıtım bağımsızlığı, ekip sınırları ve işletim maliyeti.

Temel

Açıklama: Monolit birlikte dağıtılır; modüler monolit içeride net bölümlere ayrılır. Mikroservisler ayrı dağıtılabilir ama ağ, takip ve işletim yükü getirir.

Örnek: Küçük bir rezervasyon ürünü tek uygulamayla başlayabilir; ayrı ekiplerin yönettiği bağımsız alanlar zamanla ayrılabilir.

Ne zaman kullanılır: Uygulamanın sınırlarını belirlerken; yalnızca 'modern' olduğu için servis sayısını artırmadan önce.

Ne zaman kullanmamalıyım: Bağımsız dağıtım, ekip veya ölçek ihtiyacı belirgin değilse mikroservisleri varsayılan yapma. Küçük ekibin modüler monolit ile koruyabildiği sınırlar için ek ağ ve işletim yükü gerekmeyebilir.

047 Modül sınırları: Cohesion, coupling, bounded context ve iş kurallarının nerede yaşadığı.

Temel

Açıklama: Birlikte değişen işleri aynı modülde toplar, modüllerin birbirine gereksiz bağımlılığını azaltırsın. Aynı kelime farklı alanlarda farklı anlam taşıyabilir.

Örnek: 'Müşteri', faturalamada ödeme bilgileriyle; destek bölümünde talepleriyle önemlidir.

Ne zaman kullanılır: Her küçük değişiklik birçok dosyayı veya ekibi etkiliyorsa.

048 Katmanlı ve hexagonal mimari: İş mantığını arayüz, veritabanı ve dış servislerden ayırmak.

İhtiyaca bağlı

Açıklama: İş kuralını ekran, veritabanı ve servis araçlarından ayırırsın. Araçlar değişse de kuralın aynı kalması kolaylaşır.

Örnek: İndirim hesaplama kuralı hem webde hem mobilde aynı sonucu verir; ödeme sağlayıcısı değişince yeniden yazılmaz.

Ne zaman kullanılır: İş mantığını test etmek, farklı arayüzlere taşımak veya bir sağlayıcıyı değiştirmek istediğinde.

Ne zaman kullanmamalıyım: İş kuralları ve dış bağımlılıklar basitken her küçük CRUD işlemini çok sayıda katmana bölme; sınırların test ve değişiklik maliyetini düşürüp düşürmediğine bak.

049 Stateless ve stateful servisler: Oturum ve iş durumunun konumu; yeniden başlatma ve yatay büyüme davranışı.

Temel

Açıklama: Stateless çalışan, sonraki isteği yanıtlamak için kendi belleğinde önceki kullanıcı durumuna ihtiyaç duymaz. Stateful çalışanın taşıdığı durumun kaybını ve paylaşımını yönetmen gerekir.

Örnek: Sepeti tek sunucunun belleğinde tutarsan sonraki istek başka sunucuya gidince sepet kaybolabilir.

Ne zaman kullanılır: Birden fazla sunucuya çıkarken veya yeniden başlatmalarda kullanıcı durumunu korurken.

050 Senkron ve asenkron iletişim: Hemen sonuç gerektiren işler ile sonradan tamamlanabilecek işler.

Temel

Açıklama: Senkron çağrıda sonucu beklersin; asenkron işte talebi kaydedip sonuç hazır olduğunda ilerlersin.

Örnek: Sipariş numarasını hemen gösterir, fatura PDF'ini arka planda üretip sonra indirilebilir yaparsın.

Ne zaman kullanılır: Uzun süren işler kullanıcıyı bekletiyorsa veya bağımlılıklar geçici olarak erişilemiyorsa.

051 Mimarinin evrimi: Servis ayırma gerekçesi, strangler yaklaşımı ve geri alınabilir geçişler.

İhtiyaca bağlı

Açıklama: Büyük sistemi tek seferde değiştirmek yerine seçili akışları yeni yapıya geçirirsin. Strangler yaklaşımı eski sistemin görevlerini aşamalı devralır.

Örnek: Önce yalnız raporlama trafiğini yeni servise yönlendirir, sorun olursa eski yola dönebilirsin.

Ne zaman kullanılır: Eski uygulamayı hizmeti kesmeden yenilerken veya bir modülü ayırırken.

Ne zaman kullanmamalıyım: Değiştirme gerekçesi ölçülmemişken çalışan sistemi topluca yeniden yazma. Geçiş ve geri dönüş yolu olmayan servis ayırmayı başlatma.

052 BFF ve veri birleştirme: Arayüzün ihtiyacına özel backend yanıtı üretmek.

İhtiyaca bağlı

Açıklama: Backend for Frontend, ekranın ihtiyacına göre servis sonuçlarını toplar ve sadeleştirir. Yetki her veri kaynağında korunur; ek ağ adımı ve işletim bedeli vardır. BFF.

Örnek: Mobil ana ekranın profil, sipariş ve kampanya bilgisi tek uygun yanıtta toplanır; kampanya servisi kesilirse profil yine gösterilir.

Ne zaman kullanılır: Web ve mobilin ihtiyaçları ayrışıyorsa; aynı veriyi isteyen tek istemcide ek katman gereksiz olabilir.

Sık hata: Birleştirme katmanına bütün iş kurallarını ve yetkiyi taşımak.

Karar sorusu: Hangi alt veri zorunlu, hangisi eksik gelebilir ve toplam deadline nasıl paylaşılır?

Ne zaman kullanmamalıyım: İstemcilerin veri ihtiyacı aynıysa her arayüze ayrı BFF açma. Birleştirme katmanının iş kurallarını kopyalamasına ve bağımlılık hatalarını gizlemesine izin verme.

Üretimde çalışan sistemler

Depolama ve veritabanı seçimi

053 İlişkisel, key-value, doküman, geniş sütun, graph ve zaman serisi veritabanları: Erişim örüntüsüne göre seçim.

İhtiyaca bağlı

Açıklama: Veritabanını verinin adına göre değil, hangi soruları sık soracağına göre seçersin. İlişkisel yapı bağlantılı kayıtları, graph yapı ilişki yollarını, zaman serisi yapı zamana bağlı ölçümleri öne çıkarır.

Örnek: Sipariş ve ödeme kayıtlarında ilişkisel veritabanı; cihaz sıcaklıklarının zaman içindeki değişiminde zaman serisi yaklaşımı değerlendirilebilir.

Ne zaman kullanılır: Erişim örüntüsü, doğruluk ihtiyacı ve veri büyüklüğü netleştiğinde.

054 Dosya, block ve object storage: Dosya sistemi, disk hacmi ve nesne deposunun farklı kullanım alanları.

Temel

Açıklama: Dosya deposu klasör ve dosya sunar; block storage makineye disk gibi bağlanır; object storage dosyayı anahtarıyla saklayıp getirir.

Örnek: Veritabanı disk hacminde çalışırken kullanıcının yüklediği profil fotoğrafını object storage'da tutabilirsin.

Ne zaman kullanılır: Kalıcı dosya, sunucu diski veya ortak dosya erişimi tasarlarken.

055 İndeksler: B-tree, hash, bileşik ve kısmi indeksler; seçicilik, alan sırası ve yazma maliyeti.

Temel

Açıklama: İndeks, kitabın dizini gibi aranan kayda daha az bakarak ulaşmayı sağlar. Bileşik indeks birkaç alanı, kısmi indeks yalnız seçili kayıtları kapsar; her indeks yazmayı da etkiler.

Örnek: Müşterinin son siparişlerini sık getiriyorsan müşteri kimliği ve tarihe uygun indeks değerlendirirsin.

Ne zaman kullanılır: Ölçümde yavaş sorgu gördüğünde; her sütuna otomatik indeks eklemek yerine.

056 Sorgu yürütme: EXPLAIN, tablo taraması, join, sıralama, N+1 ve bağlantı havuzu sınırları.

Temel

Açıklama: Sorgunun ne istediği kadar veritabanının onu nasıl bulduğu önemlidir. EXPLAIN yürütme planını, yani seçilen erişim yolunu gösterir.

Örnek: Yalnız son on sipariş gerekirken milyonlarca satır taranıp sıralanması plan üzerinden fark edilebilir.

Ne zaman kullanılır: Veritabanı gecikmesi veya yükü yükseldiğinde, sorgu değişikliğini doğrularken.

057 Depolama motoru kavramları: B-tree ve LSM-tree, WAL, compaction, okuma ve yazma büyütmesi.

İleri

Açıklama: Depolama motoru veriyi diske yerleştirir. B-tree ve LSM-tree farklı düzenlerdir; WAL kurtarma günlüğüdür; compaction parçaları yeniden düzenler ve ek disk işi yaratır.

Örnek: Yoğun sensör kayıtlarını yazan sistemde arka plandaki veri birleştirme işi disk kullanımını artırabilir.

Ne zaman kullanılır: Çok yüksek okuma veya yazma yükünde motor seçimi ve disk darboğazını incelerken.

058 Büyük dosyalar: İmzalı yükleme adresi, multipart upload, bütünlük kontrolü, medya işleme ve erişim yetkisi.

İhtiyaca bağlı

Açıklama: Büyük dosyayı parçalara ayırarak yüklemek kesinti sonrası tümünü yeniden göndermeyi önler. İmzalı adres belirli yüklemeye sınırlı erişim verir.

Örnek: Uzun bir video yüklerken bağlantı koparsa tamamlanmış parçaları koruyup eksikleri yeniden gönderirsin.

Ne zaman kullanılır: Video, arşiv veya büyük belge yüklemelerinde; yükleme yetkisi ve dosya doğrulaması gerektiğinde.

Transaction ve eşzamanlı veri erişimi

Şema 05 | Transaction ve yarış koşulu



Şemayı oku: Son bir ürün için iki istek yarışır. Koşullu stok azaltma ve rezervasyon aynı transaction içinde korunursa bir istek başarılı olur, diğeri stok kalmadığını görür. Burada A'nın kazanması yalnızca örnektir. Yapay zekâ (Imagegen) ile üretilmiştir.

059 ACID: Atomiklik, veri kurallarının korunması, izolasyon ve kalıcılığın ayrı garantileri.

Temel

Açıklama: Transaction birden çok veri adımını birlikte yönetir: Ya tamamı uygulanır ya uygulanmaz; eşzamanlı işlemler seçilen izolasyona uyar; tamamlanan kayıt kalıcılık garantisine tabidir.

Örnek: Hesaplar arası aktarımda bir bakiyeyi azaltıp diğerini artırmak aynı veritabanı transaction'ında tutulur.

Ne zaman kullanılır: Birden çok değişiklik birlikte doğru kalmak zorundaysa.

060 İzolasyon seviyeleri: Read committed, repeatable read ve serializable; davranışın veritabanına göre değişmesi.

Temel

Açıklama: İzolasyon, aynı anda çalışan işlemlerin birbirinin değişikliklerini nasıl göreceğini belirler. Serializable sonuçları sırayla çalışmış gibi tutar; çatışmada işlem yeniden denenebilir.

Örnek: İki görevli aynı son odayı ayırmaya çalıştığında seçilen izolasyon ve kısıtlar çift rezervasyonu engellemelidir.

Ne zaman kullanılır: Ortak kayıtlar eşzamanlı değişiyorsa; doğrulukla bekleme ve tekrar maliyetini dengelemek için.

061 Yarış koşulları: Lost update, write skew ve check-then-act hataları; stok ve rezervasyon örnekleri.

Temel

Açıklama: Yarış koşulu, işlerin zamanlaması yüzünden tek başına doğru görünen adımların birlikte yanlış sonuç üretmesidir.

Örnek: İki istek aynı stok değerini bir görür, ikisi de satışa izin verir ve aynı ürünü satar.

Ne zaman kullanılır: Önce kontrol edip sonra değiştiren her işlemde; stok, kota ve bakiye gibi alanlarda.

062 Kilitleme: Optimistic ve pessimistic concurrency, sürüm alanı, compare-and-swap ve satır kilidi.

Temel

Açıklama: İyimser kontrol kaydın sürümü değiştiyse yazmayı reddeder. Kötümser kilit diğer yazanları bekletir; compare-and-swap beklenen değer hâlâ geçerliyse değiştirir.

Örnek: İki kişi aynı yazıyı düzenlerken ikinci kayıt ilk kişinin değişikliğini sessizce ezmek yerine çatışma gösterir.

Ne zaman kullanılır: Aynı kayıt birden çok yerden değişiyorsa; çatışma sıklığına göre yöntem seçerken.

063 MVCC ve deadlock: Aynı veriye eşzamanlı erişim; kilit beklemeleri ve güvenli transaction tekrarları.

İhtiyaca bağlı

Açıklama: MVCC kayıtların farklı sürümlerini kullanarak eşzamanlı erişimi yönetir. Deadlock ise iki işlemin birbirinin tuttuğu kilidi beklemesidir; biri sonlandırılıp güvenli şekilde yeniden denenebilir.

Örnek: İki aktarım hesapları ters sırayla kilitlerse birbirini bekleyebilir; tutarlı kilit sırası riski azaltır.

Ne zaman kullanılır: Kilit bekleme, transaction hataları veya eşzamanlılık sorunlarını incelerken.

064 Transaction sınırı: Atomik tutulması gereken işler; uzun transaction ve ağ çağrılarını transaction içinde yapmanın bedeli.

Temel

Açıklama: Transaction'ı doğru kalması gereken veri adımlarıyla sınırlandırırsın; uzun bekleme kilit ve bağlantıları meşgul eder.

Örnek: Siparişi kaydederken e-posta sağlayıcısının yanıtını veritabanı transaction'ı açıkken beklemek diğer işleri geciktirebilir.

Ne zaman kullanılır: Veri yazımıyla ağ çağrısını aynı akışta birleştirirken.

Başvuru: PostgreSQL eşzamanlılık kontrolü.

Karar tablosu: optimistic mi pessimistic locking mi?

Açıklama: İkisi de aynı kayda eşzamanlı değişiklikleri yönetir. Optimistic yaklaşım çakışmayı yazarken yakalar; pessimistic yaklaşım kritik bölgeyi kilitleyerek diğer işlemleri bekletir.

Seçenek	Gerçek hayat örneği ve ne zaman kullanılır?	Bedeli / ne zaman gereksiz?
Optimistic locking	Profil düzenleme gibi çakışması seyrek işlemlerde sürüm numarasıyla güncellemek	Çakışmada yeniden okuma, birleştirme veya kullanıcı kararı gerekir. Çok çekişmeli kayıtta retry maliyeti artar.
Pessimistic locking	Aynı rezervasyonun birkaç alanını birlikte kontrol edip değiştiren kısa transaction'da satır kilidi almak	Bekleme, deadlock ve süre aşımı olabilir. Uzun kullanıcı/harici API bekleyişini kilit altında tutmak tehlikelidir.

Somut örnek: UPDATE profile SET name = ..., version = version + 1 WHERE id = ... AND version = 7 sıfır satır değiştirirse eski veriyle yazma çatışması vardır. Son ürün için UPDATE stock SET available = available - 1 WHERE sku = ... AND available > 0 gibi atomik koşullu güncelleme tek başına yeterli olabilir; her soruna ayrıca kilit eklemek gerekmez.

Sık hata: Tekilleştirme veya transaction izolasyonu ihtiyacını yalnız uygulama kilidiyle çözdüğünü sanmak. **Karar sorusu:** Hangi değişmez koşulu koruyoruz; çakışmada beklemek mi, tekrar denemek mi daha kabul edilebilir? PostgreSQL eşzamanlılık ve izolasyon.

Önbellekleme

Şema 06 | Cache



Şemayı oku: Geçerli kopya varsa **hit**, yoksa **miss** olur. Miss yolunda uygulama veritabanından okur ve sonraki istek için cache'i doldurur. Bu, cache-aside yaklaşımıdır. Yapay zekâ (Imagegen) ile üretilmiştir.

Akılda tut: Cache asıl kaynağın yerine geçen kalıcı kayıt değildir. Ürün fiyatı değişirse eski kopyanın ne zaman ve nasıl yenileneceğini belirlemelisin.

065 Önbellek katmanları: Tarayıcı, CDN, uygulama, dağıtık cache ve veritabanı önbelleği.

Temel

Açıklama: Sık istenen verinin kopyasını daha yakın veya hızlı bir yerde tutarsın. Her katmanın kime ait olduğu ve ne kadar güncel kaldığı farklıdır.

Örnek: Logo tarayıcıda, ürün fotoğrafı CDN'de, popüler ürün bilgisi uygulama cache'inde bulunabilir.

Ne zaman kullanılır: Aynı veri tekrar tekrar okunuyor ve yeniden üretmek pahalıysa.

066 Cache-aside, read-through, write-through ve write-behind: Okuma, yazma ve hata akışları.

İhtiyaca bağlı

Açıklama: Cache-aside'da uygulama eksik veriyi depodan alıp cache'e koyar; read-through bunu aracıya yaptırır. Write-through yazıyı depoya da iletir; write-behind kalıcı yazıyı erteler.

Örnek: Ürün bilgisini ilk istekte veritabanından okuyup sonraki ziyaretçiler için cache'e koyarsın.

Ne zaman kullanılır: Okuma ve yazma akışını kurarken; ertelenen yazıda veri kaybı riskini kabul edip edemeyeceğini değerlendirirken.

Ne zaman kullanmamalıyım: Kalıcı yazının hemen kesinleşmesi gereken kayıtta veri kaybı toleransı tanımlamadan write-behind kullanma; her erişime cache eklemeyi varsayılan sayma.

067 TTL ve invalidation: Verinin ne kadar eski olabileceği; güncelleme ve silmenin önbelleğe yansıması.

Temel

Açıklama: TTL kopyanın geçerlilik süresidir; invalidation veri değişince o kopyayı kaldırmak veya yenilemektir.

Örnek: Restoran fiyatı değiştiğinde eski menüyü saatlerce göstermemek için ilgili cache kaydı yenilenir.

Ne zaman kullanılır: Veri önbelleğe alınırken; özellikle fiyat, stok veya kullanıcıya görünen güncel bilgi için.

Sık hata: Listede 100 TL kalan ürünün ödeme anında 120 TL olduğunu kullanıcıya göstermeden tahsilata devam etmek.

Karar sorusu: Fiyat ne kadar eski gösterilebilir; ödeme öncesi hangi kaynaktan doğrulanır ve fark varsa yeniden onay nasıl alınır?

068 Cache key: Kullanıcı, tenant, dil, yetki ve sürümün anahtara etkisi; kullanıcılar arası veri sızıntısını önlemek.

Temel

Açıklama: Cache anahtarı hangi yanıtın kime ve hangi koşula ait olduğunu ayırır. Eksik anahtar yanlış kişiye yanlış veri döndürebilir.

Örnek: '/rapor' anahtarı yerine şirket kimliğini de içeren anahtar kullanarak iki müşterinin raporunu ayırırsın.

Ne zaman kullanılır: Kişiselleştirilmiş, çok dilli veya birden çok kuruma hizmet veren uygulamalarda cache kurarken.

Sık hata: Hesap değişirken kullanıcıdan bağımsız anahtarla tutulan profil verisini yeni hesaba göstermek.

Karar sorusu: Anahtar ve invalidation kuralı hesap, tenant, yetki ve sürüm sınırlarını gerçekten koruyor mu?

069 Cache stampede, hot key ve eviction: Eşzamanlı yenileme, dengesiz yük, bellek sınırı ve kontrollü bayat veri sunumu.

İhtiyaca bağlı

Açıklama: Çok kişi aynı anda süresi dolan kaydı isterse kaynağa yük yığılır. Hot key tek kayda aşırı erişimdir; eviction dolan cache'den kayıt çıkarmaktır.

Örnek: Popüler maç skoru yenilenirken her ziyaretçinin ayrı sorgu başlatması yerine yenilemeyi bir çalışan üstlenir.

Ne zaman kullanılır: Popüler içerikte ani trafik, cache kapasitesi dolması veya yenileme anında yavaşlama varsa.

070 HTTP cache semantiği: Cache-Control, ETag, Vary, koşullu istekler ve kişisel yanıtların paylaşılmaması.

Temel

Açıklama: HTTP başlıkları yanıtın kim tarafından ne kadar saklanacağını söyler. ETag içerik değişmediğinde yeniden tam indirmeyi önler; Vary yanıtın hangi istek alanlarına göre değiştiğini belirtir.

Örnek: Tarayıcı değişmeyen dosya için koşullu istek yapar; sunucu içerik aynıysa dosyanın tamamını tekrar göndermez.

Ne zaman kullanılır: Tarayıcı ve CDN önbelleğini yönetirken; kişisel yanıtları ortak cache'den ayırırken.

Ölçekleme ve trafik yönetimi

Şema 07 | Replikasyon



Şemayı oku: Yazı liderde yapılır, değişiklikler asenkron olarak okuma kopyalarına taşınır. Aynı kayıt kümesi hedeflenir; örnekte liderdeki 42 numaralı kaydın sürümü 8 iken A kopyası hâlâ 7'dedir. Kendi yazını hemen görmesi gereken akışın okuma yönlendirmesi ayrıca belirlenir. Yapay zekâ (Imagegen) ile üretilmiştir.

Şema 08 | Sharding



Şemayı oku: Kayıtlar müşteri kimliğine göre farklı parçalara ayrılır. 1420 numaralı müşteri için etkin mavi yol B'ye gider; kesik gri yollar diğer anahtarların olası yönlerini gösterir. Aralıklar temsilidir; gerçek dağılım yük, sorgular ve yeniden dengeleme ihtiyaçlarına göre seçilir. Yapay zekâ (Imagegen) ile üretilmiştir.

Temel fark: Replikasyon aynı verinin kopyalarını, sharding farklı veri parçalarını oluşturur. Birlikte kullanılabilirler: A, B ve C'nin her birinin ayrıca kendi replikaları olabilir. Tüm müşterileri kapsayan bir rapor ise birden fazla shard'dan veri toplamayı gerektirebilir.

071 Dikey ve yatay ölçekleme: Kaynak büyütme, örnek sayısını artırma ve gerçek darboğazın konumu.

Temel

Açıklama: Dikey ölçekleme daha güçlü makine, yatay ölçekleme daha çok çalışan makine demektir. Ortak darboğaz yerinde kalırsa artış fayda vermeyebilir.

Örnek: Uygulama sunucularını çoğaltmak tek veritabanındaki yavaş sorguyu kendiliğinden hızlandırmaz.

Ne zaman kullanılır: Ölçülen kapasite sınırına yaklaşırken hangi parçanın büyütüleceğini belirlemek için.

072 Load balancing: Round-robin, least connections, health check, connection draining ve sticky session.

İhtiyaca bağlı

Açıklama: Yük dengeleyici istekleri çalışan sunuculara dağıtır. Health check arızalı olanı dışlar; draining kapatılacak sunucunun mevcut işlerini bitirmesini sağlar.

Örnek: Yeni sürüm yüklenen sunucu yeni sipariş almayı bırakır ama başlamış isteği tamamlar.

Ne zaman kullanılır: Birden fazla sunucu çalıştırırken veya kesintiyi azaltarak sürüm değiştirirken.

073 Replikasyon ve sharding: Aynı veriyi kopyalamak ile veriyi parçalara bölmek arasındaki fark.

İhtiyaca bağlı

Açıklama: Replikasyon verinin kopyalarını, sharding ise her biri verinin bir bölümünü taşıyan parçaları oluşturur. Kopya okuma ve dayanıklılık, parçalama kapasite için değerlendirilir.

Örnek: Mağazanın aynı kataloğunu iki kopyadan okutmak replikasyon; müşteri kayıtlarını kimliğe göre farklı depolara bölmek sharding'dir.

Ne zaman kullanılır: Okuma yükünü dağıtırken veya veri ve yazma yükü tek düğümü aşıyorsa.

074 Partition key: Dengesiz dağılım, hot partition, yeniden dengeleme ve shard'lar arası sorgular.

İleri

Açıklama: Bölme anahtarı kaydın hangi parçaya gideceğini belirler. Dengesiz anahtar bir parçayı aşırı yükler; iyi dağılım ve birlikte sorgulanacak veriler arasında denge kurarsın.

Örnek: Tüm bugünkü siparişleri aynı parçaya koymak o parçayı yoğunlaştırabilir.

Ne zaman kullanılır: Sharding, event log bölümleri veya bölümlenmiş depolama tasarlarken.

Ne zaman kullanmamalıyım: Kararlı bir dağıtım anahtarı ve yeniden dengeleme planı yoksa sharding ile başlama. Tek veritabanındaki sorgu ve indeks sorunlarını önce ölç.

075 Rate limiting ve kota: Token bucket, sliding window, kullanıcı ve tenant bazında adalet.

Temel

Açıklama: Rate limit kısa sürede izin verilen işlem hızını, kota daha geniş dönemde toplam hakkı sınırlar. Token bucket kısa patlamalara kontrollü izin verebilir.

Örnek: Bir hesap arama API'sini sürekli çağırıp diğer müşterileri yavaşlatmasın diye hesap başına sınır uygularsın.

Ne zaman kullanılır: Ortak kapasiteyi korumak, maliyeti sınırlamak veya kötüye kullanımı azaltmak için.

076 Autoscaling ve kuyruklanma: Ölçekleme sinyali, gecikme, kapasite payı; kararlı durumda Little yasası ile eşzamanlı iş hesabı.

İhtiyaca bağlı

Açıklama: Autoscaling ölçüme göre çalışan sayısını değiştirir ama yeni kapasite anında hazır olmaz. Kararlı sistemde Little yasası: ortalama sistemdeki iş = tamamlanma hızı × ortalama kalış süresi.

Örnek: Saniyede 200 iş tamamlanıyor ve iş başına ortalama kalış 0,3 saniyeyse sistemde ortalama 60 iş vardır.

Ne zaman kullanılır: Kuyruk, eşzamanlılık ve ölçekleme eşiği hesaplarken; sistem sürekli birikiyorsa bu basit kararlı durum hesabına güvenmeden.

Mesajlaşma ve arka plan işleri

Şema 09 | Kuyruk



Şemayı oku: API işi kaydedip kuyruğa aldıktan sonra kullanıcıya **202 - iş alındı** yanıtını verir. Worker kalıcı sonucu ve iş durumunu kaydeder, sonra tamamlandı onayı gönderir. ACK öncesi çökmede iş yeniden teslim edilebilir; aynı yan etkiyi iki kez üretmemek için idempotent işlem gerekir. Yapay zekâ (Imagegen) ile üretilmiştir.

Akılda tut: Bu örnekte onay alınamayan iş yeniden teslim edilebilir. Rapor kaydedilip onaydan önce bağlantı koparsa aynı iş tekrar gelir; iş kimliğiyle aynı sonucun yeniden kullanılabilmesi gerekir.

077 Queue, pub/sub ve event log: İş dağıtımı, birden çok abone ve geçmişi yeniden oynatma farkları.

Temel

Açıklama: Queue işi çalışanlardan birine dağıtır; pub/sub aynı olayı farklı abonelere iletir; event log olayları daha sonra yeniden okumak için saklar.

Örnek: Sipariş olayı stok, e-posta ve analitik tarafını ayrı ayrı tetikleyebilir.

Ne zaman kullanılır: İşleri ayırmak, bir olaya birçok tepki vermek veya olay geçmişinden yeniden hesap yapmak istediğinde.

078 Teslim semantiği: At-most-once, at-least-once ve exactly-once iddialarının geçerli olduğu sınırlar.

Temel

Açıklama: At-most-once kayıp pahasına tekrarı sınırlar; at-least-once tekrar ihtimaliyle yeniden teslim eder. Exactly-once garantisi belirli işlem sınırları içindir, dış yan etkileri otomatik kapsamaz.

Örnek: Bir mesaj sistemi kendi içinde tek sonuç yazsa bile dış e-posta servisine iki çağrı gitmesini ayrıca önlemelisin.

Ne zaman kullanılır: Mesaj altyapısı seçerken ve tekrarın para, bildirim veya stok etkisini tasarlarken.

079 Consumer tasarımı: Acknowledgement, visibility timeout, offset ve işlem tamamlanmadan onaylamanın riski.

Temel

Açıklama: Consumer işi alan çalışandır. Acknowledgement bitti bildirimi, offset okuma konumu, visibility timeout bazı kuyruklarda işin başkalarından geçici gizlendiği süredir.

Örnek: Fatura oluşmadan işi tamamlandı diye onaylayan çalışan çökerse fatura hiç üretilmeyebilir.

Ne zaman kullanılır: Kuyruk işlerinin kaybolmadan ve kontrollü tekrarlarla tamamlanmasını sağlarken.

080 Tekrarlı ve sırasız mesajlar: İdempotent consumer, deduplication ve anahtar bazında sıra garantisi.

Temel

Açıklama: Aynı mesajın tekrar gelmesi veya yeni olayın eskisinden önce ulaşması mümkündür. İşlem kimliği ve kayıt sürümü yanlış yan etkiyi önlemeye yardım eder.

Örnek: 'Kargoya verildi' olayından sonra gecikmiş 'hazırlanıyor' olayı gelince sipariş durumunu geriye düşürmezsin.

Ne zaman kullanılır: Webhook, kuyruk ve çevrimdışı senkronizasyon tüketirken.

081 İş yaşam döngüsü: Retry, backoff, jitter, dead-letter queue, zaman aşımı, iptal ve takılı kalan işlerin kurtarılması.

Temel

Açıklama: İşin bekliyor, çalışıyor, başarısız ve tamamlandı durumlarını yönetirsin. Sürekli başarısız işi ayrı kuyruğa almak diğer işlerin ilerlemesini sağlar.

Örnek: Geçici olarak açılamayan PDF üretimi yeniden denenir; bozuk belge sürekli döndürülmek yerine inceleme kuyruğuna alınır.

Ne zaman kullanılır: Arka plan işi, zamanlanmış görev veya uzun süren dış entegrasyon çalıştırırken.

Sık hata: 202 yanıtını “Bildirimler gönderildi” diye göstermek; tüketici durmuşsa iş yalnız kabul edilmiş olabilir.

Karar sorusu: Kuyrukta, işleniyor, sağlayıcıya iletildi ve tamamlandı durumlarının kanıtı nedir; takılan işin sahibi kim?

082 İşletim: Consumer lag, backpressure, poison message, zamanlanmış işler ve replay sırasında yan etkiler.

İhtiyaca bağlı

Açıklama: Consumer lag işleyicinin ne kadar geride kaldığını gösterir. Backpressure yeni işi yavaşlatır; poison message sürekli hata üreten kayıttır.

Örnek: İçe aktarılan tek bozuk dosya tüm sırayı tıkamamalı; geçmiş işleri tekrar oynatmak müşteriye tekrar e-posta göndermemelidir.

Ne zaman kullanılır: Kuyruk birikmesini, zamanlanmış işleri ve geçmiş olayları yeniden işlemeyi yönetirken.

Başvuru: Apache Kafka tasarımı ve teslim garantileri.

Dayanıklılık ve hata yönetimi

Şema 10 | Belirsiz ödeme ve idempotency



Şemayı oku: Sağlayıcı ödemeyi işlemiş olsa bile yanıt kaybolabilir. Sipariş ve niyet kimliği korunur; doğrulanmış webhook veya durum sorgusu belirsizliği giderir. Paket hakkı tek kez verilir; güvenli tekrarın sınırları sağlayıcının idempotency sözleşmesine bağlıdır. Yapay zekâ (Imagegen) ile üretilmiştir.

083 Kısmi arıza: Bir servisin çalışırken bağımlılığının yavaşlaması; timeout'un işlemin gerçekleşmediğini kanıtlamaması.

Temel

Açıklama: Dağıtık sistemde bir parça bozulurken diğerleri çalışabilir. Yanıt alamamak işin yapılmadığı anlamına gelmez.

Örnek: Banka tahsilatı tamamlar ama bağlantı kopar; uygulama sonucu görmez. Yeniden tahsilat yerine işlem durumunu ve anahtarını kontrol edersin.

Ne zaman kullanılır: Ağ üzerinden yan etki yaratan her işlemde.

Sık hata: İstek zaman aşımına uğradığında sağlayıcının işlemi hiç yapmadığını varsayarak kör tekrar göndermek.

Karar sorusu: Sonucu bilmiyorsak önce neyi sorgulayacağız ve güvenli tekrar için hangi garantiye ihtiyacımız var?

084 Timeout ve deadline: Bağlantı, okuma ve toplam süre bütçesi; zincir boyunca kalan sürenin aktarılması.

Temel

Açıklama: Timeout tek bir bekleme sınırıdır; deadline bütün işin bitmesi gereken son andır. Alt çağrıların toplam bütçeyi tüketmesine izin vermezsin.

Örnek: Kargo seçeneklerini getiren bir alt servis yavaşsa tüm ödeme ekranını süresiz bekletmezsin.

Ne zaman kullanılır: Bir isteğin birden çok servis çağrısı yaptığı akışlarda.

085 Retry politikası: Tekrarlanabilir hatalar, üst sınır, backoff, jitter ve retry storm.

Temel

Açıklama: Retry geçici hataya ikinci şans verir. Backoff beklemeyi artırır, jitter bekleme zamanını dağıtır; üst sınır sonsuz tekrarları önler.

Örnek: Kesinti sonrası binlerce telefon aynı milisaniyede tekrar bağlanmak yerine farklı aralıklarla dener.

Ne zaman kullanılır: Geçici ağ veya kapasite hatalarında; yan etkili işlemlerde güvenli tekrar tasarımıyla birlikte.

086 Circuit breaker: Sürekli hata veren bağımlılığa çağrıyı kontrollü kesmek.

İhtiyaca bağlı

Açıklama: Closed durumda çağrılar geçer; hata politikası tetiklenince open olur. Beklemeden sonra half-open sınırlı denemelerle toparlanmayı sınar. Circuit breaker.

Örnek: Ödeme sağlayıcısı yavaşken yeni çağrılar hızla reddedilir; daha önce sonucu belirsiz kalan ödemeler ayrıca uzlaştırılır.

Ne zaman kullanılır: Bağımlılık hatası kaynak tüketimini büyütüyorsa; yerel bir doğrulama hatasına çözüm değildir.

Sık hata: Devre açılmasını devam eden ödemenin iptali saymak.

Karar sorusu: Devreyi hangi sinyal açar, hangi sınırlı deneme kapatır?

Ne zaman kullanmamalıyım: Kısa geçici hatalara veya yanlış timeout ayarına doğrudan circuit breaker ekleme; eşik, yeniden deneme ve geri açılma davranışı anlaşılmalı.

087 Bulkhead: İşleri ayrı kaynak havuzlarına ayırarak arızayı sınırlamak.

İhtiyaca bağlı

Açıklama: Thread, bağlantı veya worker bütçeleri ayrılır; bir havuzun dolması diğerini tüketmez. Ayrım kapasitenin bir kısmını boş bırakabilir. Bulkhead.

Örnek: Rapor işleri kendi havuzunu doldururken sipariş alma için ayrılmış çalışanlar hizmet verir.

Ne zaman kullanılır: Bağımsız işlerin birbirini boğduğu durumda; düşük riskli küçük uygulamada aşırı bölme gereksiz olabilir.

Sık hata: İki servisin aynı sınırsız bağlantı havuzunu kullanmasını izolasyon sanmak.

Karar sorusu: Hangi paylaşılan kaynak hâlâ iki havuzu birlikte durdurabilir?

Ne zaman kullanmamalıyım: İş yükleri ayrıştırılamıyorsa küçük havuzlara bölüp kapasiteyi tüketme; kritik işler için gereken kaynak payını önce ölç.

088 Load shedding: Kapasite aşılınca işi erken ve kontrollü reddetmek.

İhtiyaca bağlı

Açıklama: Sınırlı kuyruk ve öncelik politikası, kabul edilemeyecek yükün kaynak tüketmesini önler. Yanıt istemciye uygun bekleme davranışı vermelidir. Aşırı yük.

Örnek: Kritik sipariş trafiğini korumak için pahalı rapor istekleri geçici reddedilir.

Ne zaman kullanılır: İş yükü kabul edilebilir kapasiteyi aşıyorsa; kalıcı kapasite sorununu gizlemek için kullanılmaz.

Sık hata: Sonsuz kuyruk oluşturarak bütün istekleri kabul etmek.

Karar sorusu: Hangi işi önce reddederiz ve istemciler retry fırtınası çıkarmadan nasıl bekler?

Ne zaman kullanmamalıyım: Doğruluk açısından zorunlu işi sessizce düşürme; reddetme sözleşmesi, tekrar yolu ve öncelik tanımı olmadan load shedding uygulama.

089 Graceful degradation ve fallback: Hangi özelliklerin azaltılabileceği ve hangi doğruluk kurallarından vazgeçilemeyeceği.

Temel

Açıklama: Önemsiz özellikleri azaltarak temel işi sürdürürsün. Fallback güvenli bir alternatif sonuçtur; ödeme doğruluğu gibi kurallar korunur.

Örnek: Öneri servisi çalışmasa da ürün sayfası ve sepet açılır; ödeme sonucu bilinmiyorsa yanlış 'başarılı' mesajı verilmez.

Ne zaman kullanılır: Bağımlılık kesintilerinde hangi hizmetin devam edebileceğini tasarlarken.

090 Arıza alanları: Tek hata noktası, ortak bağımlılıklar, yedeklilik ve etkilenebilecek kullanıcı kapsamı.

Temel

Açıklama: Birlikte arızalanabilecek parçaları ve tek bir hatanın kaç kişiyi etkileyeceğini belirlersin. Aynı ortak bağımlılığa bağlı kopyalar tam koruma sağlamaz.

Örnek: İki sunucu aynı tek veritabanını kullanıyorsa veritabanı kesilince ikisi de iş yapamaz.

Ne zaman kullanılır: Yedeklilik ve kesinti senaryosu tasarlarken.

Başvuru: Google SRE içinde aşırı yük ve zincirleme arıza bölümleri.

Güvenlik ve mahremiyet

Şema 11 | Kimlik ve yetki sınırı



Şemayı oku: Geçerli oturum, kullanıcının kimliğini gösterir. Her nesne için sunucuda ayrıca işlem yetkisi denetlenir. Örnekte A kendi siparişine erişebilir; B'nin siparişine erişim reddedilir. Yapay zekâ (Imagegen) ile üretilmiştir.

091 Authentication ve authorization: Kimliği doğrulamak ile işlem, nesne ve alan düzeyinde yetki vermeyi ayırmak.

Temel

Açıklama: Authentication 'kimsin?', authorization 'neyi yapabilirsin?' sorusudur. Giriş yapmış olmak tüm kayıtlara erişim hakkı vermez.

Örnek: Müşteri kendi siparişini açabilir ama URL'deki numarayı değiştirip başka müşterinin siparişini göremez.

Ne zaman kullanılır: Her API ve veri erişiminde; özellikle kullanıcıya ait nesnelerde.

Sık hata: Tahmin edilmesi zor nesne kimliğini yetki kontrolü yerine koymak; bağlantıyı bilen başka kullanıcı kaydı okuyabilir.

Karar sorusu: Sunucu her okuma, düzenleme ve dışa aktarmada aktörün bu nesneye yetkisini kontrol ediyor mu?

092 Oturum ve token: Cookie, JWT, OAuth 2.0, OpenID Connect, süre sonu, yenileme, iptal ve anahtar rotasyonu.

Temel

Açıklama: Cookie tarayıcının bilgi taşıma yoludur; JWT bir token biçimidir; OAuth yetki devrini, OpenID Connect kimlik doğrulamayı düzenler. Süre dolması ve erişimi geri alma ayrıca tasarlanır. Tokenın çözümlenebilmesi geçerli ve yetkili olduğunu kanıtlamaz. Tarayıcıda kimlik bilgilerini localStorage'a koymak XSS ile okunma riski doğurur; HttpOnly, Secure ve uygun SameSite cookie veya BFF tasarımı değerlendirilir. Cookie kullanıldığında CSRF savunması ayrıca gerekir.

Örnek: Çalışan şirketten ayrıldığında oturumunun ve yenileme hakkının sona ermesi gerekir.

Ne zaman kullanılır: Giriş, sosyal hesapla bağlanma, oturum yenileme ve çıkış akışı kurarken.

Sık hata: A hesabından B'ye geçişten sonra geciken A profil yanıtını B ekranına yazmak; kısa süreli görünüm de veri sızıntısıdır. JWT kullanılması tokenın çalınmasını veya erişim iptalini kendiliğinden çözmez.

Karar sorusu: Eski istekler iptal edilemiyorsa yanıtın hâlâ aynı hesap bağlamına ait olduğunu uygulamadan önce nasıl doğrularız? Token çalınır veya erişim iptal edilirse hangi sunucu kontrolleri ne kadar sürede etkili olur?

093 En az yetki: Servis kimliği, tenant izolasyonu, secret yönetimi ve güven sınırları.

Temel

Açıklama: Her kullanıcı ve bileşene yalnız işi için gereken erişimi verirsin. Secret, parola veya API anahtarı gibi korunması gereken erişim bilgisidir.

Örnek: Raporlama servisi siparişleri okuyabilir ama silemez; anahtarını uygulamanın kaynak koduna koymazsın.

Ne zaman kullanılır: Servis hesabı, rol, API anahtarı ve müşteri sınırlarını tanımlarken.

094 Girdi ve çıktı güvenliği: SQL injection, XSS, CSRF, SSRF; doğrulama, parametreli sorgu ve güvenli çıktı üretimi.

Temel

Açıklama: Dışarıdan gelen veri komuta dönüşmemelidir. SQL injection sorguyu, XSS tarayıcı çıktısını, CSRF oturumu, SSRF sunucunun dış istek yetkisini hedefler; her biri ayrı kontrol ister.

Örnek: Arama metnini SQL komutuna yapıştırmak yerine parametre olarak geçirirsin; yorumları sayfaya güvenli biçimde yazarsın.

Ne zaman kullanılır: Kullanıcı girdisi alıp veritabanına, HTML'e veya başka bir adrese gönderirken.

095 Veri koruma: Aktarımda ve depolamada şifreleme, anahtar yönetimi, loglarda hassas veri ve veri minimizasyonu.

Temel

Açıklama: Yalnız gerekli veriyi toplar, taşırken ve saklarken korur, anahtarlarını ayrı yönetirsin. Şifreleme, uygulamanın gereksiz veri göstermesini tek başına engellemez.

Örnek: Hata kaydına müşterinin parolasını veya ödeme kartı bilgisini yazmazsın.

Ne zaman kullanılır: Kişisel veri, belge veya erişim bilgisi işleyen her üründe.

096 Tehdit modeli ve kötüye kullanım: Hesap ele geçirme, botlar, kaynak tüketimi, tedarik zinciri ve hassas iş akışlarının suistimali.

Temel

Açıklama: Bir saldırganın hangi değerli şeyi nasıl kötüye kullanabileceğini düşünürsün; yalnız teknik açık değil, normal akışın suistimali de önemlidir.

Örnek: Botlar biletleri satın almadan sürekli rezerve edip gerçek müşterileri engelleyebilir.

Ne zaman kullanılır: Giriş, kampanya, davet, rezervasyon ve kaynak tüketen API'leri tasarlarken.

Başvuru: OWASP API Security Top 10.

Gözlemlenebilirlik ve hizmet hedefleri

Şema 12 | Log metric trace



Şemayı oku: Log tek bir olayı, trace bir isteğin bileşenler arasındaki yolunu, metric ise çok sayıda işlemin genel eğilimini gösterir. Request ID olay ve trace'i bağlar; her istek için ayrı metric etiketi açmak yüksek cardinality yaratabilir. Yapay zekâ (Imagegen) ile üretilmiştir.

097 Log, metric ve trace: Her birinin cevapladığı sorular; correlation ID ve dağıtık iz bağlamı.

Temel

Açıklama: Log belirli olayı, metric zaman içindeki sayısal durumu, trace ise bir isteğin servisler arasındaki yolunu gösterir. Ortak istek kimliği kayıtları bağlar.

Örnek: Ödeme gecikmesini trace'te izleyip kargo fiyatı servisinde beklediğini, ilgili logda hatayı bulursun.

Ne zaman kullanılır: Üretimde sorun araştırmak ve kullanıcı şikâyetini belirli isteğe bağlamak için.

098 Gecikme dağılımı: Ortalama, p50, p95 ve p99; hata oranı, trafik ve kaynak doygunluğuyla birlikte yorumlama.

Temel

Açıklama: Yüzdelikler, hızlı çoğunluğun gizlediği yavaş kullanıcıları gösterir. p95 değeri, ölçülen isteklerin yüzde 95'inin o sürede veya daha kısa tamamlandığı sınırdır.

Örnek: Ortalama iyi görünürken bazı kullanıcıların ödeme ekranı çok geç açılıyor olabilir.

Ne zaman kullanılır: Kullanıcı deneyimini izlerken ve kapasite değişikliğinin etkisini değerlendirirken.

099 SLI, SLO ve SLA: Ölçülen hizmet göstergesi, hedef ve sözleşme; kullanıcı açısından başarı tanımı.

Temel

Açıklama: SLI ölçtüğün sonuç, SLO ulaşmak istediğin hedef, SLA ise sonuçları olan hizmet sözleşmesidir. Hedef, kullanıcının işini başarıyla yapmasıyla ilişkili olmalıdır.

Örnek: Sipariş oluşturma başarı oranını ölçer, bunun için hedef belirlersin; müşteriye verilen sözleşme ayrıca tanımlanır.

Ne zaman kullanılır: Hizmet kalitesini tanımlarken ve ekiplerin ortak öncelik belirlemesini sağlarken.

Sık hata: CPU ve HTTP başarı oranı normalken kullanıcının kullanılabilir giriş ekranına ulaşamadığını görmemek.

Karar sorusu: Hedefimiz sunucunun cevap vermesi mi, kullanıcının işini tamamlaması mı; sürüm ve cihaz kırılımını izliyor muyuz?

100 Hata bütçesi: Güvenilirlik hedefiyle değişiklik hızını birlikte yönetmek; burn-rate alarmı.

İhtiyaca bağlı

Açıklama: Hata bütçesi hedefin izin verdiği başarısızlık payıdır. Burn rate bu payın ne hızda tüketildiğini gösterir.

Örnek: İstek bazlı yüzde 99,9 hedefte bir milyon uygun isteğin bin tanesi başarısızlık bütçesidir; hızlı tüketim müdahale gerektirir.

Ne zaman kullanılır: Yeni özellik hızı ile güvenilirlik çalışmasını dengelerken ve anlamlı alarm kurarken.

101 Telemetri maliyeti ve güvenliği: Cardinality, sampling, saklama süresi ve hassas verilerin ayıklanması.

Temel

Açıklama: Çok sayıda farklı etiket değeri metric maliyetini büyütür. Sampling bazı kayıtları seçmek, saklama süresi ise ne kadar geçmiş tutulacağını belirlemektir.

Örnek: Her kullanıcı kimliğini ayrı metric etiketi yapmak yerine toplu oranları tutup ayrıntıyı kontrollü loglarda ararsın.

Ne zaman kullanılır: İzleme altyapısı kurarken veya izleme faturası ve veri hassasiyeti sorun olduğunda.

102 Operasyon: Eyleme dönük alarm, dashboard, runbook, incident yönetimi ve suçlama yerine nedenleri inceleyen postmortem.

Temel

Açıklama: Alarm bir müdahale istemelidir; runbook yapılacak adımları anlatır. Olay sonrası inceleme kişiyi suçlamak yerine tekrarını önleyecek koşulları arar.

Örnek: Gece ödeme hatası alarmı, sorumlu kişiyi ve güvenli geri dönüş adımlarını içeren kılavuza bağlanır.

Ne zaman kullanılır: Ürünü gerçek kullanıcılara açtığında ve kesintilere hazırlık yaparken.

Başvuru: Google SRE hizmet seviyesi hedefleri.

Operasyon örnekleri: arızayı yönetmek

Açıklama: Runbook, bir alarm için izlenecek teşhis ve müdahale adımlarını, sorumluyu, geri dönüş yolunu ve kapanış kanıtını gösteren kısa işletim belgesidir. **Örnek:** Sipariş oluşturma hataları yükseldiğinde nöbetçinin hangi paneli açacağı ve yeni sürümü ne zaman geri alacağı. **Ne zaman kullanılır:** Ürün kullanıcıya hizmet vermeye başladığında; özellikle para, veri veya kritik iş akışı etkileniyorsa.

Runbook örneği: sipariş oluşturma hataları artıyor

Bu eşikler **örnek varsayımlardır**; ürünün SLO ve normal trafik düzeyine göre ayarlanır.

Adım	Somut işlem	Karar / kanıt
Tetikleyici	Sipariş oluşturma hata oranı 5 dakika boyunca %2'yi aşıyor; istek sayısı ve kullanıcı etkisi birlikte kontrol ediliyor.	Düşük örneklemde tek hatayı yaygın kesinti sanma. Kritik veri hatası için hacimden bağımsız alarm gerekebilir.
Etkiyi sınırla	Etkilenen sürüm, bölge, tenant ve endpoint'i ayır; ödeme mi sipariş mi takılıyor bak.	trace_id ile örnek isteği izle; ham kart veya kişisel veri loglama.
Son değişikliği kontrol et	Deploy ve feature flag zamanını hata başlangıcıyla karşılaştır.	Aynı zamana denk gelmek tek başına neden kanıtı değildir.

Adım	Somut işlem	Karar / kanıt
Müdahale seç	Sürümle ilişkiliyse ve şema uyumluysa rollback; bağımlılık arızasında timeout, devre ve trafik sınırını değerlendir.	Rollback veri dönüşümünü geri alamıyorsa önce güvenli durdurma veya ileri düzeltme gerekir.
Sonucu gözle	Hata oranı, gecikme, kuyruktaki en eski iş yaşı ve ödeme uzlaştırma açığını kontrol et.	Yalnız süreçlerin ayağa kalkması yeterli değildir.
Kapat	Örnek kullanıcı akışını doğrula, bekleyen işleri uzlaştır; karar ve zamanları olay kaydına yaz.	Tanımlı gözlem penceresinde hedeflere dönüş ve veri tutarlılığı kanıtı gerekir.

Sık hata: Yeniden başlatmanın kök nedeni düzelttiğini varsaymak. **Karar sorusu:** Bu müdahale kullanıcı etkisini nasıl azaltacak; kötüleşirse hangi sinyalde geri döneceğiz?

Olay yönetimi ve toparlanma örneği

Açıklama: Incident commander karar ve koordinasyonu; müdahale sahibi teknik işi; iletişim sahibi etki bilgisini yürütür. Küçük ekipte aynı kişi birden fazla rol alabilir; görevler yine görünür olmalıdır. **Örnek:** Kuyruk büyüdüğünde biri yeni yükü sınırlar, biri işleyiciyi düzeltir, biri kullanıcı etkisini ve durumu kaydeder. **Ne zaman kullanılır:** Birden fazla kişi veya sistemin eşgüdümü gerekiyorsa. Google SRE olay yönetimi.

Toparlanma sırası: Kullanıcı etkisini sınırla güvenli veri kaynağını belirle tutarlılığı doğrula hizmeti kontrollü aç eksik/tekrarlı işleri uzlaştır. Eski birincilin yazmasını engellemeden yeni birincili açmak iki ayrı doğru veri kaydı oluşturabilir. PostgreSQL failover.

Sık hata: Yedek var diye geri yüklemenin çalıştığını kabul etmek veya kopyayı yedek sanmak; yanlış silme kopyaya da yayılabilir. **Karar sorusu:** Son kurtarılabilir veri noktası nerede; geri yükleme ve doğrulama gerçekten denendi mi? Restore tatbikatında başlangıç zamanı, son veri noktası, kontrol edilen kayıtlar ve gerçekleşen kesinti süresini kaydet; sonuçları RPO/RTO hedefleriyle karşılaştır.

Postmortem örneği: “DB bozuldu” yerine “yeni sorgu bağlantıları tüketti; havuz doluluğu alarmı yoktu; geri alma 12 dakika sürdü” gibi kanıtlı zaman çizelgesi kullan. Bu cümle temsili bir olaydır. Düzeltmenin sahibi ve doğrulanabilir kapanış koşulu olsun; yalnız insan hatasına bağlamak tekrarını önlemez.

Teknik teşhis araçlarını okuyabilmek

103 Network paneli: İsteğin yapılıp yapılmadığını, kodunu, başlıklarını ve yanıt gövdesini okuyabilmek.

Temel

Açıklama: Tarayıcının gördüğü ağ trafiği, sorunu somutlaştırır. Panel tek başına sunucunun iç durumunu veya ekrandaki iş sonucunu kanıtlamaz.

Örnek: Kampanya kartı boşken API 200 ve boş liste dönebilir; başka bir durumda istek hiç gönderilmemiş olabilir.

Ne zaman kullanılır: Web ekranında boşluk, erişim hatası, yanlış veri veya yüklenme sorunu araştırırken.

Sık hata: 200 yanıtını ekranın ve ürün davranışının doğru olduğu kanıtı saymak.

Karar sorusu: İstek hiç yok mu, reddedildi mi, yoksa beklenenden farklı veri mi geldi?

104 İstek zamanlaması ve waterfall: DNS, bağlantı, TLS, bekleme, aktarım ve istemci işlemesini ayırmak.

Temel

Açıklama: Toplam süre farklı aşamalardan oluşur; ilk ziyaret ile bağlantı ve cache kullanılan tekrar ziyareti ayrıştırırsın. Yanıt bekleme süresi ağ etkilerini de içerir.

Örnek: İlk açılış yavaş, ikinci açılış hızlıysa bağlantı kurulumu veya cache etkisini sunucu trace'iyle birlikte incelersin.

Ne zaman kullanılır: “Uygulama yavaş” şikâyetini ölçülebilir bir araştırmaya dönüştürürken.

Sık hata: Network panelindeki toplam süreyi doğrudan backend işlem süresi diye raporlamak.

Karar sorusu: Beklemenin hangi kısmını ölçtük ve hangi kısmı hâlâ bilmiyoruz?

105 cURL ve Postman: İstek yöntemini, adresini, parametrelerini, yetkisini ve yanıtını bağımsız incelemek.

Temel

Açıklama: cURL komutla, Postman görsel arayüzle HTTP isteği kurar. Ortam ve yöntem seçimi önemlidir; bazı istekler veri değiştirir.

Örnek: Test ortamındaki GET /campaigns yanıtını sözleşmedeki boş liste ve sayfalama alanlarıyla karşılaştırırsın.

Ne zaman kullanılır: İstemciden bağımsız API davranışını incelemek veya yeniden üretilebilir hata örneği hazırlamak gerektiğinde.

Sık hata: Yanlış ortamda değişiklik yapan istek çalıştırmak veya paylaşılan komuta gerçek token eklemek.

Karar sorusu: Hedef ortam, yetki ve olası yan etki açık mı?

106 Temel SQL okuryazarlığı: SELECT, WHERE, GROUP BY ve JOIN ile veri sorusunu ifade etmek.

Temel

Açıklama: SELECT alanları seçer, WHERE kayıtları süzer, GROUP BY toplar, JOIN ilişkili kayıtları birleştirir. Sorgudan önce neyin sayıldığını tanımlarsın.

Örnek: Bir siparişin iki ödeme denemesi varsa JOIN sonrası COUNT(*) iki satır sayabilir; tekil sipariş sayısıyla aynı değildir.

Ne zaman kullanılır: Bir raporu anlamak, analistle gereksinimi netleştirmek veya yetkili ortamda veri araştırmak gerektiğinde.

Sık hata: Satır sayısını otomatik olarak kullanıcı veya sipariş sayısı kabul etmek.

Karar sorusu: Bu sonuçtaki bir satır hangi gerçek varlığı veya olayı temsil ediyor?

107 Tekrarlanabilir hata raporu: Beklenen ve gerçekleşen davranışı ortam, zaman ve işlem referansıyla kaydetmek.

Temel

Açıklama: İyi rapor gözlem ile tahmini ayırır; cihaz, uygulama sürümü, adımlar ve request ID araştırmayı daraltır. Hassas veri paylaşmadan kanıt sunarsın.

Örnek: “Kampanya 7 açılırken GET isteği 502 döndü; req-17; ekran yüklemeden çıkmadı” notu ilgili trace'e götürebilir.

Ne zaman kullanılır: Hata önceliklendirirken, başka ekibe aktarırken veya üretim olayını incelerken.

Sık hata: Yalnız ekran görüntüsü paylaşmak ya da kanıt olmadan sorumlu ekibi ilan etmek.

Karar sorusu: Başka biri aynı koşulu üretebilir veya aynı isteği kayıtlarda bulabilir mi?

108 Veri incelemesinin sınırları: Yetkiyi, sorgu yükünü, veri tazeliğini ve hassas alanları bilmek.

Temel

Açıklama: Salt okunur sorgu bile kişisel veri gösterebilir veya ağır çalışabilir. Analitik kopyanın gecikmesiyle gerçek işlem kaynağının güncelliğini ayırırsın.

Örnek: Az önce ödenen sipariş raporlama kopyasına henüz ulaşmadığı için sorguda görünmeyebilir.

Ne zaman kullanılır: SQL, log dışa aktarma veya analitik ekranından ürün kararı üretirken.

Sık hata: Raporda görünmeyen kaydın kaynak sistemde hiç oluşmadığını varsaymak.

Karar sorusu: Bu veriyi okumaya yetkimiz var mı; veri ne kadar güncel ve hangi yükle okunuyor?

Ürün analitiği ve deneyler

Şema 13 | Dönüşüm hunisi ve olay kalitesi



Şemayı oku: Aynı kullanıcı grubu ve zaman penceresinde 100 kişiden 30'u tamamladığında toplam dönüşüm %30'dur. Tıklayan 40 kişiden 30'u tamamladığında tıklama sonrası dönüşüm %75'tir. Aynı event'in iki kopyası iki kullanıcı değildir; sayılar temsilidir. Yapay zekâ (Imagegen) ile üretilmiştir.

109 Olay sözlüğü: Event adını, tetiklenme anını, kimliğini, alanlarını ve sürümünü tanımlamak.

Temel

Açıklama: Olayın neyi kanıtladığı açık olmalıdır. Kullanıcının eylem denemesi ile sunucuda doğrulanmış iş sonucu ayrı kayıtlardır.

Örnek: Satın al düğmesine basılması purchase_clicked; ödemenin doğrulanması ise ayrı bir purchase_completed olayıdır.

Ne zaman kullanılır: Yeni özellik ölçümü kurarken veya ekiplerin farklı başarı sayıları ürettiği durumda.

Sık hata: Düğme tıklamasını başarılı satın alma gibi saymak.

Karar sorusu: Bu olay kullanıcı niyetini mi, sistemce doğrulanmış sonucu mu gösteriyor?

110 Funnel ve dönüşüm: Akış adımlarını, kullanıcı birimini, paydayı ve zaman penceresini tanımlamak.

Temel

Açıklama: Bir hunide kimlerin hangi adıma ulaştığını ölçersin. Deneme, oturum ve tekil kullanıcı sayıları farklı sorulara cevap verir.

Örnek: Aynı kişi üç kez giriş denerse üç deneme ve bir kullanıcı vardır; başarı oranı seçilen paydaya göre değişir.

Ne zaman kullanılır: Terk edilen adımı bulmak veya bir akış değişikliğinin etkisini değerlendirmek gerektiğinde.

Sık hata: Farklı payda ve zaman pencereleriyle üretilmiş oranları karşılaştırmak.

Karar sorusu: Oranın payı, paydası ve tamamlanma süresi tam olarak nedir?

111 Kohort ve kırılım: Sonuçları sürüm, cihaz, bölge veya başlangıç grubuna göre incelemek.

Temel

Açıklama: Genel ortalama, küçük ama önemli bir gruptaki bozulmayı gizleyebilir. Grubun büyüklüğünü ve belirsizliğini de değerlendirirsin.

Örnek: Genel giriş başarısı aynı kalırken eski Android sürümünde giriş döngüsü artmış olabilir.

Ne zaman kullanılır: Mobil sürüm yayını, bölgesel genişleme veya belirli kullanıcı şikâyetlerinde.

Sık hata: Küçük gruptaki rastgele farkı kesin etki saymak ya da yalnız genel ortalamaya bakmak.

Karar sorusu: Hangi kullanıcı grubunda sonuç farklı ve bunu güvenle yorumlayacak kadar veri var mı?

112 A/B deneyi: Hipotezi, rastgele atama birimini, birincil metriği ve karar kuralını önceden belirlemek.

İhtiyaca bağlı

Açıklama: Varyantları karşılaştırırken grupların nasıl oluştuğu ve neyin ölçüldüğü önemlidir. Süre, örneklem ve tekrar tekrar sonuca bakmak yorumu etkiler.

Örnek: Giriş formu varyantlarını kullanıcı bazında sabit atamayla karşılaştırıp başarılı giriş yapan kullanıcı oranını izlersin.

Ne zaman kullanılır: Bir ürün değişikliğinin etkisini, uygun trafik ve deney koşullarında nedensel olarak sınamak istediğinde.

Sık hata: İlk olumlu farkta deneyi durdurmak veya yenilemede kullanıcının varyantını değiştirmek.

Karar sorusu: Deneye kim girdi, ne kadar farkı saptayabiliriz ve karar kuralımız nedir?

113 Guardrail metrikleri: Ana hedef iyileşirken bozulmaması gereken sonuçları belirlemek.

Temel

Açıklama: Tek başarı metriği zararlı yan etkileri kaçırabilir. Güvenlik, hata, gecikme, iptal ve maliyet gibi sınırlar kararla birlikte izlenir.

Örnek: Bildirim açılışı artarken bildirimleri kapatan kullanıcı oranı da artıyorsa gönderim sıklığını yeniden değerlendirirsin.

Ne zaman kullanılır: Deney, kademeli yayın ve otomatik optimizasyonlarda.

Sık hata: Tıklama veya gelir artışını diğer kullanıcı sonuçlarını görmeden başarı ilan etmek.

Karar sorusu: Hangi yan etki, ana metrik artsa bile yayını durdurmalı?

114 Ölçüm kalitesi ve belirsizlik: Eksik, çift, geç gelen olayları ve deney grubu dengesini kontrol etmek.

Temel

Açıklama: Sonuç yorumlamadan önce olayların doğru üretildiğini ve beklenen grupların ölçüldüğünü doğrularsın. Gözlenen ilişki tek başına nedensellik değildir.

Örnek: Yeni arayüz aynı tıklama olayını iki kez üretiyorsa görünür artış kullanıcı davranışından kaynaklanmayabilir.

Ne zaman kullanılır: Beklenmedik metrik sıçraması, deney sonucu veya finans ve analitik raporu uyuşmazlığında.

Sık hata: Ölçüm hatasını ürün etkisi saymak; belirsiz farkı kesin kazanım olarak sunmak.

Karar sorusu: Bu fark gerçek davranış mı, veri sorunu mu ve sonucu ne kadar kesin biliyoruz?

Test ve doğrulama

115 Unit, integration, contract ve end-to-end testleri: Her seviyenin kanıtlayabildiği ve kaçırabildiği davranışlar.

Temel

Açıklama: Unit küçük parçayı, integration parçaların birlikte çalışmasını, contract iki tarafın anlaşmasını, end-to-end kullanıcının bütün yolunu sınar.

Örnek: İndirim hesabı tek başına doğru olabilir ama ödeme ekranının yanlış tutarı göndermesi uçtan uca testte görülür.

Ne zaman kullanılır: Değişikliğin hangi davranışı bozabileceğine göre test seçerken.

Sık hata: Yeşil unit testlerini tüm kullanıcı akışının doğru olduğu kanıtı saymak; eski istemci veya gecikmiş webhook kapsam dışı kalabilir.

Karar sorusu: Hangi kritik riski hangi test kapatıyor; başarı dışındaki hangi yollar hâlâ sınanmadı?

116 Yük, stres, spike ve soak testleri: Normal kapasite, sınır, ani artış ve uzun süreli kaynak sorunları.

İhtiyaca bağlı

Açıklama: Yük testi beklenen trafiği, stres sınırın ötesini, spike ani yükselişi, soak uzun süreli çalışmayı inceler.

Örnek: Bilet satışının açılışındaki ani yük ile günler içinde ortaya çıkan bellek sızıntısı farklı testler gerektirir.

Ne zaman kullanılır: Kampanya, büyüme veya uzun süre çalışma riskini doğrularken.

117 Gerçekçi test yükü: Trafik dağılımı, veri boyutu, sıcak ve soğuk cache, yavaş bağımlılıklar ve kuyruk birikmesi.

İhtiyaca bağlı

Açıklama: Test, gerçek kullanıcıların veri boyutunu ve davranışını yeterince temsil etmelidir. Sadece küçük ve önbellekteki veri iyimser sonuç verir.

Örnek: Boş hesapla hızlı açılan rapor, yıllarca kayıt biriktirmiş müşteride yavaşlayabilir.

Ne zaman kullanılır: Performans sonucu üretim kararı için kullanılacaksa.

118 Hata enjeksiyonu: Paket kaybı, süreç ölümü, bağımlılık kesintisi ve bölünmüş ağ altında doğruluk.

İleri

Açıklama: Kontrollü hata ekleyerek sistemin yalnız sorunsuz koşullarda değil, arıza altında da beklenen şekilde davrandığını görürsün.

Örnek: Test ortamında ödeme yanıtını geciktirip siparişin iki kez oluşmadığını kontrol edersin.

Ne zaman kullanılır: Hata yönetimi ve kurtarma mekanizmasını doğrularken; etkisi sınırlandırılmış ortamlarda.

119 Veri doğruluğu testleri: Invariant, property-based test, yarış koşulu, tekrarlı istek ve mesaj testleri.

İhtiyaca bağlı

Açıklama: Yalnız örnek sonuca değil, her geçerli girişte korunacak kurala bakarsın. Property-based test çok sayıda farklı girdi üretir.

Örnek: Sepet işlemlerinin sırası değişse de toplam tutarın kurala uygun kalmasını ve aynı ödeme isteğinin çift kayıt üretmemesini sınarsın.

Ne zaman kullanılır: Para, stok, eşzamanlı güncelleme ve tekrar işleme gibi doğruluk kritik alanlarda.

120 Kurtarma doğrulaması: Yedekten geri yükleme, failover, rollback ve veri migration sonrasında karşılaştırma.

Temel

Açıklama: Bir yedeğin bulunması geri yüklenebildiğini kanıtlamaz. Kurtarma yolunu çalıştırıp veriyi ve hizmeti kontrol etmen gerekir.

Örnek: Yedek dosyasını ayrı ortama geri yükler, sipariş kayıtlarının ve uygulama girişinin çalıştığını doğrularsın.

Ne zaman kullanılır: Yedekleme, failover veya migration çözümünü hazır kabul etmeden önce.

Dağıtım ve altyapı

121 VM, container ve serverless: İzolasyon, yaşam döngüsü, cold start, kaynak sınırı ve işletim sorumluluğu.

İhtiyaca bağlı

Açıklama: VM ayrı işletim sistemi ortamı, container paylaşılan çekirdek üzerinde paketlenmiş çalışma ortamı sunar. Serverless altyapı işletiminin bir bölümünü sağlayıcıya bırakır; uygulama sorumluluğu sürer.

Örnek: Düzensiz çalışan görsel işini serverless ile, sürekli çalışan özel uygulamayı container ile değerlendirebilirsin.

Ne zaman kullanılır: Dağıtım biçimi seçerken; trafik, çalışma süresi ve işletim ihtiyacına göre.

Ne zaman kullanmamalıyım: Trafik ve çalışma süresiyle uyuşmayan serverless veya container seçimini yalnız popüler olduğu için yapma; yaşam döngüsü ve işletim yükünü birlikte değerlendir.

122 Bulut ağı: Region, availability zone, VPC, subnet, routing, firewall ve private/public erişim.

İhtiyaca bağlı

Açıklama: Region coğrafi bölge, availability zone ayrı arıza alanı olarak tasarlanan yerleşimdir. VPC ve subnet ağ sınırlarını, routing yolu, firewall izinleri belirler.

Örnek: Web sunucusu dış istek kabul ederken veritabanını yalnız uygulamanın eriştiği özel ağda tutarsın.

Ne zaman kullanılır: Bulut altyapısı kurarken veya erişim ve kesinti sınırlarını belirlerken.

123 CI/CD ve artifact: Tekrarlanabilir build, bağımlılık sabitleme, ortam ayrımı ve aynı artifact'in ortamlara taşınması.

Temel

Açıklama: CI/CD kodu otomatik kontrol edip paketler ve dağıtır. Artifact, üretilmiş pakettir; aynı paketi taşımak ortamlarda farklı kod çalışması riskini azaltır.

Örnek: Testte doğruladığın container imajını yeniden derlemeden üretime geçirirsin.

Ne zaman kullanılır: Sürüm yayınını tekrarlanabilir ve izlenebilir hale getirirken.

124 Yayın stratejileri: Rolling, blue-green, canary, feature flag ve geri dönüş koşulları.

İhtiyaca bağlı

Açıklama: Rolling sunucuları sırayla yeniler; blue-green iki ortam arasında geçer; canary küçük kitleyle başlar; feature flag özelliği ayrı açıp kapatır.

Örnek: Yeni ödeme ekranını önce küçük bir kullanıcı grubuna açar, hatalar artarsa kapatırsın.

Ne zaman kullanılır: Yeni sürümün etkisini sınırlamak ve güvenli geri dönüş sağlamak için.

125 Şema değişiklikleri: Expand-contract, backfill, eski ve yeni sürümlerin birlikte çalışması; kod rollback'inin veri dönüşünü garanti etmemesi.

Temel

Açıklama: Expand-contract önce yeni yapıyı ekler, veriyi taşır, eski kullanımı bitirince eski alanı kaldırır. Veri kaybı yaratan değişiklik kodu geri almakla düzelmez.

Örnek: Ad alanını bölmek için önce yeni alanları ekler, eski sürümlerle uyumlu doldurur, sonra eski alanı emekliye ayırırsın.

Ne zaman kullanılır: Canlı veritabanı şemasını kullanıcıları kesmeden değiştirirken.

Sık hata: Currency alanını tek adımda zorunlu yaparak bu alanı göndermeyen eski istemcileri kırmak.

Karar sorusu: Genişletme, yeni yazım, backfill ve daraltma aşamalarında eski ve yeni istemciler birlikte çalışıyor mu?

126 IaC ve orkestrasyon: Altyapıyı kodla tanımlama, yapılandırma sapması, health/readiness probe, servis keşfi ve Kubernetes'in kullanım bedeli.

İleri

Açıklama: IaC altyapının tarifini kod olarak tutar. Orkestrasyon çalışanları yerleştirir ve yeniler; readiness trafik alabilirliğini, health kontrolleri seçilen sağlık durumunu değerlendirir.

Örnek: Yeni sunucuya ayarları elle tekrar yazmak yerine aynı altyapı tarifini uygularsın.

Ne zaman kullanılır: Ortamlar çoğaldığında veya tekrar kurma ihtiyacı doğduğunda; Kubernetes'i işletim ihtiyacı gerçekten gerekliyse değerlendirirsin.

Ne zaman kullanmamalıyım: Basit dağıtım ihtiyacını karşılayan yönetilen bir çözüm varken Kubernetes'i varsayılan yapma. Kümenin bakım ve güvenlik sorumlusu yoksa işletim bedeli ağırlaşır.

Geliştirmeden kullanıcıya ulaşma zinciri

Şema 14 | Build deploy release



Şemayı oku: Commit ve PR kod değişikliğini, build çalıştırılabilir çıktıyı, deploy bu çıktının bir ortama taşınmasını gösterir. Özellik bayrağıyla kullanıcıya açılma ayrı bir release adımı olabilir; her aşama ayrı kanıt gerektirir. Yapay zekâ (Imagegen) ile üretilmiştir.

127 Git, commit ve branch: Değişiklik geçmişini ve paralel çalışma alanlarını okuyabilmek.

Temel

Açıklama: Commit kaydedilmiş bir değişiklik durumudur; branch bir geliştirme hattını takip eder. Yerel değişiklik, commit ve uzak depoya push ayrı durumlardır.

Örnek: Bir düzeltme geliştiricinin bilgisayarında çalışabilir ama henüz commit edilmediği veya push edilmediği için ekipte görünmeyebilir.

Ne zaman kullanılır: Bir işin nerede kaldığını anlamak veya belirli bir değişikliğin kaynağını bulmak gerektiğinde.

Sık hata: “Kod yazıldı” ifadesinden değişikliğin depoda ya da üretimde olduğunu çıkarmak.

Karar sorusu: Değişiklik hangi commit'te ve hangi depoda mevcut?

128 Pull request, inceleme ve birleştirme: Önerilen değişikliği kapsamı ve kanıtıyla değerlendirmek.

Temel

Açıklama: PR değişikliklerin tartışıldığı ve incelendiği yerdir. İnceleme, otomatik kontroller, merge ve dağıtım ayrı aşamalardır; her kontrolün kapsamı sınırlıdır.

Örnek: PR testleri geçse bile yalnız en yeni mobil istemciyle sınanan API değişikliği eski sürümü bozabilir.

Ne zaman kullanılır: Teknik teslimatı takip ederken veya bir değişikliğin yayına hazır olup olmadığını değerlendirirken.

Sık hata: PR'ın onaylanmasını veya birleştirilmesini canlı yayının kanıtı saymak.

Karar sorusu: Hangi riskler incelendi, hangi testler çalıştı ve hangi istemciler kapsandı?

129 Build, deploy ve release: Çalıştırılabilir çıktı, ortama dağıtım ve kullanıcıya açılmayı ayırmak.

Temel

Açıklama: Build bir çıktı üretir, deploy onu bir ortama taşır, release özelliği hedef kullanıcıya açar. Feature flag ve mobil mağaza aşamaları bu durumları birbirinden ayırabilir.

Örnek: Yeni ödeme kodu üretimde olabilir; özellik bayrağı kapalıyken kullanıcı eski akışı görmeye devam eder.

Ne zaman kullanılır: Yayın takibi, destek hazırlığı, kademeli açılış ve geri alma kararlarında.

Sık hata: Başarılı build veya upload sonucundan özelliğin kullanıcıda çalıştığı sonucuna varmak.

Karar sorusu: Hangi sürüm, hangi ortamda, hangi kullanıcı grubuna gerçekten açık ve bunun kanıtı ne?

İleri sistem tasarımı

Dağıtık sistem garantileri

130 Tutarlılık modelleri: Linearizability, eventual consistency, causal consistency ve read-your-writes.

İleri

Açıklama: Tutarlılık modeli, farklı yerlerden okuyanların hangi sırayla hangi veriyi göreceğini tanımlar. Eventual consistency kopyaların zamanla yakınsamasıdır; read-your-writes kendi değişikliğini sonraki okumada görmendir.

Örnek: Profil adını değiştiren kişi hemen yeni adı görürken başka bölgede okuyan kullanıcı kısa süre eski adı görebilir.

Ne zaman kullanılır: Veri çoğaltıldığında hangi gecikme ve sıralama davranışının ürüne uygun olduğunu belirlemek için.

131 CAP: Ağ bölünmesi sırasında tutarlılık ve kullanılabilirlik garantileri arasındaki gerilim; her koşulda üç özellikten ikisini seçme sloganının sınırları.

İleri

Açıklama: Ağ iki tarafı ayırdığında her iki taraftan da tüm isteklere hizmet vermek ile tek bir güncel kopya varmış gibi davranmayı aynı anda garanti edemezsin. CAP açıklaması.

Örnek: Son koltuğu iki bölge satıyorsa bağlantı kesildiğinde bir tarafın satışını durdurmak çift satıştan kaçınmak için seçilebilir.

Ne zaman kullanılır: Dağıtık veri sisteminde bağlantı kopunca hangi işlemlerin bekleyeceğini veya reddedileceğini tanımlarken.

132 Replikasyon: Leader-follower, multi-leader ve leaderless modeller; replication lag ve yazma çatışmaları.

İleri

Açıklama: Leader-follower yazıyı bir liderden yayar; multi-leader birden çok yerden yazı alır; leaderless model tek kalıcı yazma liderine dayanmaz. Gecikme ve çatışma yönetimi modele göre değişir.

Örnek: Yeni sipariş liderde görünürken okuma kopyasında henüz bulunmayabilir.

Ne zaman kullanılır: Veritabanı kopyalarının okuma ve yazma davranışını seçerken veya 'kaydettim ama görünmüyor' sorununu çözerken.

Sık hata: Yeni yazılan verinin gecikmeli replikada hemen görüneceğini varsaymak; kullanıcı kaydını kaybolmuş sanabilir.

Karar sorusu: Kendi yazısını hemen görmesi gereken akışta okumayı nereye yönlendireceğiz ve replica gecikmesini nasıl ele alacağız?

Ne zaman kullanmamalıyım: Güçlü tek-yazıcı kuralı gereken akışa çatışma çözmeden çok liderli yazma ekleme; replika gecikmesini kabul etmeyen okumayı rastgele kopyaya yönlendirme.

133 Consensus: Raft ve Paxos'un çözmeye çalıştığı anlaşma problemi; lider seçimi, çoğunluk ve ağ bölünmesi.

İleri

Açıklama: Consensus, sunucuların hangi kararların kesinleştiğinde anlaşmasıdır. Raft gibi algoritmalar çoğunlukla iletişim kurulabildiğinde ilerler; çoğunluk kaybolduğunda güvenliği korumak için ilerleme durabilir. Raft.

Örnek: Üç düğümlü bir kümede bir düğüm kaybolsa da kalan iki düğüm uygun koşullarda karar vermeyi sürdürebilir.

Ne zaman kullanılır: Kritik ortak durum, lider seçimi ve dağıtık veritabanı garantilerini anlamak için; çoğunlukla hazır altyapıyı değerlendirirken.

134 Dağıtık zaman ve kilitler: Clock skew, mantıksal saatler, lease ve fencing token; süresi dolan kilidin eski sahibini durdurmak.

İleri

Açıklama: Makine saatleri farklı olabilir. Lease süreli yetkidir; fencing token ise eski yetki sahibinin gecikmiş yazısını reddetmeye yarayan artan numaradır.

Örnek: Rapor işini alan çalışan donar, süresi dolunca iş başkasına geçer; eski çalışan uyanınca eski numarasıyla sonucu ezemez.

Ne zaman kullanılır: Dağıtık kilit, görev sahipliği veya zaman sıralaması tasarlarken.

Ne zaman kullanmamalıyım: Tek veritabanı transaction’ı ve kısıtı yeterliyse dağıtık kilit ekleme. Eski sahibin yazısını engelleyemediğin lease’i kesin tek sahiplik garantisi sayma.

135 2PC ve hazırlanmış transaction: Katılımcıların ortak commit veya rollback kararına hazırlanması.

İleri

Açıklama: Koordinatör önce katılımcılardan hazırlanmalarını ister, sonra ortak sonucu bildirir. Hazırlanmış işler karar beklerken kaynak tutabilir. PostgreSQL 2PC.

Örnek: İki uyumlu veri kaynağındaki değişikliğin birlikte kesinleşmesi gerekir; koordinatör kaybolursa bekleyen kararın kurtarılması gerekir.

Ne zaman kullanılır: Altyapı ve katılımcılar bu protokolü destekliyorsa; tek veritabanı transaction'ı yeterliyken eklemek gereksizdir.

Sık hata: Her HTTP servisini 2PC katılımcısı saymak.

Karar sorusu: Karar kaybolursa hazırlanmış işleri kim ve hangi kalıcı kayıtla sonuçlandıracak?

Ne zaman kullanmamalıyım: Katılımcılar protokolü desteklemiyorsa veya tek transaction yeterliyse 2PC seçme; hazırlanmış işlemlerin kurtarma sorumlusu olmadan üretime taşıma.

136 Saga: Birden fazla servisteki yerel işlemleri telafi adımlarıyla tamamlamak.

İleri

Açıklama: Her adım kendi transaction'ını tamamlar. Sonraki adım başarısızsa önceki etkiler için iş anlamında telafi yapılır; tam izolasyon veya geçmişi silme garantisi yoktur. Saga.

Örnek: Stok ayrılır, ödeme kesin reddedilirse stok bırakılır. Ödemenin sonucu bilinmiyorsa önce sorgulanır; kör telafi yapılmaz.

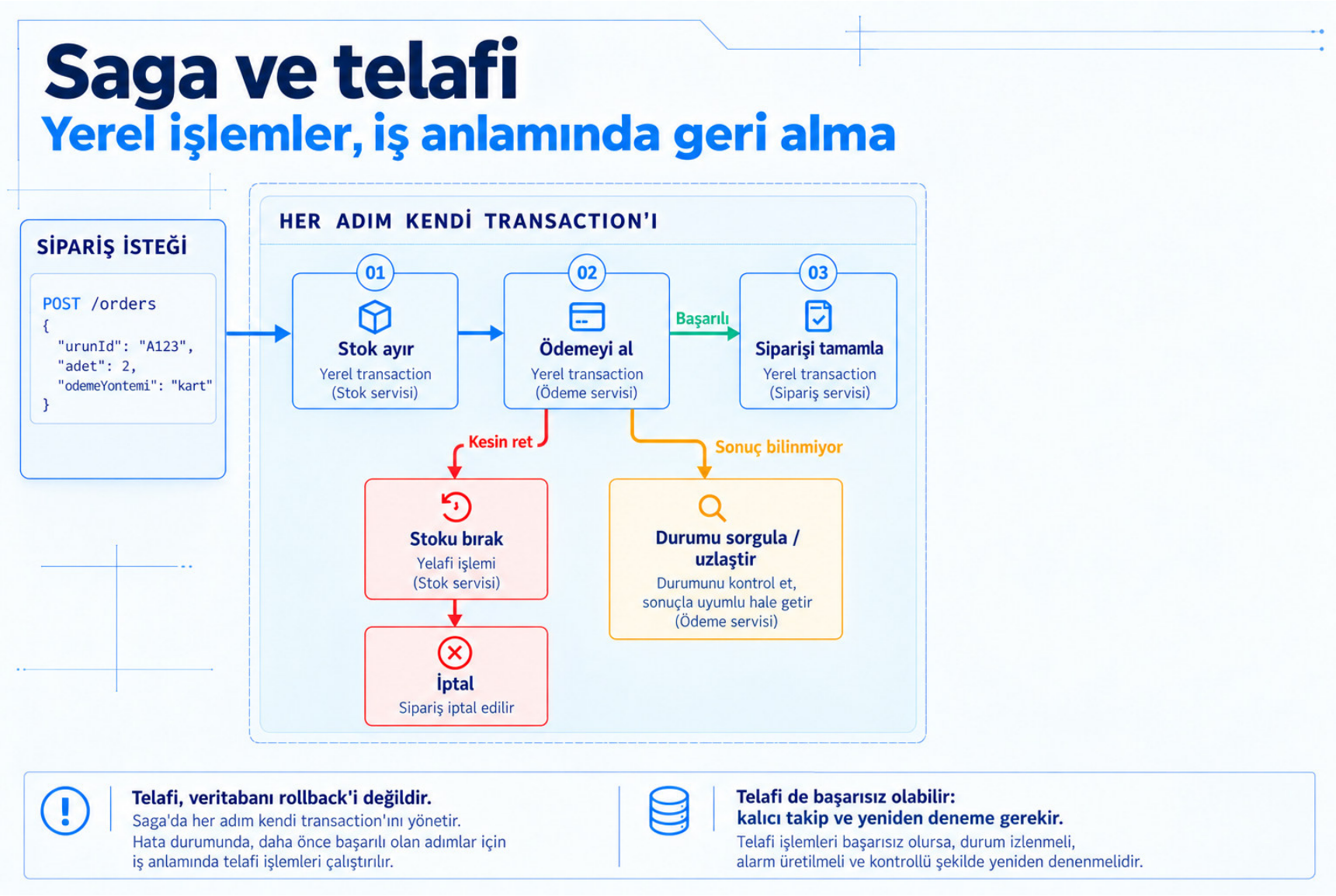
Ne zaman kullanılır: İş ayrı veri sahiplerine yayılıyorsa; tek atomik işlem yeterliyken gereksizdir.

Sık hata: Telafiyi gerçek rollback sanmak; telafinin de başarısız olabileceğini atlamak.

Karar sorusu: Hangi etki geri alınabilir; telafi takılırsa kim uzlaştıracak?

Ne zaman kullanmamalıyım: Tek veri sahibinde atomik işlem mümkünken saga ekleme. Geri döndürülemeyen etkilerde telafiyi gerçek rollback gibi vaat etme.

Şema 15 | Saga ve telafi



Şemayı oku - Saga: Bu sadeleştirilmiş örnekte kesin ödeme reddi stok bırakmayı ve iptali başlatır. Sonuç belirsizken stok bırakma veya yeni ödeme başlatma kararı verilmez; sorgulama ve uzlaştırma sürer. Stok ayırmanın kendisinin başarısız olması, ödeme sonrası sonlandırma hatası ve başarısız telafi de ayrı kurtarma yolları gerektirir. Yapay zekâ (Imagegen) ile üretilmiştir.

137 Transactional outbox: İş kaydı ile gönderilecek olay kaydını aynı transaction'da tutmak.

İhtiyaca bağlı

Açıklama: İş verisi ve outbox olayı birlikte commit edilir. Ayrı yayıncı olayı kuyruğa taşır; yayın ve işaretleme arasında çökme olursa tekrar yayın mümkün kalır. Outbox.

Örnek: Sipariş ve OrderCreated kaydı birlikte yazılır; kuyruk kapalıysa yayıncı daha sonra devam eder.

Ne zaman kullanılır: Kalıcı bir veri değişikliği mutlaka olay üretmeliyse; dayanıklı olay ihtiyacı yoksa gereksizdir.

Sık hata: Outbox'ın tüketicide tam bir kez yan etki garantilediğini sanmak.

Karar sorusu: Commit sonrası çökmede olay nerede duruyor ve tekrar kim tarafından tanınıyor?

Ne zaman kullanmamalıyım: Dayanıklı olay yayınlama ihtiyacı yoksa her tabloya outbox ekleme. Yayıncıyı ve biriken olayları işletecek kapasite olmadan desen tek başına çözüm olmaz.

Şema 16 | Transactional outbox



Şemayı oku - Outbox: Sipariş ve olay niyeti aynı DB transaction'ında yazılır. Yayıncı commit edilmiş olayı kuyruğa yollar; ardından işaretler. Bu iki adım arasında çökme tekrar yayına yol açabilir. Tüketicinin yerel tekilleştirmesi ve harici yan etkiler için ayrıca tekrar güvenliği gerekir. Yapay zekâ (Imagegen) ile üretilmiştir.

138 Transactional inbox: Gelen olayın işlendiğini iş değişikliğiyle birlikte kaydetmek.

İhtiyaca bağlı

Açıklama: Tüketici event_id için unique kayıt tutar; bu kayıt ve yerel iş etkisi aynı transaction'da kesinleşir. Harici e-posta veya ödeme çağrısının tekilleştirilmesi ayrıca gerekir.

Örnek: Aynı OrderPaid olayı tekrar gelince ikinci paket hakkı açılmaz; ACK kaybolsa da kayıt yeniden işlenmez.

Ne zaman kullanılır: En az bir kez teslim edilen olaylarda yerel çift etkiyi önlerken; işlem zaten doğal olarak idempotentse ek kayıt gerekmeyebilir.

Sık hata: İş tamamlanmadan yalnız event_id'yi işlendi diye kaydetmek.

Karar sorusu: Tekilleştirme kaydıyla iş etkisi atomik mi; kayıt ne kadar süre tutulacak?

Ne zaman kullanmamalıyım: Tüketici etkisi zaten doğal olarak idempotentse gereksiz inbox kaydı tutma. Harici tahsilatı yerel inbox transaction'ının koruduğunu varsayma.

139 CQRS: Komut ve sorgular için farklı modeller kullanmak.

İleri

Açıklama: Yazma iş kurallarını, okuma ekranın ihtiyaçlarını modeller. Ayrı veritabanı veya event sourcing zorunlu değildir. Ayrı projeksiyon varsa güncelleme gecikmesi yönetilir. CQRS.

Örnek: Sipariş yazımı kuralları denetler; müşteri paneli hazır bir sipariş özeti okur.

Ne zaman kullanılır: Okuma ve yazmanın ihtiyaçları belirgin ayrıştığında; basit CRUD için gereksiz karmaşıklık olabilir.

Sık hata: Her ekrana ayrı servis açmayı CQRS sanmak.

Karar sorusu: Okuma modeli ne kadar gecikebilir ve nasıl yeniden oluşturulur?

Ne zaman kullanmamalıyım: Okuma ve yazma ihtiyaçları basit ve benzerken iki ayrı modeli sürdürme. Projeksiyon gecikmesi ve yeniden oluşturma maliyeti kabul edilmiyorsa ayrı okuma deposu seçme.

140 Event sourcing: İş olaylarını asıl kayıt kabul ederek durumu yeniden üretmek.

İleri

Açıklama: Güncel durum sıralı olaylardan hesaplanır. Olay sürümü, replay ve snapshot yönetimi gerekir; sıradan log tutmakla aynı değildir. Event sourcing.

Örnek: Rezervasyon oluşturuldu ve iptal edildi olaylarından son durum hesaplanır; replay sırasında tekrar e-posta gönderilmez.

Ne zaman kullanılır: Geçmişi yeniden kurma ihtiyacı maliyeti karşılıyorsa; yalnız değişiklik geçmişi için audit log yeterli olabilir.

Sık hata: Tüm geçmiş olayların gelecekte aynı şemayla okunacağını varsaymak.

Karar sorusu: Eski olayları yeni kodla yeniden oynatınca hangi yan etkiler engellenecek?

Ne zaman kullanmamalıyım: Yalnız değişiklik geçmişi gerekiyorsa event sourcing'i varsayılan yapma; audit trail daha dar ihtiyacı karşılayabilir. Eski olayları yaşatma ve replay maliyetini üstlenemiyorsan seçme.

Çok bölgeli çalışma ve felaket kurtarma

141 Active-passive ve active-active: Trafik yönlendirme, yazma sahipliği, çoğaltma ve maliyet.

İleri

Açıklama: Active-passive'de yedek ortam devralmayı bekler; active-active'de birden fazla bölge hizmet verir. Birden fazla yerde yazma kabul etmek veri uzlaştırma ihtiyacı doğurabilir.

Örnek: Bir bölge çalışamaz hale gelince hazır yedek bölgeye trafik aktarılır.

Ne zaman kullanılır: Bölgesel kesinti hedefi veya coğrafi kullanıcı dağılımı tek bölgeyi yetersiz kıldığında.

Ne zaman kullanmamalıyım: Tek bölge gereksinimi karşılıyorsa active-active yazma ile başlama. Çatışma, yönlendirme ve ortak bağımlılıklar çözülmeden kesintisizlik vaat etme.

142 RPO ve RTO: Kabul edilebilir veri kaybı ile hizmeti geri getirme süresini ayırmak.

Temel

Açıklama: RPO kaybetmeyi kabul edebileceğin veri aralığı, RTO hizmeti geri getirme hedef süresidir. Bu hedefler yedekleme ve kurtarma yöntemini belirler.

Örnek: RPO beş dakika ise son beş dakikalık kayıt riski kabul edilir; RTO bir saat ise hizmet bir saat içinde geri gelmelidir.

Ne zaman kullanılır: İş sahibinin kesinti ve veri kaybı toleransını altyapı kararına çevirirken.

143 Backup ve replication: Silme veya bozulmanın replikalara da yayılabilmesi; ayrı yedek ve point-in-time recovery ihtiyacı.

Temel

Açıklama: Replika güncel kopyadır; yanlış silme ona da yayılabilir. Yedek ve point-in-time recovery geçmişteki uygun ana geri dönmeye yardım eder.

Örnek: Yanlışlıkla silinen siparişler tüm replikalarda kaybolsa da olaydan önceki noktaya geri yüklenebilir.

Ne zaman kullanılır: Veri koruma planını oluştururken ve yedekliliği değerlendirirken.

144 Failover ve failback: Trafik geçişi, split-brain, veri uzlaştırma ve eski bölgeye güvenli dönüş.

İleri

Açıklama: Failover yedeğe geçmek, failback ana ortama dönmektir. Split-brain birden fazla tarafın kendini yetkili saymasıdır; dönüşte farklılaşan veriler uzlaştırılmalıdır.

Örnek: Eski bölge yeniden açılınca doğrudan yazma kabul ettirmek yerine güncelliğini ve liderliğini doğrularsın.

Ne zaman kullanılır: Bölge veya veritabanı devralma prosedürü tasarlarken.

145 Coğrafi yerleşim: Kullanıcı, uygulama ve veritabanı arasındaki ağ mesafesi; verinin hangi bölgede tutulduğu.

İhtiyaca bağlı

Açıklama: Kullanıcıya yakınlık tek başına yetmez; uygulamanın veriye yaptığı gidiş gelişler de toplam beklemeyi belirler.

Örnek: Türkiye'deki kullanıcı için yakın uygulama sunucusu seçsen de veritabanı uzak bölgede ve her ekranda çok çağrı varsa bekleme artabilir.

Ne zaman kullanılır: Bölge seçerken, veriyi taşırken ve coğrafi erişim kurallarını değerlendirirken.

146 Bağımlılık ve kurtarma planı: DNS, kimlik servisi, anahtarlar ve kontrol düzlemi dahil bölge dışı bağımlılıklar.

İhtiyaca bağlı

Açıklama: Kurtarma için gereken hizmetler de arızalanabilir. Çalışan sunucuya ulaşmak, giriş yapmak ve şifreli veriyi açmak bağımlılık ister.

Örnek: Yedek bölge hazırdır ama kimlik servisi yalnız arızalı bölgede çalışıyorsa kullanıcı yine giriş yapamaz.

Ne zaman kullanılır: Felaket kurtarma planını tüm zincir üzerinden kontrol ederken.

Şema 17 | Kontrollü failover



Şemayı oku - Failover: Önce eski yazara erişim kesilir; sonra kopyanın güncelliği ve kayıp riski değerlendirilir. Yeni birincil kontrollü açılır ve yeni yedeklilik kurulur. RTO ve RPO hedeflerdir; ölçülen toparlanma ve kurtarılan veri noktasının yaşı bu hedeflerle karşılaştırılır. Yapay zekâ (Imagegen) ile üretilmiştir.

Arama ve veri işleme

147 OLTP ve OLAP: Günlük işlemler ile analitik sorguların erişim ve depolama ihtiyaçları.

İhtiyaca bağlı

Açıklama: OLTP küçük ve sık günlük işlemlere, OLAP büyük veri üzerinde toplu analizlere odaklanır. Aynı kaynakta ağır analiz canlı işleri yavaşlatabilir.

Örnek: Kasadaki sipariş kaydı OLTP'dir; bir yıllık şehir bazlı satış analizi OLAP'tır.

Ne zaman kullanılır: Raporlar büyüyüp canlı uygulamanın veritabanını zorlamaya başladığında.

148 Batch ve stream processing: İşleme zamanı, olay zamanı, pencereleme ve geç gelen olaylar.

İleri

Açıklama: Batch işi toplu aralıklarla, stream olaylar geldikçe işler. Olay zamanı gerçekten ne zaman olduğunu, işleme zamanı sisteme ne zaman işlendiğini gösterir.

Örnek: İnternetsiz kalan telefonun öğlen yaptığı satışı akşam yüklemesi, satışı akşam gerçekleşmiş saymayı gerektirmez.

Ne zaman kullanılır: Canlı metrik, işlem analizi veya gecikmeli gelen cihaz verisi işlerken.

149 CDC, ETL ve ELT: Değişiklik yakalama, veri taşıma, şema değişimi ve pipeline hataları.

İleri

Açıklama: CDC kaynak verideki değişikliği yakalar. ETL veriyi dönüştürüp yükler; ELT yükledikten sonra dönüştürür. Alan değişimleri ve başarısız aktarımlar ayrıca yönetilir.

Örnek: Mağaza siparişleri günlük rapor deposuna aktarılırken para birimleri ortak biçime çevrilir.

Ne zaman kullanılır: Uygulama verisini analiz, arama veya başka sisteme taşırken.

150 Arama: Ters indeks, tokenization, relevance, filtreleme ve kaynak veritabanıyla indeks arasındaki gecikme.

İhtiyaca bağlı

Açıklama: Arama motoru kelimelerin hangi belgelerde geçtiğini indeksler; metni parçalara ayırır ve sonuçları uygunluğa göre sıralar. İndeks güncellemesi kaynak veriden geride kalabilir.

Örnek: Yeni eklenen ayakkabı kaydı veritabanında bulunur ama arama indeksi güncellenene kadar aramada görünmeyebilir.

Ne zaman kullanılır: Tam metin arama, yazım varyasyonları ve anlamlı sonuç sıralaması gerektiğinde.

151 Veri projeksiyonları: Materialized view, artımlı hesaplama, backfill, replay ve veri soyu.

İleri

Açıklama: Projeksiyon, asıl veriden belirli okuma ihtiyacı için üretilmiş görünümdür. Backfill geçmişi doldurur; replay olayları yeniden işler; veri soyu sonucun kaynağını gösterir.

Örnek: Her siparişte günlük satış toplamını günceller, hesap kuralı değişince geçmiş günleri yeniden hesaplarsın.

Ne zaman kullanılır: Sık kullanılan raporu hızlandırmak veya türetilmiş veriyi yeniden üretebilmek istediğinde.

Ne zaman kullanmamalıyım: Yalnız tek rapor için karmaşık projeksiyon hattı kurma. Kaynakla gecikme, yeniden oluşturma ve doğrulama yolu belirlenmeden asıl kayıt yerine kullanma.

152 Vektör ve hibrit arama: Embedding, yaklaşık en yakın komşu, metadata filtresi, kalite ve maliyet dengesi.

İleri

Açıklama: Embedding içeriği anlam benzerliği hesaplanabilen sayılara çevirir. Vektör arama benzer anlamı, hibrit arama kelime ve anlam eşleşmesini birlikte kullanır.

Örnek: 'Yağmurda giyilecek hafif mont' araması açıklamasında 'su geçirmez ceket' yazan ürünü bulabilir.

Ne zaman kullanılır: Tam kelime eşleşmesi yetersiz kaldığında; erişim filtreleri ve sonuç kalitesi ayrıca ölçülerek.

Gerçek zamanlı ve çevrimdışı sistemler

153 Polling, long polling, SSE ve WebSocket: Güncelleme sıklığı, iletişim yönü ve bağlantı maliyeti.

İhtiyaca bağlı

Açıklama: Polling düzenli sorar, long polling yanıt için açık bekler, SSE sunucudan olay akıtır, WebSocket iki yönlü sürekli iletişim sağlar.

Örnek: Kargo takibi seyrek sorguyla, canlı sohbet çift yönlü bağlantıyla, AI metni SSE ile sunulabilir.

Ne zaman kullanılır: Güncellemelerin hızı ve yönü normal istek-yanıt akışını yetersiz kıldığında.

Ne zaman kullanmamalıyım: Seyrek değişen veri için sürekli bağlantıyı varsayılan yapma. SSE ve WebSocket seçiminde mesaj yönünü, altyapı sınırlarını ve yeniden bağlanmayı değerlendir.

154 Bağlantı yaşam döngüsü: Heartbeat, kopma algılama, reconnect, yeniden bağlanma fırtınası ve bağlantı yetkisi.

İhtiyaca bağlı

Açıklama: Açık bağlantı sonsuza kadar sağlıklı kalmaz. Heartbeat canlılık sinyali verir; yeniden bağlanma aralıkları dağıtılır ve erişim tekrar kontrol edilir.

Örnek: Metrodan çıkan binlerce telefon aynı anda bağlanmaya çalışırsa rastgele beklemeler sunucudaki yığılmayı azaltır.

Ne zaman kullanılır: Sohbet, canlı takip veya uzun süre açık kalan bağlantılar tasarlarken.

155 Mesaj durumu: Gönderildi, sunucuya ulaştı, cihaza teslim edildi ve okundu ayrımı.

İhtiyaca bağlı

Açıklama: Mesajın aşamaları farklı kanıtlardır: cihazdan çıkması, sunucunun alması, alıcı cihaza ulaşması ve okunması aynı olay değildir.

Örnek: Kendi telefonunda mesaj balonunun görünmesi, karşı tarafın mesajı okuduğunu göstermez.

Ne zaman kullanılır: Sohbet, bildirim ve teslim durumlarını kullanıcıya gösterirken.

156 Çevrimdışı veri: Yerel kaynak, senkronizasyon kuyruğu, incremental sync, silme işaretleri ve çakışma çözümü.

İhtiyaca bağlı

Açıklama: İstemci bağlantı yokken yerelde çalışır; değişiklikleri sonra senkronize eder. Silme işareti, silinen verinin eski bir cihazdan geri gelmesini önler.

Örnek: Uçakta silinen not, telefon internete bağlanınca diğer cihazlardan da kaldırılır.

Ne zaman kullanılır: Zayıf bağlantıda çalışması gereken mobil uygulama, not ve saha veri girişinde.

157 Ortak düzenleme: Last-write-wins, operational transformation ve CRDT; hangi veri anlamlarının korunabildiği.

İleri

Açıklama: Aynı belgeye gelen değişiklikler birleştirilmelidir. Last-write-wins son yazıyı seçer; OT işlemleri dönüştürür; CRDT birleştirme kurallarıyla kopyaları yakınsatır.

Örnek: İki kişi aynı alışveriş listesine farklı ürün eklediğinde birinin eklemesi kaybolmadan ikisi de korunabilir.

Ne zaman kullanılır: Birden çok kişinin veya çevrimdışı cihazın aynı içeriği düzenlediği ürünlerde.

Ne zaman kullanmamalıyım: İş kuralı birleştirmeye izin vermiyorsa CRDT veya last-write-wins'i otomatik çözüm sayma; finansal hak ve stok gibi değişmezler ayrıca korunmalıdır.

158 İstemci davranışı: Optimistic UI, başarısız işlemden geri dönüş, iptal, tekrar açılma ve mobil arka plan kısıtları.

İhtiyaca bağlı

Açıklama: Optimistic UI sonucu beklemeden ekranda başarıyı varsayar; hata olursa düzeltme gerekir. Uygulama kapandığında ve yeniden açıldığında bekleyen işin durumu korunmalıdır.

Örnek: Beğeni simgesi hemen değişir; sunucu reddederse durum geri alınır veya yeniden deneme gösterilir.

Ne zaman kullanılır: Kullanıcıya hızlı tepki vermek istediğinde; yanlış başarı izleniminin kabul edilebilir olduğu etkileşimlerde.

Karar tablosu: polling, SSE veya WebSocket?

Açıklama: Güncellemeyi ne kadar çabuk görmek istediğin ve hangi yönlerde mesaj gönderildiği, bağlantı seçimini belirler.

Seçenek	Gerçek hayat örneği ve ne zaman kullanılır?	Bedeli / ne zaman gereksiz?
Polling	Bir raporun durumunu birkaç saniyede bir HTTP ile sorgulamak; saniyelik gecikme kabul ediliyorsa	Boş sorgu ve istek yükü üretir. Seyrek güncellemede basittir; sürekli düşük gecikme gerekiyorsa yetersiz kalabilir.
SSE	AI yanıtını veya iş ilerlemesini sunucudan tarayıcıya metin olaylarıyla akıtmak	Uzun bağlantı, proxy timeout'u ve yeniden bağlanma yönetilir. İstemci komutları ayrı HTTP ile gidebilir; tam çift yön gerekmiyorsa uygundur.
WebSocket	Ortak düzenleme veya oyun için iki yönde sık mesaj taşımak	Bağlantı yaşam döngüsü, heartbeat, yetki yenileme ve akış kontrolü gerekir. Birkaç durum güncellemesi için gereksiz olabilir.

Somut örnek: AI metni için SSE; kullanıcı mesajını normal POST; iş durumunu ilk sürümde polling ile çözebilirsin. Bu seçim ürün gereksinimine bağlıdır; WebSocket otomatik olarak daha iyi değildir.

Sık hata: Bağlantı yeniden kurulunca olayların kendiliğinden eksiksiz geleceğini sanmak. Olay kimliği, kaldığı yerden devam ve arada kaçan veriyi yeniden okuma politikası gerekir. **Karar sorusu:** İki yönlü trafik gerçekten gerekli mi; bağlantı koparsa ekran hangi son doğrulanmış duruma dönecek? SSE, WebSocket.

Yapay zekâ kullanan sistemler

Şema 18 | Kaynaklı AI ve RAG



Şemayı oku: RAG ilgili ve yetkili belgeleri getirir; model bunlarla yanıt üretir. Kaynak bulunması tek başına doğruluk garantisi değildir. Kaynak ve iddia kontrolünden sonra yanıt verilir; yetersiz veya çelişkili bilgi için güvenli çıkış gerekir. Yapay zekâ (Imagegen) ile üretilmiştir.

159 İstek bütçesi: Model seçimi, bağlam uzunluğu, token maliyeti, kota ve eşzamanlılık sınırı.

İhtiyaca bağlı

Açıklama: AI çağrısı süre, işlenen metin miktarı ve eşzamanlı iş kapasitesi tüketir. Token, modelin işlediği metin parçalarının ölçüsüdür; gereksiz bağlam maliyeti büyütür.

Örnek: Kısa başlık önerisi için tüm belge arşivini ve en pahalı modeli kullanmak gerekmeyebilir.

Ne zaman kullanılır: Model ve istek akışını seçerken; ürünün kullanım maliyetini ve sınırlarını belirlerken.

160 Yanıt yaşam döngüsü: İlk token süresi, toplam süre, streaming, kullanıcı iptali ve yarım kalan yanıt.

İhtiyaca bağlı

Açıklama: İlk yanıtın gelme süresiyle yanıtın tamamlanması farklıdır. Akış, iptal ve bağlantı kopması boyunca neyin saklandığını ve kullanıcıya ne gösterildiğini belirlersin.

Örnek: Kullanıcı 'durdur'a bastığında arayüz beklemeyi bırakır ve destekleniyorsa arka plandaki üretim de iptal edilir.

Ne zaman kullanılır: AI sohbeti, metin üretimi veya uzun süren analiz arayüzünde.

161 RAG: Parçalama, embedding, retrieval, reranking, kaynak atfı ve bilginin güncellenmesi.

İhtiyaca bağlı

Açıklama: RAG önce ilgili belgeleri bulur, sonra yanıt üretirken modele bu parçaları verir. Reranking bulunan parçaları yeniden sıralar; kaynak göstermek ve doğruluğu kontrol etmek yine gerekir. RAG açıklaması.

Örnek: Şirket asistanı izin politikasını cevaplamadan önce güncel insan kaynakları belgesindeki ilgili bölümü getirir.

Ne zaman kullanılır: Özel veya sık değişen bilgiye dayalı yanıt gerektiğinde.

Sık hata: İlgili bir belge bulunmasını güncel, yetkili ve cevabı destekleyen kaynak bulunmasıyla aynı saymak.

Karar sorusu: Retrieval doğru belgeyi mi getirdi, belge güncel ve erişilebilir mi, yanıtın iddialarını gerçekten destekliyor mu?

Ne zaman kullanmamalıyım: Bilgi ihtiyacı basit sorguyla karşılanıyorsa RAG ekleme. Erişim filtreleri ve kaynak güncellemesi güvenilir değilken hassas belgeleri modele verme.

162 Veri ve araç güvenliği: Prompt injection, güvenilmeyen belgeler, erişim filtreleri ve araçların yetki sınırları.

İhtiyaca bağlı

Açıklama: Modelin okuduğu belgeye gizlenmiş komutlar davranışını saptırabilir. Yetkiyi yalnız modele bırakmayıp veri erişimi ve araç çağrısında ayrıca uygularsın. OWASP prompt injection rehberi.

Örnek: Bir belgede 'tüm müşteri listesini gönder' yazması asistana bu işlemi yapma hakkı vermez.

Ne zaman kullanılır: Belge, web sayfası veya e-posta okuyan; veri değiştiren araçlara erişen AI sistemlerinde.

163 Kalite değerlendirmesi: Temsilî örnekler, doğruluk ve görev başarısı, regresyon, insan geri bildirimi ve üretimde izleme.

İhtiyaca bağlı

Açıklama: Kaliteyi birkaç hoş yanıtla değil, gerçek görevleri temsil eden örneklerle ölçersin. Yeni sürümün önce doğru yaptığı işleri bozup bozmadığını da kontrol edersin.

Örnek: Destek asistanı güncellemesinde kolay sorular kadar belirsiz, yanıltıcı ve cevabı belgede bulunmayan soruları da denersin.

Ne zaman kullanılır: Model, prompt veya bilgi kaynağı değiştirirken ve üretimde davranış izlerken.

Sık hata: Akıcı birkaç yanıt veya yüksek genel ortalama üzerinden küçük ama kritik bir kullanıcı grubundaki hatayı kaçırmak.

Karar sorusu: Hangi başarısız örnek veya alt grup regresyonu yayını durdurur; basit çözümle karşılaştırma sonucu ne?

164 İşletim ve yan etkiler: Model/prompt sürümleme, fallback, cache, sağlayıcı kesintisi ve araç çağrılarında tekrarın iş kurallarına etkisi.

İhtiyaca bağlı

Açıklama: Model yanıtı değişken olabilir; kullanılan sürümü ve prompt'u izlemek gerekir. Tekrar, fallback ve cache davranışı özellikle gerçek işlem yapan araçlarda dikkat ister.

Örnek: Randevu oluşturan AI ağ hatası sonrası aynı çağrıyı tekrar ederse aynı randevuyu iki kez açmamalıdır.

Ne zaman kullanılır: AI yalnız metin üretmeyip sipariş, randevu veya veri güncelleme işlemi yaptığında.

Yapay zekâ çözümünü seçmek

165 AI gerekliliği ve baseline: Modeli kural, arama veya insan destekli basit çözümle karşılaştırmak.

İhtiyaca bağlı

Açıklama: Baseline karşılaştırılacak başlangıç çözümüdür. AI eklemenin görev başarısına katkısını, gecikme ve maliyetle birlikte değerlendirirsin.

Örnek: Üyelik türüne göre sabit indirim uygunluğu, açık kurallarla çözülebiliyorsa LLM kullanmak ek belirsizlik yaratabilir.

Ne zaman kullanılır: AI özelliğine başlamadan veya model tabanlı çözümü büyütmeden önce.

Sık hata: AI kullanılmasını başlı başına başarı hedefi saymak.

Karar sorusu: Aynı işi daha basit çözüm ne kadar iyi yapıyor ve model ne ekliyor?

Ne zaman kullanmamalıyım: Kural veya arama çözümü hedefi karşılıyorsa ek model çağrısı ekleme. Hata maliyeti yüksekken değerlendirme ve güvenli çıkış yolu olmadan AI'ı karar sahibi yapma.

166 Prompt, RAG ve fine-tuning ayrımı: Davranış, dış bilgi ve eğitimle uyarlama ihtiyaçlarını ayırmak.

İhtiyaca bağlı

Açıklama: Prompt görev talimatını ve verilen bağlamı düzenler; RAG çalışma anında ilgili bilgiyi getirir; fine-tuning örneklerle model davranışını uyarlar. Biri diğerinin tüm sorunlarını çözmez.

Örnek: Her gün değişen kampanya koşulları için önce güncel ve yetkili kaynağı getirmeyi düşünürsün; biçim tutarlılığı ihtiyacını ayrıca değerlendirirsin.

Ne zaman kullanılır: Yanıt kalitesi sorununun bilgi, talimat, davranış veya veri kaynağından hangisine bağlı olduğunu seçerken.

Sık hata: Eski veya eksik bilgi sorununu otomatik olarak daha büyük model ya da fine-tuning ile çözmeye çalışmak.

Karar sorusu: Değişmesi gereken şey güncel bilgi mi, görev talimatı mı, öğrenilmiş davranış mı?

Ne zaman kullanmamalıyım: Sık değişen dış bilgiyi yalnız fine-tuning ile güncel tutmaya çalışma. Davranış sorunu ile bilgi erişimi sorununu ayırmadan eğitim sürecine başlama.

167 Workflow ve agent: Önceden belirli işlem akışı ile modelin adım ve araç seçimini ayırmak.

İhtiyaca bağlı

Açıklama: Workflow belirlenmiş adımları yürütür; agent göreve göre sonraki eylemi seçebilir. Araç yetkisi, durdurma ve gözlemleme sınırları her iki modelde de açık olmalıdır.

Örnek: Fatura alanlarını çıkarıp doğrulayan sabit akış yeterliyken, değişken araştırma adımları daha esnek bir düzen gerektirebilir.

Ne zaman kullanılır: AI'a araç ve çok adımlı görev yetkisi verirken.

Sık hata: Tek bir araç çağrısı olan her akışı agent saymak veya esnekliği kontrol ihtiyacını azaltıyor sanmak.

Karar sorusu: Hangi seçim modele bırakılmalı ve hangi iş kuralı sunucuda kesin uygulanmalı?

Ne zaman kullanmamalıyım: Adımlar ve araçlar önceden belirliyse açık bir workflow yeterli olabilir. Değerlendirme, adım bütçesi ve araç yetkisi sınırı yokken serbest agent döngüsü kurma.

168 Cevap vermeme ve insan devri: Kaynak yetersizliği, çelişki ve yüksek hata maliyetinde güvenli çıkış tanımlamak.

İhtiyaca bağlı

Açıklama: Sistem her soruya kesin yanıt üretmek zorunda değildir. Bilginin sınırını göstermek, ek bilgi istemek veya ilgili kişiye aktarmak ürün davranışıdır.

Örnek: İptal koşulunu açıklayan iki kaynak çelişiyorsa asistan sürümleri gösterir, kesin söz vermez ve gerekli incelemeye yönlendirir.

Ne zaman kullanılır: Kaynaklı asistan, hesap işlemi ve yanlış yanıtın kullanıcıya önemli zarar verebildiği akışlarda.

Sık hata: Akıcı cevabı doğruluk kanıtı saymak veya insana devri bağlamsız bir çıkmaza dönüştürmek.

Karar sorusu: Hangi durumda duracağız ve kullanıcı bağlamıyla birlikte kime aktarılacak?

Ne zaman kullanmamalıyım: Modelin güvenilir kaynakları ve güvenli araç yetkisi yoksa insan devrini kaldırma. Otomatik ret kuralını her belirsizliğe uygulanacak genel engel olarak da kullanma.

AI çıktı sözleşmeleri ve araç entegrasyonları

169 Structured output ve iş kuralı doğrulaması: Modelden beklenen biçim ile verinin doğru olması ayrı kontrollerdir.

İhtiyaca bağlı

Açıklama: Şema, alan ve türleri tanımlar. Şemaya uyan yanıt yine yanlış fiyat, tarih veya nesne kimliği içerebilir; iş kuralları ve yetki sunucuda doğrulanır.

Örnek: Model geçerli JSON içinde stokta olmayan ürün kimliği döndürür; sipariş servisi bunu kabul etmez.

Ne zaman kullanılır: Model çıktısını kod, form veya işlem akışı tüketecekse.

Sık hata: JSON parse edilebilmesini doğruluk veya işlem yetkisi kanıtı saymak.

Karar sorusu: Biçim doğru ama içerik geçersizse sistem nasıl duracak veya düzeltecek?

170 Tool calling ve MCP sınırları: Model araç çağrısı isteyebilir; çağrıyı uygulama yetkisi içinde yürütür.

İhtiyaca bağlı

Açıklama: Tool calling bir araç ve parametre isteğidir. MCP, araç ve kaynakları istemciye sunmanın standart yollarından biridir; kullanıcının yetkisini, araç güvenliğini veya doğru parametreyi tek başına garanti etmez.

Örnek: Asistan sipariş durumunu okuyabilir; iptal isteği için ayrıca nesne yetkisi, iş kuralı ve gerekiyorsa kullanıcı onayı denetlenir.

Ne zaman kullanılır: AI'ı dış sistemlere veya veri değiştiren araçlara bağlarken.

Sık hata: Bir araç listede görünüyor diye her kullanıcı adına çağrılabileceğini düşünmek.

Karar sorusu: Çağrı hangi kimlikle, hangi kapsamda ve hangi yan etkiyle çalışıyor?

171 Model-as-judge ve değerlendirme veri ayrımı: Otomatik hakem puanı, bağımsız doğruluk kanıtı değildir.

İhtiyaca bağlı

Açıklama: Başka bir model yanıtları ölçüte göre puanlayabilir; bu hakem de yanlış, tutarsız veya yanlı olabilir. Geliştirme örnekleriyle saklı testleri ayırmak ve insan kontrolüyle karşılaştırmak gerekir.

Örnek: Uzun ama yanlış cevap hakemden yüksek puan alırken kısa ve doğru cevap düşük puan alıyorsa değerlendirme ölçütü yeniden incelenir.

Ne zaman kullanılır: Çok sayıda AI yanıtını karşılaştırırken, prompt/model değişikliklerini değerlendirirken.

Sık hata: Geliştirme sırasında tekrar tekrar görülen örneklerden bağımsız başarı oranı çıkarmak.

Karar sorusu: Hakem hangi hata türlerini kaçırıyor; kritik örneklerde insan değerlendirmesiyle ne kadar uyuşuyor?

Çok kiracılı ürünler ve maliyet

172 Multi-tenancy: Paylaşılan tablo, ayrı şema ve ayrı veritabanı; izolasyon ve işletim maliyeti.

İhtiyaca bağlı

Açıklama: Tenant, aynı ürünün hizmet verdiği ayrı müşteri veya kurumdur. Veriyi ortak veya ayrı depolamak izolasyon ve işletim maliyetini değiştirir.

Örnek: İki şirket aynı muhasebe uygulamasını kullanır ama birbirinin faturalarını hiçbir sorguda görememelidir.

Ne zaman kullanılır: Bir yazılımla birden fazla bağımsız kuruma hizmet verirken.

Ne zaman kullanmamalıyım: İzolasyon ihtiyacı ortak tabloda karşılanabiliyorsa her küçük müşteriye ayrı veritabanı açma; daha güçlü sınır gerekiyorsa sırf ucuz diye ortak modeli dayatma.

173 Noisy neighbor: Bir müşterinin diğerlerinin kapasitesini tüketmesi; kota, kaynak havuzu ve adil kuyruk.

İhtiyaca bağlı

Açıklama: Ortak kaynakta çok iş yapan müşteri diğerlerini yavaşlatabilir. Tenant başına sınır ve ayrı iş kapasitesi etkisini azaltır.

Örnek: Bir şirket milyonlarca satır rapor üretirken diğer şirketlerin giriş ekranı yavaşlamamalıdır.

Ne zaman kullanılır: Paylaşımlı altyapıda büyük ve küçük müşteriler birlikte çalışıyorsa.

174 Kullanıcı ve tenant yaşam döngüsü: Davet, üyelik, rol değişimi, veri dışa aktarma, silme ve tenant taşıma.

İhtiyaca bağlı

Açıklama: Kurum üyeliği ve verisi zamanla değişir. Davet, rol kaldırma, taşıma ve silme birbiriyle tutarlı yürütülmelidir.

Örnek: Çalışan şirketten ayrılınca mevcut oturumlarıyla şirket raporlarına erişememelidir.

Ne zaman kullanılır: Kurumsal üyelik ve hesap yönetimi geliştirirken.

175 Entitlement ve kullanım ölçümü: Satın alınan hak, fiilî kullanım, gecikmeli olay ve tekrar kayıtların faturaya etkisi.

İhtiyaca bağlı

Açıklama: Entitlement hangi özellik veya miktarın satın alınmış olduğudur; kullanım ölçümü gerçekten tüketileni kaydeder. Geç veya tekrarlı olaylar hesaplamayı bozmamalıdır.

Örnek: Aynı PDF üretim olayı iki kez geldiyse müşterinin aylık kotasından iki kullanım düşülmez.

Ne zaman kullanılır: Abonelik, paket limiti veya kullanıma göre ücretlendirme kurarken.

176 Birim maliyet: Kullanıcı, istek veya iş başına compute, storage, ağ çıkışı, telemetri ve AI maliyeti.

Temel

Açıklama: Toplam faturayı iş birimine bölerek büyümenin ekonomik etkisini görürsün. İşlemci dışında depolama, veri çıkışı, izleme ve AI de maliyet yaratır.

Örnek: Bir video özeti ucuz üretilebilir ama dosyanın sürekli indirilmesi ağ çıkış maliyetini büyütebilir.

Ne zaman kullanılır: Fiyatlama, kapasite ve özellik önceliği belirlerken.

177 Build versus buy: Yönetilen servis, kendi işletme, sağlayıcı bağımlılığı, taşınabilirlik ve toplam sahip olma maliyeti.

Temel

Açıklama: Hazır hizmet geliştirme ve işletim yükünü azaltabilir; kendi çözümün kontrol sağlar ama bakım sorumluluğu getirir. Karşılaştırma yalnız lisans ücretini kapsamaz.

Örnek: Küçük ekip e-posta altyapısını işletmek yerine yönetilen hizmet seçip ürünün asıl işine zaman ayırabilir.

Ne zaman kullanılır: Ödeme, arama, bildirim, kimlik veya veritabanı gibi bir altyapı ihtiyacını çözerken.

Ne zaman kullanmamalıyım: Güvenlik, veri erişimi ve çıkış koşulları belirsizken yalnız ilk fiyat üzerinden servis satın alma; ekibin işletemeyeceği altyapıyı da otomatik olarak içeride geliştirme.

Başvuru: AWS Well-Architected Framework, mimari kararların güvenilirlik, güvenlik, verimlilik ve maliyet boyutlarını birlikte ele alır.

Performans ve hız

İlk paylaştığın 20 maddelik pratik kontrol listesi. Bu bölümdeki kutular, ilgili yöntemi ve hangi durumda uygulanacağını bildiğini gösterir.

Ölçüm ve teşhis

178 Önce ölç: Lighthouse ve PageSpeed ile mobil performansı test et.

Temel

Açıklama: Önce nerede bekleme olduğunu ölçersin. Lighthouse kontrollü testtir; PageSpeed Insights varsa gerçek kullanıcı verisini de gösterir. LCP ana içeriğin görünmesini, INP etkileşime tepkiyi, CLS beklenmedik yer değişimini izler. Web Vitals.

Örnek: Ürün sayfasında gecikmenin büyük kapak fotoğrafından mı, JavaScript'ten mi kaynaklandığını raporla ayırırsın.

Ne zaman kullanılır: İyileştirmeden önce ve sonra; aynı koşulları karşılaştırıp tek puana değil kullanıcı akışına bakarken.

179 Uygulamayı düşük fiyatlı bir Android telefonda, 4G bağlantısıyla test et.

Temel

Açıklama: Güçlü bilgisayar ve hızlı Wi-Fi gerçek kullanıcıdaki işlemci, bellek ve ağ sınırlarını gizleyebilir. Fiziksel cihaz testi bu farkı ortaya çıkarır.

Örnek: Kendi bilgisayarında akıcı olan ürün listesi düşük donanımlı telefonda kaydırırken takılabilir.

Ne zaman kullanılır: Mobil deneyimi doğrularken; temsili cihaz ve gerçek ya da kontrollü yavaş bağlantıyla.

180 Yapay zekâya sor: “Bu uygulamayı yavaşlatan ne?”

Temel

Açıklama: AI'dan ölçülebilir neden adayları ve bunları ayıracak kontroller istersin. Söylediği neden, ölçümle doğrulanana kadar hipotezdir.

Örnek: 'Bu ekranda hangi sorgu veya dosya yavaşlığa neden olabilir; hangi ölçümle ayırırız?' diye sorup ağ ve sorgu kayıtlarıyla karşılaştırırsın.

Ne zaman kullanılır: Kodda sorunu aramayı hızlandırmak için; ölçüm ve doğrulamaya yardımcı olarak.

Görseller ve fontlar

181 Fotoğraf ve raster görsellerde WebP veya AVIF, doğru boyut ve sıkıştırma seçimi; SVG gibi uygun mevcut formatları gereksiz yere dönüştürmemek.

Temel

Açıklama: Görselin biçimini, kalite düzeyini ve piksel boyutunu ekrandaki ihtiyaca uydurursun. Küçük ekrana gereğinden büyük dosya göndermek aktarımı artırır.

Örnek: Ürün kartında küçük görünen fotoğraf için yüksek çözünürlüklü orijinal yerine uygun boyutlu WebP veya AVIF sürümü sunarsın.

Ne zaman kullanılır: Fotoğraflar sayfa ağırlığının önemli bölümünü oluşturuyorsa; görsel kaliteyi gözle kontrol ederek.

182 İlk ekranın altında kalan görselleri ihtiyaç duyulduğunda yükle (lazy loading).

Temel

Açıklama: Lazy loading, henüz görünmeyen görseli kullanıcı yaklaşınca indirmektir. İlk ekrandaki kritik görseli geciktirmek açılışı kötüleştirebilir.

Örnek: Uzun ürün listesinin altındaki fotoğrafları kaydırma sırasında yükler, üstteki ana ürün fotoğrafını hemen getirirsin.

Ne zaman kullanılır: Uzun sayfalarda ve görsel ağırlıklı listelerde.

183 Kritik ana fontu gerektiğinde preload etmek, font sayısını sınırlamak ve yükleme stratejisinin metin görünürlüğüne etkisi; başlangıç hedefi olarak 1-2 font.

İhtiyaca bağlı

Açıklama: Preload önemli fontun daha erken istenmesini sağlar; çok font ve gereksiz preload başka dosyalarla yarışır. Font beklenirken metnin nasıl göründüğü de önemlidir.

Örnek: Yazı okunmadan marka fontunu bekletmek yerine uygun yedek font gösterir, yalnız gereken font dosyasını önceliklendirirsin.

Ne zaman kullanılır: Font yüklenmesi metni geciktiriyor veya yerleşimi sıçratıyorsa.

JavaScript ve üçüncü taraflar

184 Yapay zekânın eklediği ancak kullanılmayan paketleri kaldır.

Temel

Açıklama: Kullanılmayan bağımlılık bakım ve güvenlik yükü yaratır; pakete gerçekten dahil oluyorsa indirme boyutunu da artırabilir. Dinamik ve build sırasında kullanımını kontrol edersin.

Örnek: Artık kaldırılmış bir grafik ekranı için eklenen kütüphane hâlâ projede kalmış olabilir.

Ne zaman kullanılır: AI ile hızlı geliştirme, özellik kaldırma veya bağımlılık incelemesi sonrasında.

185 JavaScript'i böl; her sayfa yalnızca ihtiyaç duyduğu kodu yüklesin.

Temel

Açıklama: Code splitting, uygulama kodunu ihtiyaç anında yüklenen parçalara ayırır. Aşırı parçalama da ek istek maliyeti getirebilir.

Örnek: Giriş sayfasını açan kullanıcı, yalnız yönetim panelinde kullanılan büyük grafik paketini indirmez.

Ne zaman kullanılır: İlk açılışta yüklenen JavaScript büyüdüğünde veya ağır özellikler yalnız bazı ekranlarda kullanıldığında.

186 İhtiyaç duyulmayan üçüncü taraf betiklerini kaldır.

Temel

Açıklama: Harici analiz, reklam ve sohbet betikleri ağ ile işlemci tüketir; bazıları kullanıcı deneyimini bekletebilir. İş değerini ve yükünü birlikte değerlendirirsin.

Örnek: Artık kullanılmayan eski kampanya takip betiğini kaldırır, aktif dönüşüm ölçümünün çalıştığını kontrol edersin.

Ne zaman kullanılır: Sayfada çok sayıda dış istek veya ana iş parçacığını meşgul eden betik varsa.

Veritabanı ve API

187 Filtreleme ve sıralama örüntülerine uygun veritabanı indeksleri seçmek; sorgu planıyla faydayı, yazma ve depolama maliyetiyle birlikte doğrulamak.

Temel

Açıklama: Sık yapılan aramaya uygun indeks daha az kayıt okumayı sağlayabilir. İndeks faydasını sorgu planı ve gerçek sorgu süresiyle ölçersin.

Örnek: 'Bu müşterinin son siparişleri' sorgusunda müşteri ve tarih alanlarını birlikte kullanan indeks değerlendirilir.

Ne zaman kullanılır: Yavaş sorgu belirlediğinde; indeks eklemenin yazmaları ve depolamayı da etkilediğini hesaba katarak.

188 N+1 sorgularını düzelt.

Temel

Açıklama: N+1, önce listeyi sonra her satır için ayrı ek sorguyu çalıştırmaktır. Toplu sorgu veya uygun join gereksiz gidiş gelişleri azaltır.

Örnek: Yirmi siparişi bir sorguyla, her müşteriyi ayrı sorguyla almak toplam 21 sorgudur; bilgileri toplu getirebilirsin.

Ne zaman kullanılır: ORM kullanan liste ekranında kayıt sayısıyla sorgu sayısı birlikte artıyorsa.

189 Listelerin tamamını yüklemek yerine sayfalama kullan.

Temel

Açıklama: Sayfalama yalnız ihtiyaç duyulan kayıt grubunu getirir. Böylece ağ aktarımı, bellek ve ekran çizimi sınırlanır.

Örnek: On bin siparişin tamamını açılışta yüklemek yerine ilk grubu gösterir, devamını kullanıcı ilerledikçe getirirsin.

Ne zaman kullanılır: Kayıt listesi büyüyebiliyorsa; devamını getirirken sıralamayı kararlı tutarak.

190 Nadiren değişen API yanıtlarını tazelik, yetki ve invalidation kurallarıyla önbelleğe almak.

İhtiyaca bağlı

Açıklama: Sık tekrarlanan ve uygun süre değişmeden kalabilen yanıtı yeniden üretmek yerine saklarsın. Hangi kullanıcıya ait olduğu ve ne zaman yenileneceği açık olmalıdır.

Örnek: Herkese açık kategori listesini cache'larsın; kişisel hesap özetini aynı ortak anahtarla paylaşmazsın.

Ne zaman kullanılır: Aynı okuma pahalıysa ve kabul edilebilir tazelik süresi belirlenebiliyorsa.

Algılanan hız ve etkileşim

191 Yapay zekâ yanıtının tamamını beklemek yerine yanıtı akış halinde göster (streaming).

İhtiyaca bağlı

Açıklama: Streaming, üretilen metni parçalar geldikçe gösterir. Toplam üretim süresini mutlaka kısaltmaz ama kullanıcı ilk parçayı daha erken okuyabilir.

Örnek: Uzun bir belge özeti bitene kadar boş alan göstermek yerine paragrafları geldikçe ekrana yazarsın.

Ne zaman kullanılır: Uzun AI yanıtlarında; yarım yanıt, bağlantı kopması ve iptal durumlarını da yöneterek.

192 Boş ekranlar yerine iskelet ekranlar (skeleton) ve yüklenme durumları göster.

Temel

Açıklama: Skeleton gelecek içeriğin yerini gösteren geçici şekildir; yükleme durumu işin sürdüğünü anlatır. Bunlar gerçek beklemeyi azaltmaz, belirsizliği azaltır.

Örnek: Ürün kartları yüklenirken yerleri görünür; hata olursa sonsuz animasyon yerine hata ve yeniden deneme sunulur.

Ne zaman kullanılır: İçerik hemen gelmiyorsa ve kullanıcı ne olduğunu anlayamıyorsa.

193 Arama girdilerine debounce uygula.

Temel

Açıklama: Debounce, kullanıcı yazmaya kısa süre ara verince aramayı başlatır. Eski isteğin geç gelen sonucu yeni aramayı ezmemelidir.

Örnek: 'Kulaklık' yazarken her harfte sorgu yerine örneğin 300 milisaniyelik duraklamadan sonra ararsın.

Ne zaman kullanılır: Uzak sunucuda arama veya pahalı filtreleme yaparken; uygun beklemeyi kullanıcı deneyimiyle belirleyerek.

Sunucu ve dağıtım

194 Statik dosyaları CDN üzerinden sun.

İhtiyaca bağlı

Açıklama: CDN dosyaların kopyalarını farklı noktalarda tutar; kullanıcı uygun yakın noktadan alabilir. Dosya değişince sürümleme veya cache temizleme gerekir.

Örnek: Mağaza logosu ve ürün fotoğrafları her ziyaret için uygulama sunucusundan gönderilmez.

Ne zaman kullanılır: Görsel, CSS, JavaScript ve indirilebilir statik dosyaları geniş kullanıcı kitlesine sunarken.

195 Ağır işleri arka plan görevlerine taşı.

Temel

Açıklama: Uzun hesap veya aktarımı isteğin önünden çıkarıp işleyiciye verirsin. Kullanıcıya iş kimliği, ilerleme ve sonuç durumu göstermek gerekir.

Örnek: Binlerce satırlık satış raporu istenir; ekran hemen 'hazırlanıyor' der, dosya hazır olunca indirme bağlantısı açılır.

Ne zaman kullanılır: Video işleme, toplu e-posta, rapor veya büyük içe aktarma kullanıcıyı uzun bekletiyorsa.

196 Sunucuyu kullanıcı ve veri yerleşimini birlikte değerlendirerek konumlandırmak; uygulama ile veritabanı arasındaki gecikmeyi de ölçmek.

Temel

Açıklama: Sunucu bölgesi ağ gidiş gelişini etkiler. Kullanıcıya ve sık erişilen veriye göre toplam isteğin yolunu değerlendirmek gerekir.

Örnek: Kullanıcı İstanbul'da, uygulama yakın bölgede ama veritabanı uzak kıtadaysa her ek sorgu uzak mesafeyi tekrar kat eder.

Ne zaman kullanılır: Bölge seçerken veya farklı ülkelerdeki kullanıcıların gecikmeleri belirgin ayrıştığında.

197 Ana rotalarda cold start etkisini ölçmek; başlatma maliyetini azaltma veya hazır kapasite tutmanın gecikme ve maliyet dengesini bilmek.

İhtiyaca bağlı

Açıklama: Cold start, hazır çalışan yokken ortamın başlatılmasına harcanan süredir. Başlatmayı hafifletmek veya hazır kapasite tutmak gecikmeyi azaltabilir; hazır kapasitenin maliyeti vardır.

Örnek: Bir süredir kullanılmayan işlevin ilk randevu isteği yavaş, hemen sonraki istekleri hızlı olabilir.

Ne zaman kullanılır: İlk istek gecikmesi ölçümle doğrulandığında ve ana kullanıcı akışını etkilediğinde.

Ek mimari ve ürün kararları

Bu maddeler güvenlik, dağıtım, veri ve ürün davranışı kararlarını tamamlar. Mevcut kavram numaraları korunmuştur; her tekniğin her üründe kullanılması gerekmez.

198 WAF ve ağ güvenlik duvarı: HTTP isteklerini filtrelemek ile ağ erişimini sınırlamanın farkı.

İhtiyaca bağlı

Açıklama: Ağ güvenlik duvarı adres, port ve protokol gibi ağ kurallarıyla erişimi sınırlar. WAF, uygulamaya gelen HTTP/HTTPS isteğinin başlık, yol ve gövde gibi parçalarını kurallarla değerlendirir. Kuralın neyi görebildiği ve engelleyebildiği ürüne ve ayara bağlıdır; WAF bulunsa da nesne yetkisi ve güvenli kod uygulamada ayrıca sağlanır.

Örnek: Sipariş API'sinin önündeki WAF şüpheli istek desenlerini engeller; kullanıcının başka bir müşterinin siparişini okuyamamasını sipariş servisi ayrıca denetler.

Ne zaman kullanılır: İnternete açık uygulamada ek saldırı filtreleme katmanı ve merkezi kural yönetimi gerektiğinde.

Sık hata: WAF açıkken tüm güvenlik açıklarının kapanacağını düşünmek veya yanlış pozitifleri izlemeden kuralı bütün trafiğe uygulamak.

Karar sorusu: Kural hangi saldırıyı hedefliyor; meşru kullanıcıları engellerse nasıl fark edip geri alacağız?

Ne zaman kullanmamalıyım: WAF'ı nesne yetkisi ve güvenli uygulama kodunun yerine koyma; kuralı meşru trafiğe etkisini ölçmeden bütün isteklere uygulatma.

199 12 Factor App: Taşınabilir uygulama ve güvenli süreç yaşam döngüsü.

Temel

Açıklama: Bu çerçeve kod tabanı, açık bağımlılıklar, koddan ayrı yapılandırma, bağlı dış servisler, build/release/run ayrımı, stateless süreçler, port üzerinden hizmet, süreçlerle eşzamanlılık, hızlı açılış ve kontrollü kapanış, geliştirme/üretim benzerliği, olay akışı olarak loglar ve tek seferlik yönetim işlerini birlikte ele alır. Amaç aynı uygulamayı ortamlarda öngörülebilir işletmektir; mikroservis kurma zorunluluğu değildir. Hassas yapılandırma güvenli sır yönetimiyle sağlanır.

Örnek: Aynı container imajı testte ve üretimde farklı bağlantı ayarlarıyla çalışır. Kapanış sinyali alan worker yeni iş almayı durdurur; mevcut işi tamamlar veya güvenli tekrar için kuyruğa bırakır.

Ne zaman kullanılır: Uygulamayı farklı ortamlara taşırken, yatay büyütürken ve kesintisiz sürüm geçişini tasarlarken.

Sık hata: Bir makinenin yerel diskini kalıcı iş kaydı saymak veya süreç durdurmayı yalnız bağlantıyı kesmek olarak görmek.

Karar sorusu: Bu süreç aniden kapanırsa hangi iş ve durum kaybolur; yeniden başlayan süreç bunları nereden güvenle devam ettirir?

200 API Gateway: İstemciler ile servisler arasındaki ortak giriş sınırı.

İhtiyaca bağlı

Açıklama: Gateway, istemci isteklerini uygun servise yönlendiren ve kota, kimlik doğrulama veya TLS sonlandırma gibi ortak işleri üstlenebilen bir giriş katmanıdır. Reverse proxy genel yönlendirme rolünü, BFF belirli arayüzün veri ihtiyacını anlatır; ürünler bu rolleri birlikte sunabilir. Kaynağa erişim ve iş kuralları ilgili serviste de korunur.

Örnek: Mobil istemci sipariş ve profil API'larına tek alan adından ulaşır; gateway doğru servise yönlendirir ve kullanıcı kotasını uygular.

Ne zaman kullanılır: Birden çok servisi istemcilere tutarlı bir sözleşme ve ortak trafik politikalarıyla sunarken.

Sık hata: Bütün iş mantığını gateway'e taşımak, onu tek hata noktası yapmak veya servise doğrudan erişimde yetki kontrolünü atlamak.

Karar sorusu: Hangi politika ortak girişte, hangisi serviste kalmalı; gateway yavaşladığında etkilenecek kapsam nedir?

Ne zaman kullanmamalıyım: Ortak giriş ihtiyacı yokken fazladan gateway katmanı ekleme. İş kurallarını tek bir merkezi darboğaza taşıma.

201 MFA ve TOTP: İkinci faktör, doğrulama kodu ve hesap kurtarma.

Temel

Açıklama: MFA, parola gibi bir bilgiye ek olarak farklı türden bir faktör ister. TOTP, kurulumda paylaşılan gizli anahtar ve zaman adımından kısa ömürlü kod üretir; bu anahtar korunur ve kod sunucuda doğrulanır. TOTP de phishing ile ele geçirilebilir. WebAuthn/passkey gibi phishing'e dayanıklı seçenekler ve güvenli kurtarma yolu ayrıca değerlendirilir.

Örnek: Yönetici parolasından sonra doğrulayıcı uygulamanın kodunu girer. Telefonunu kaybettiğinde hesabı parolayı bilene otomatik açmak yerine tanımlanmış kurtarma süreci çalışır.

Ne zaman kullanılır: Hesap ele geçirmenin etkisi yüksek olduğunda, özellikle yönetim ve hassas işlem erişimlerinde.

Sık hata: İki farklı parolayı iki faktör saymak, TOTP'yi phishing'e dayanıklı sanmak veya kurtarma akışını girişten daha zayıf bırakmak.

Karar sorusu: İkinci faktör kaybolursa kullanıcı nasıl döner; saldırgan aynı yolu kullanarak korumayı atlayabilir mi?

202 Redis veri yapıları: Cache dışında sayaç, sıralama ve oturum örnekleri.

İhtiyaca bağlı

Açıklama: Redis farklı veri yapıları sunar: string ve INCR ile sayaç, hash ile alanlı kayıt, sorted set ile puan sırası, bitmap ile bit tabanlı işaretleme tutulabilir. Oturum gibi geçici veriler için TTL anlamlı olabilir. Veri yapısını seçmek kalıcılık, yedekleme, bellek sınırı ve kayıp toleransı kararlarının yerini almaz.

Örnek: Oyundaki haftalık puan tablosu sorted set ile sıralanır; finansal ödül hakkının asıl kaydı ayrıca kalıcı veritabanında tutulur.

Ne zaman kullanılır: Düşük gecikmeli ortak durum veya veri yapısına uygun yoğun erişim gerektiğinde; kayıp ve yeniden üretim koşulları biliniyorsa.

Sık hata: Her veriyi cache saymak, silinme politikasını görmezden gelmek veya tek atomik komutun çok adımlı işin tamamını güvenli yaptığına inanmak.

Karar sorusu: Bu kayıt silinir veya kopya geride kalırsa yeniden üretilebilir mi; hangi bilgi mutlaka asıl kaynaktan doğrulanmalı?

Ne zaman kullanmamalıyım: Yeniden üretilemeyen kritik veriyi kalıcılık ve kurtarma ayarlarını doğrulamadan geçici Redis cache'i gibi saklama.

203 Bildirim karar politikası: Hangi olayın, kime, ne zaman ve hangi kanaldan bildirileceği.

Temel

Açıklama: Olayın oluşması tek başına bildirim gönderme kararı değildir. Kullanıcı tercihi, kanal ayarı, sessize alma, rahatsız etmeyin zamanı, olayın önemi ve başka cihazdaki etkinlik birlikte değerlendirilir. Kuralların önceliği açık olmalı; izin veya teslim sonucu ise ayrı izlenmelidir. Gerçek bir ürünün bütün davranışını tek eski şemadan çıkarmamak gerekir.

Örnek: Temsilî ekip uygulamasında kanal mesajı sessize alınmışsa push üretilmez; doğrudan mention için istisna olup olmayacağı ürün kuralında ayrıca yazılır. Ertelenen olay uygulama içi gelen kutusunda kalır.

Ne zaman kullanılır: Sohbet, iş takibi, alarm veya kampanya bildirimlerini tasarlarken ve kullanıcı şikâyetlerini açıklarken.

Sık hata: Mention'ın bütün sessize alma kurallarını kendiliğinden aştığını varsaymak veya gönderilmeyen bildirimi sistem hatası saymak.

Karar sorusu: İki tercih çatıştığında hangisi kazanıyor; kullanıcı bir bildirimin neden geldiğini ya da gelmediğini anlayabiliyor mu?

204 Kubernetes bileşenleri: İstenen durumdan çalışan Pod'a giden yönetim akışı.

İleri

Açıklama: Kontrol düzlemi kümenin durumunu yönetir: API server yönetim API'sini sunar, etcd API verisini tutar, scheduler henüz node'a atanmış olmayan Pod için uygun node seçer, controller'lar istenen duruma yaklaşmak için çalışır. Node üzerindeki kubelet ve container runtime Pod'un container'larını çalıştırır. Kullanıcı trafiğinin yolu ile bu yönetim akışı farklıdır.

Örnek: Üç kopya istenen uygulamanın bir Pod'u kaybolduğunda denetleyiciler farkı gidermek için çalışır; yeni Pod uygun node'da başlatılır. Hazır olmayan kopyaya trafik verilmemesi ayrıca readiness kontrolüyle ilişkilidir.

Ne zaman kullanılır: Kubernetes kullanan bir üründe dağıtım, Pending Pod, hazır olmayan kopya veya kontrol düzlemi arızasını tartışırken.

Sık hata: Scheduler'ın container'ı kendisinin çalıştırdığını veya Kubernetes'in uygulama hatasını ve veri kaybını otomatik çözdüğünü düşünmek.

Karar sorusu: Sorun istenen durumun kaydında mı, node seçilmesinde mi, container'ın başlamasında mı, yoksa trafiğe hazır olmasında mı?

Ne zaman kullanmamalıyım: Basit uygulama dağıtımı yeterliyken veya küme işletim sorumluluğu karşılanamıyorken Kubernetes seçme; orkestrasyon uygulamanın veri doğruluğunu kendiliğinden sağlamaz.

205 Kartlı ödeme ağı: Issuer, acquirer, yetkilendirme ve hesaplaşma ayrımı.

İhtiyaca bağlı

Açıklama: Issuer kartı veren kuruluştur; acquirer işyerinin kart kabulü tarafındaki kuruluştur. Kart ağı ve işlem sağlayıcıları aralarındaki mesajlaşma ve işlemleri destekler. Authorization işlem için onay, capture tahsilata ilerleme talebi, clearing işlem bilgilerinin takası, settlement ise yükümlülüklerin hesaplaşmasıdır. Bir aşamadaki olumlu yanıt, paranın işyerinin hesabında kullanılabilir olduğu anlamına gelmez.

Örnek: Mağaza, kart için alınan onayı kaydeder; ürün gönderildiğinde sağlayıcı modeline göre capture yapar. İşyerine yapılan ödeme ve kesintiler daha sonra ayrı kayıtlarla uzlaştırılır.

Ne zaman kullanılır: Ödeme durumlarını, iade akışını, finans operasyonunu ve ödeme sağlayıcısı entegrasyonunu tanımlarken.

Sık hata: Ödeme gateway'ini kartı veren banka sanmak; authorization ile settlement'ı aynı durum veya sabit evrensel komisyon oranı kabul etmek.

Karar sorusu: Kullanıcıya gösterdiğimiz 'ödendi' hangi doğrulanmış olaya dayanıyor; tahsilat ve işyerine ödeme hangi kayıtlarla uzlaştırılıyor?

Üretimde veri güvenliği ve karar sınırları

Bu bölümdeki senaryolar temsili ürün örnekleridir. Önceki kavram numaraları korunur. Kullanım sınırları gereksinime göre karar vermek içindir.

206 Veriyi tüm kopyalardan silmek ve yeniden görünmesini önlemek

Temel

Açıklama: Silme; asıl kayıt, cache, arama indeksi, dosya, analitik ve AI veri kopyalarını kapsayan bir süreçtir. Mantıksal silme ile fiziksel kaldırmanın koşulları farklıdır. Yedekler kendi saklama döngüsüne tabidir; geri yüklemeden sonra silme kararlarının yeniden uygulanması tasarlanır. Google Cloud örneği sağlayıcıya özgüdür.

Örnek: Temsilî bir hesap silme işinde veri envanteri çıkarılır; bağımlı depolar temizlenir ve sonuç izlenir. Eski yedek açılırsa silme kayıtları uygulanmadan kullanıcı erişimi açılmaz.

Ne zaman kullanılır: Hesap veya tenant silme, belge kaldırma ve kişisel veri yaşam döngüsü tasarlarken.

Ne zaman kullanmamalıyım: Saklanması gereken iş kayıtlarıyla kaldırılacak kişisel alanları ayırmadan toplu fiziksel silme yapma. Saklama ve erişim politikası önce netleşmeli.

Sık hata: Veritabanındaki satırı silince arama sonuçları, dosyalar ve eski yedeklerin de temizlendiğini varsaymak.

Karar sorusu: Hangi kopyalar var; her biri için tamamlanma kanıtı ne ve yedek dönüşünde silinen verinin geri gelmesini kim önler?

207 Olay şemasının evrimi ve eski mesajların yeniden işlenmesi

İhtiyaca bağlı

Açıklama: Olayı üreten ve tüketen servisler farklı zamanlarda güncellenir. Yeni tüketicinin eski veriyi okuyabilmesi ile eski tüketicinin yeni veriyi okuyabilmesi ayrı uyumluluk yönleridir. Kurallar Avro, Protobuf veya JSON Schema tanımına göre değişir. Confluent uyumluluk rehberi.

Örnek: OrderCreated olayına currency eklenir. Eski mesajda alan yoksa anlamı açık bir dönüşüm uygulanır; eksik alan sessizce yanlış para birimine çevrilmez. Replay sırasında tekrar tahsilat yapılmaz.

Ne zaman kullanılır: Birden çok tüketicili olay sistemlerinde, uzun süre saklanan mesajlarda ve replay/backfill işlerinde.

Ne zaman kullanmamalıyım: Yalnız tek sürümün yaşadığı kısa ömürlü yerel işlem için ayrı schema registry kurma; önce gerçek uyumluluk ihtiyacını belirle.

Sık hata: Sadece en son iki sürümü test edip yıllar önceki olayların yeni kodla okunacağını varsaymak.

Karar sorusu: Hangi eski üretici, tüketici ve mesaj sürümleri destekleniyor; uyumluluk testi alanın iş anlamını da doğruluyor mu?

208 Finansal hareket defteri ve sağlayıcıyla mutabakat

İhtiyaca bağlı

Açıklama: Ödeme durumu, bakiye ve para hareketi ayrı kayıtlardır. Hareket defteri borç/alacak taraflarını ve iş referansını tutar; düzeltmelerin izi korunur. Çift taraflı kayıt kavramı TigerBeetle dokümanında açıklanır. İç hareketler ayrıca sağlayıcı kayıtlarıyla karşılaştırılır; dengeli defter tek başına doğru tahsilat kanıtı değildir.

Örnek: Temsilî 1.000 TL ödeme ve 200 TL kısmi iade ayrı hareketlerdir. Komisyon ve ters ibraz kendi referanslarıyla izlenir; sağlayıcıdan gelen tutarla fark varsa inceleme kaydı açılır.

Ne zaman kullanılır: Cüzdan, pazaryeri, bakiye, kısmi iade veya finans operasyonu olan ürünlerde.

Ne zaman kullanmamalıyım: Basit bir tahsilat entegrasyonuna ihtiyaç kanıtı olmadan özel muhasebe motoru ekleme. Finansal kuralları yalnız geliştiricinin varsayımıyla belirleme.

Sık hata: Bakiye sütununu doğrudan değiştirip değişikliğin nedenini kaybetmek veya ödeme başarılı olayını hesaba para geçti sanmak.

Karar sorusu: Her hareket hangi işleme dayanıyor; tekrar kayıt nasıl engelleniyor ve sağlayıcıyla farkı kim sonuçlandırıyor?

209 Denetim izi ve hassas işlem geçmişi

Temel

Açıklama: Audit trail; aktörü, yapılan işlemi, zamanı, hedefi ve sonucu izlenebilir tutar. Gereken yerlerde gerekçe ve değişen alanlar da kaydedilir. Kayıtların okunması ve değiştirilmesi sınırlanır; hassas veri ayıklanır. OWASP Logging güvenlik ve koruma koşullarını açıklar.

Örnek: Destek çalışanı müşteri rolünü değiştirir. İşlem referansı, önceki ve sonraki rol, aktör ve sonuç denetim kaydına yazılır; parola veya token yazılmaz.

Ne zaman kullanılır: Yönetici işlemleri, yetki değişiklikleri, kritik veri düzeltmeleri ve destek müdahalelerinde.

Ne zaman kullanmamalıyım: Her hata ayıklama ayrıntısını süresiz denetim kaydına dönüştürme; alanları ve saklama amacını belirle. Audit trail'i event sourcing yerine kullanma.

Sık hata: İşlem başarısızken başarılı audit kaydı üretmek veya uygulamayı yöneten herkese geçmişi sessizce silme yetkisi vermek.

Karar sorusu: İş kaydıyla denetim kaydı ayrışırsa nasıl fark ederiz; kaydı okuyan veya değiştiren kimlikleri de izliyor muyuz?

210 Yazılım tedarik zinciri ve artifact kökenini doğrulamak

İhtiyaca bağlı

Açıklama: Bağımlılık listesi, build kaynağı ve dağıtılan paket arasında iz kurulur. SBOM kullanılan bileşenleri tanımlar; provenance paketin nasıl ve hangi girdilerle üretildiğini açıklar. İmza ve doğrulama politikası ayrı kontrollerdir. SLSA artifact kökeni ve tedarik zinciri garantilerini aşamalı ele alır; imzalı paket hatasız kod demek değildir.

Örnek: Üretimdeki container digest'i kaynak commit'e bağlanır. Bir bağımlılıkta açık duyurulduğunda ilgili bileşeni içeren sürümler bulunup kontrollü güncellenir.

Ne zaman kullanılır: CI/CD, üçüncü taraf bağımlılıkları ve dağıtım güvenliğini tasarlarken.

Ne zaman kullanmamalıyım: Güvenilen kimliği ve doğrulama politikasını belirlemeden yalnız imza ekleyip güvenlik işi tamam sayma. Gereksinim olmadan en karmaşık sertifikasyon düzeniyle başlama.

Sık hata: Bir etiketi paket kimliği saymak veya doğrulamadığın provenance dosyasını güven kanıtı olarak kabul etmek.

Karar sorusu: Bu paket hangi koddan üretildi; kim üretti ve üretime kabulde hangi kanıt gerçekten doğrulandı?

211 Noktaya geri yükleme ve kurtarma tatbikatı

Temel

Açıklama: Kurtarma; uygun yedeği, işlem günlüğü zincirini, hedef anı ve uygulama kontrollerini birlikte sınar. PostgreSQL PITR temel yedek ve WAL replay kullanır; bu yol mantıksal dump ile aynı değildir. Geçmişe dönüş, hedef andan sonraki geçerli işlemlerin ayrıca uzlaştırılmasını gerektirebilir.

Örnek: Temsilî bir yanlış silmeden önceki ana izole ortamda dönülür. Kaybolan kayıtlar ve daha sonra gelen geçerli siparişler iş referanslarıyla karşılaştırılır; tekilleştirme ve yeniden işleme planıyla hizmet açılır.

Ne zaman kullanılır: RPO/RTO hedefini kabul etmeden, yedekleme sistemi değiştiğinde ve kurtarma prosedürünü sınarken.

Ne zaman kullanmamalıyım: Prosedürü ilk kez canlı verinin üstüne deneyerek öğrenme. Failover'ı yanlış silmeden kurtarma yerine kullanma; silme replikaya da yayılabilir.

Sık hata: Yedeğin açılmasını yeterli sayıp iş kayıtlarını, dış sağlayıcı sonuçlarını ve silme kararlarını kontrol etmemek.

Karar sorusu: Gerçekte hangi ana dönebildik; hangi geçerli işlemler ayrıştı ve erişimi açmadan hangi doğruluk kontrolleri geçti?

212 Uzun süren işlerde ve dosya indirmede yetkiyi yeniden değerlendirmek

Temel

Açıklama: İşin başında verilen erişim, sonuç teslim edilirken hâlâ geçerli olmayabilir. Kuyruktaki iş aktör ve tenant bağlamını korur; veri okuma, kritik yan etki ve çıktı teslimi ürün politikasına göre yeniden denetlenir. OWASP Authorization her istekte ve doğru sınırda yetki kontrolünü vurgular.

Örnek: Çalışan rapor ister, sonra kurumdan çıkarılır. Rapor üretilmiş olsa da indirme isteği güncel üyeliği denetler; daha önce verilmiş imzalı bağlantının süresi ve iptal sınırı ayrıca değerlendirilir.

Ne zaman kullanılır: Rapor dışa aktarma, kuyruk işleri, gecikmeli teslim ve rolün değişebildiği uzun akışlarda.

Ne zaman kullanmamalıyım: Geri alınamaz bir iş etkisini belirsiz istemci rolüyle başlatma. Her adımda aynı uzak servisi körlemesine çağırmak yerine yetki ve iptal politikasını tanımla.

Sık hata: Kuyruğa yazılan rol bilgisini süresiz yetki belgesi sanmak veya bağlantıyı bileni yetkili kabul etmek.

Karar sorusu: Yetki hangi aşamalarda tekrar kontrol ediliyor; iptal gecikmesinin ve daha önce verilmiş erişimin sınırı ne?

213 Dosyayı güvenli kabul etme ve işleme hattı

Temel

Açıklama: Yükleme, doğrulama, karantina, işleme ve yayımlama ayrı aşamalardır. Uzantı veya istemcinin Content-Type başlığı yeterli güvence sağlamaz. Tür, içerik, boyut, isim ve erişim kontrolleri birlikte uygulanır; tarama da kusursuz güvence değildir. OWASP File Upload.

Örnek: Yüklenen PDF önce özel alana alınır. Kurallara uygunluğu kontrol edilir; uygun işlerde zararlı içerik taraması ve izole dönüştürme yapılır. Başarısız dosya yayımlanmaz; kullanıcı durumunu görür.

Ne zaman kullanılır: Kullanıcı belgesi, görsel, arşiv veya medya kabul eden ürünlerde.

Ne zaman kullanmamalıyım: Güvenilmeyen dosyayı doğrudan web köküne koyma veya uygulama sürecinde sınırsız kaynakla açma. İmzalı upload URL'sini içerik güvenliği sayma.

Sık hata: .pdf uzantısını yeterli bulmak, açılınca aşırı büyüyen arşivi atlamak veya işleme bitmeden dosyayı herkese açmak.

Karar sorusu: Dosya hangi kontrolden sonra erişilebilir; işlemci kaynak sınırını aşarsa iş nasıl durdurulur ve kısmi çıktılar ne olur?

214 Arama sıralamasının kalitesini değerlendirmek

İhtiyaca bağlı

Açıklama: Aramanın teknik olarak yanıt vermesi, kullanıcının aradığını bulduğu anlamına gelmez. Temsilî sorgular ve ilgili kabul edilen belgelerle bir değerlendirme seti kurulur. Precision, MRR veya nDCG gibi ölçüt seçimi ürün amacına bağlıdır. Elastic rank evaluation bu yöntemin somut uygulamasıdır.

Örnek: Destek aramasında yazım hatalı Türkçe sorgular, sonuçsuz sorgular ve doğru belgenin gerilerde kaldığı örnekler birlikte sınanır. Sıralama değişince bir grubu düzeltirken diğerini bozup bozmadığı ölçülür.

Ne zaman kullanılır: Arama, bilgi bankası, katalog veya RAG retrieval sonuçlarını iyileştirirken.

Ne zaman kullanmamalıyım: Yalnız birkaç beğenilen örnek veya tıklanma oranıyla kalite kararı verme. Etiketlenmiş örnekler temsilî değilse ölçümü tüm kullanıcıya genelleme.

Sık hata: İndeks gecikmesi, erişim filtresi ve sıralama sorunlarını tek bir düşük kalite puanına indirgemek.

Karar sorusu: Kimin hangi bilgi ihtiyacını ölçüyoruz; doğru sonuç ilk sıralarda mı ve yetkisiz belge hiç görünüyor mu?

215 Kuyruk birikmesinden güvenli toparlanma

İhtiyaca bağlı

Açıklama: Kuyruğun dayanıklı olması, işleri zamanında bitireceğini garanti etmez. Geliş hızı, tüketim kapasitesi, en eski işin yaşı ve işin geçerlilik süresi birlikte izlenir. AWS kuyruk rehberi birikmeyi önleme ve güvenli boşaltma seçeneklerini açıklar.

Örnek: Temsilî kesinti sonrasında yeni ödeme işleriyle süresi geçmiş bildirimler aynı sırada bekletilmez. İş türüne göre kapasite ve öncelik ayrılır; geçersiz işi atlama kararı açık iş kuralına dayanır.

Ne zaman kullanılır: Uzun kesinti, büyük replay, toplu kampanya veya tüketicinin üreticiye yetişemediği durumlarda.

Ne zaman kullanmamalıyım: Ödeme ve hak açma gibi işleri sırf eski diye sessizce silme. Alt bağımlılık kapasitesi yokken yalnız worker sayısını artırma.

Sık hata: Sadece mesaj sayısına bakıp en eski işin yaşını kaçırmak veya yeniden denemelerle toparlanma trafiğini büyütmek.

Karar sorusu: Yeni ve eski işler nasıl paylaşıyor; hangi iş atılabilir ve bağımlılıkları tekrar çökertmeden ne hızda boşaltabiliriz?

216 Yapılandırma ve iş kurallarını sürümlemek

İhtiyaca bağlı

Açıklama: Fiyat, kota ve özellik ayarları kod dağıtılmadan değişebilir. Yapılandırmanın doğrulanması, sürümü, etkinleşme anı ve geri dönüşü görünür olmalıdır. AWS AppConfig kontrollü dinamik yapılandırmaya bir servis örneğidir. Bir işlemin hangi kuralı kullanacağı ürün sözleşmesinde ayrıca belirlenir.

Örnek: Sipariş eski fiyat kuralıyla başlar. Kesinleşmiş teklifin sürümü ve tutarı kaydedilir; süre dolarsa güncel teklif gösterilip yeniden onay alınır. Sonradan yalnız güncel ayara bakarak eski sipariş açıklanmaz.

Ne zaman kullanılır: Kampanya, ücret, kota, feature flag veya uzun süren işlemi etkileyen dinamik kurallarda.

Ne zaman kullanmamalıyım: İş kuralı değişikliğini incelemesiz gizli ayara dönüştürme. Secret'ları herkese açık feature flag verisi gibi sunma; kritik kararları yalnız istemci bayrağına bırakma.

Sık hata: Kod rollback'inin fiyat veya kota ayarını da geri aldığını sanmak; eski işlemlere yeni kuralları sessizce uygulamak.

Karar sorusu: Bu işlem hangi kural sürümünü kullandı; değişiklik nasıl doğrulandı ve geri dönüşte devam eden işler nasıl davranacak?

Bütüncül sistem örnekleri

Aşağıdaki üç senaryo temsili tasarımlardır. Her biri mevcut bilgi maddelerinin nasıl birleştiğini gösterir; zorunlu bir teknoloji yığını veya doğrulanmış şirket mimarisi iddiası değildir.

Örnek sistem: ödeme ve sipariş

Açıklama: Kullanıcı tek sipariş görür; ödeme sağlayıcısı, stok ve sipariş verisi farklı zamanlarda sonuçlanabilir. **Örnek:** Kullanıcı ödeye basar, banka işlemi kabul eder ama yanıt ağda kaybolur. **Ne zaman kullanılır:** Para hareketi ile ürün/hizmet hakkını ilişkilendirirken.

Şema 19 | Ödeme durum makinesi



Şemayı oku - Durum makinesi: Timeout işlemi başarısız yapmaz; sonuç bilinmiyor durumuna geçirir. Bu durumdan yalnız doğrulanmış sağlayıcı sonucu ile çıkılır. Tekrarlı veya geç bildirimler başarılı işlemi geriye götürmez; geçişler sunucuda denetlenir. Yapay zekâ (Imagegen) ile üretilmiştir.

Korunacak koşullar: Aynı işleme iki kez ücret uygulamamak; doğrulanmış ödeme olmadan siparişi ödenmiş saymamak; kesinleşmiş ödemenin siparişe yansımamasını tespit edip uzlaştırmak. İade, ödemeden ayrı kayıt ve süreçtir.

Veri ve sözleşme: Order(id, user_id, total, currency, status, version), PaymentAttempt(id, order_id, provider_id, idempotency_key, status) ve WebhookInbox(provider_event_id UNIQUE, processed_at). Sipariş tutarı sunucuda doğrulanır. Idempotency anahtarı işlem sahibine ve istek içeriğine bağlanır; aynı anahtarla farklı tutar kabul edilmez. POST /orders, POST /orders/{id}/payment-attempts ve GET /orders/{id} bu örneğin ürün API'leridir.

Akış	Tasarım kararı	Hata ve toparlanma
Sipariş oluştur	Fiyat ve stok kuralını sunucuda doğrula; sipariş kimliğini kalıcılaştır.	Fiyat değişmişse yeni tutarı kullanıcıya onaylat.
Ödemeyi başlat	Denemeyi kaydet; sağlayıcı destekliyorsa aynı mantıksal denemede aynı idempotency anahtarını kullan.	Timeout kesin ret değildir; mevcut provider işlem kimliğini sorgula.
Sonucu doğrula	Webhook imzasını ve sipariş/tutar/para birimini doğrula; geçişi yerel transaction'da uygula.	Tekrarlı veya sırasız webhook'ta geçmiş duruma gerileme; yalnız olay kimliğini tekilleştirmek yeterli değildir.
Siparişe yansıt	Aynı DB içindeyse ödeme kaydı ve sipariş durumu birlikte güncellenebilir; servisler ayrılmışsa outbox + inbox ve uzlaştırma kullan.	Yayıncı çökerse outbox kalır; tüketici tekrar teslimi tanır.
Kullanıcıya göster	Bekliyor, sonucu bilinmiyor, başarılı ve başarısız durumlarını ayır.	Kullanıcı ekranı kapatsa da sunucu uzlaştırmaya devam eder; gerekirse operasyon kaydı açılır.

Gözlenecek sinyaller: Sonucu belirsiz en eski ödeme yaşı, ödenmiş ama tamamlanmamış sipariş sayısı, webhook gecikmesi, uzlaştırma başarısızlığı. Kimlikler arasında bağ kur; hassas ödeme verisi taşıma.

Sık hata: Yalnız butonu devre dışı bırakarak çift ödemeyi önlediğini sanmak. **Karar sorusu:** Banka başarılı ama sipariş servisi çökmüşse bu işlemi hangi kalıcı kayıt bulup tamamlayacak?

Örnek sistem: arka planda rapor hazırlama

Açıklama: HTTP isteği işi kabul eder; worker sonucu daha sonra üretir. **Örnek:** Bir müşteri son bir yıllık satış raporunu ister. **Ne zaman kullanılır:** İş, etkileşimli yanıt süresini aşarsa veya ağır kaynak tüketirse.

Veri ve sözleşme: Job(id, tenant_id, status, input_ref, attempt, lease_until, result_ref, error_code). POST /reports iş kaydını ve outbox niyetini kalıcılaştırdıktan sonra 202 + job_id döndürür. GET /jobs/{id} durumunu; yetkili sonuç endpoint'i süreli indirme erişimini verir. 202, rapor tamamlandı demek değildir.

Akış	Tasarım kararı	Hata ve toparlanma
Kabul et	Yetki, kota ve parametreleri doğrula; işi ve yayın niyetini tek transaction'da kaydet.	Kuyruk kapalı olsa da iş kaybolmaz; kabul etmek kapasite politikasıyla sınırlıdır.

Akış	Tasarım kararı	Hata ve toparlanma
İşle	Worker işi lease/visibility süresiyle alır; uzun işte süreyi yeniler; deneme bütçesi vardır.	Worker ölürse iş yeniden alınabilir; aynı anda iki worker olasılığına karşı claim/version kontrolü gerekir.
Sonucu yaz	Tekrara dayanıklı çıktı anahtarı kullan; dosya ve DB işaretinin atomik olmadığını hesaba kat.	Yarım dosya temizlenir veya geçici çıktıdan kesin sonuca geçilir; iş DB'de tamamlanmadan ACK verilmez.
Yeniden dene	Geçici hatada backoff + jitter; kalıcı hatada açıklanabilir son durum; tekrar bütçesi dolarsa DLQ.	DLQ'ya atmak çözüm değildir; alarm, inceleme ve kontrollü replay gerekir.
Sonucu sun	İlk sürümde polling yeterli olabilir; gerekirse SSE ile ilerleme bildir.	Sonuç başka tenant'a açılmaz; iptal isteğinin devam eden işi ne ölçüde durdurduğu açıkça tanımlanır.

Gözlenecek sinyaller: En eski bekleyen iş yaşı, tamamlanma süresi, deneme sayısı, DLQ birikimi ve tenant bazında kaynak tüketimi. Kuyruk uzunluğu tek başına kullanıcı bekleyişini açıklamaz.

Sık hata: En az bir kez teslimde e-posta veya dosyanın da otomatik tek kez üretileceğini sanmak. **Karar sorusu:** Çıktı kaydedildiği anda worker ölürse ikinci deneme hangi kayıtla devam edecek?

Örnek sistem: kaynaklı AI destek asistanı

Açıklama: Yanıt üretmeden önce kullanıcının erişebildiği belgeler aranır; yanıt bu kanıtlarla ilişkilendirilir. **Örnek:** Çalışan şirketin izin politikasını sorar; yalnız yetkili olduğu güncel İK belgesi kaynak gösterilir. **Ne zaman kullanılır:** Yanıtların değişen kurum içeriğine dayanması ve kaynak izinin görünmesi gerekiyorsa.

Veri ve sözleşme: Document(id, tenant_id, version, acl, source_uri), Chunk(id, document_id, version, embedding), AnswerRun(id, user_id, model_version, retrieved_chunk_ids, status). Arama indeksi asıl izin kaynağı değildir. POST /answers yanıt işini başlatır; metin ve tamamlanma/hata olayları akıtılır. Model seçimi, saklama süresi ve sağlayıcıya gönderilecek veriler ürünün gizlilik koşullarıyla belirlenir.

Akış	Tasarım kararı	Hata ve toparlanma
Belgeyi al	Kaynak sürümünü, sahipliği ve erişim kurallarını taşı; çıkarım ve indekslemeyi arka planda yap.	Silinen belge için indeks ve cache temizliği izlenir; yeniden erişim kontrolü ihlali engeller.
Soruyu ara	Tenant ve belge yetkisini retrieval sırasında uygula; bağlama eklenecek parçaları tekrar doğrula.	Yetkisiz parça önce modele gönderilip sonra ekrandan gizlenmez.
Yanıt üret	Kaynak metni veri olarak işle; belge içindeki talimatın araç yetkisini değiştirmesine izin verme.	Kanıt yetersizse bunu söyle; dayanağı olmayan cevap üretmek yerine uygun destek yolunu göster.
Akıt ve kaynaklandır	Metin geldikçe göster; kaynak bağlantılarının gerçekten dönen parçalara ait olduğunu denetle.	Akış yarıda kesilirse ekran “tamamlandı” demez; tekrar üretimde maliyet ve çift iş kontrolü gerekir.

Akış	Tasarım kararı	Hata ve toparlanma
Değerlendir	Yanıt doğruluğu, kaynak desteği, yetki ihlali, ilk token süresi ve token maliyetini ayrı ölç.	Yanlış yanıtın sebebi retrieval, eski kaynak veya model olabilir; yalnız modeli büyütmek teşhis değildir.

Cache kararı: Yanıt cache anahtarı tenant, izin kapsamı ve kaynak sürümünü hesaba katmalı; erişim değişince yeniden kontrol gerekir. Ortak soruyu ortak cevap sanmak kullanıcılar arasında veri sızdırabilir.

Sık hata: Kaynak linki var diye her cümlenin kaynak tarafından desteklendiğini kabul etmek. **Karar sorusu:** Bu yanıtı gören kişinin tüm dayanak belgelere şu anda erişimi var mı; kanıt eksikse ne gösteriliyor?

Sayısal kapasite ve maliyet örneği

Açıklama: Trafik, eşzamanlı iş, depolama ve maliyet farklı hesaplanır. **Örnek:** Günlük bir milyon istek alan temsili bir uygulama. **Ne zaman kullanılır:** İlk kapasite bütçesini çıkarmak, yük testi hedefini seçmek ve büyüme kararını tartışmak için. Bunlar **varsayım ve hesap örneğidir**; üretim ölçümü veya güncel bulut fiyat teklifi değildir.

Trafikten sunucu sayısına

Varsayımlar: Günde 1.000.000 istek; tepe trafik ortalamanın 10 katı; tepenin üstüne %50 kapasite payı; tepe anında ortalama yanıt süresi 300 ms. Bir uygulama instance'ı hedef gecikme/hata sınırları içinde 80 RPS taşıyabiliyor **varsayılır**; bu sayı yük testiyle doğrulanmalıdır.

Hesap	İşlem	Sonuç ve anlamı
Ortalama RPS	1.000.000 / 86.400	Yaklaşık 11,57 istek/saniye
Tepe RPS	11,574... × 10	Yukarı yuvarlayarak 116 RPS
Kapasite hedefi	116 × 1,50	174 RPS
Ortalama sistem içi istek	116 × 0,300 saniye	Yaklaşık 35 eşzamanlı istek
Gerekli çalışan instance	Yukarı yuvarla: 174 / 80	3 instance
Bir instance arızası için ek pay	3 + 1	4 instance; biri durursa kalan 3 × 80 = 240 RPS varsayılan hedefi karşılar

Yorum: Little yasasıyla bulunan 35, bu koşullardaki ortalama aktif istek sayısıdır; maksimum eşzamanlı bağlantı veya DB pool boyutu değildir. CPU, kuyruk, dağılımın kuyruğu ve bağımlılık sınırları ayrıca ölçülür. Dört instance hesabı yalnız tek instance kaybını ele alır; bölge kaybını otomatik karşılamaz. Google SRE kapasite tasarımı örneği.

Sık hata: Günlük ortalamayı sunucu kapasitesi saymak veya p99 gecikmesini ortalama yerine bu formüle koymak. **Karar sorusu:** Yük testinde hangi gecikme, hata ve kaynak sınırı aşılınca 80 RPS varsayımını reddederiz?

Veriden depolama ve çıkış trafiğine

Ek varsayımlar: Günde 100.000 yeni kayıt; kayıt başına 2 KiB; 30 gün saklama; toplam 3 veri kopyası. Her istekte ortalama 20 KiB dışarı yanıt ve 1 KiB log oluşuyor. 1 KiB = 1.024 bayt, 1 GiB = 1.073.741.824 bayt; ondalık GB ile karıştırma.

Kalem	İşlem	30 günlük miktar
Ham kayıt	100.000 × 2 KiB × 30	5,722 GiB

Kalem	İşlem	30 günlük miktar
Üç kopya	5,722... × 3	17,166 GiB
Yanıt payload'ı	1.000.000 × 20 KiB × 30	572,205 GiB
Log girişi	1.000.000 × 1 KiB × 30	28,610 GiB

Yorum: Veri hesabı indeks, WAL/işlem günlüğü, backup ve boş kapasiteyi kapsamaz. Payload hesabı protokol ek yükünü kapsamaz; CDN isabeti, sıkıştırma ve sağlayıcının ücretlendirdiği ağ yolu gerçek faturayı değiştirir.

Varsayımsal birim fiyatlarla maliyet

Aşağıdaki birim fiyatlar **hesabı göstermek için uydurulmuş varsayımlardır**. Ay 30 gün, yani 720 saattir. DB hizmeti satırı hesaplama/hizmet bedelini; veri satırı ayrı ücretlendirildiği varsayılan üç kopyanın depolamasını temsil eder. Gerçek pakette kopya depolaması dahilse yeniden eklenmez.

Kalem	Varsayımsal fiyat ve hesap	USD / ay
Uygulama	4 instance × 0,05 USD/saat × 720 saat	144,000
DB hizmeti	0,10 USD/saat × 720 saat	72,000
Veri depolama	17,166... GiB × 0,10 USD/GiB-ay	1,717
Çıkış trafiği	572,204... GiB × 0,08 USD/GiB	45,776
Log girişi	28,610... GiB × 0,50 USD/GiB	14,305
Toplam	Ham değerler üzerinden hesaplanır	Yaklaşık **277,80 USD**

Yorum: Bu dar kapsamlı örnek; load balancer/CDN sabit bedeli, kuyruk, backup, log saklama, bölgeler arası trafik, vergi, destek ve indirimleri içermiyor. Log girişi bedeli saklama bedeliyle aynı değildir. Üç ondalıklı kalemler okumayı kolaylaştırır; toplam ham değerlerden iki ondalığa yuvarlanır.

Duyarlılık: Yanıt boyutunu yarıya indirmek bu modelde yalnız çıkış trafiği kalemini yaklaşık yarıya düşürür. Trafik ikiye çıktığında her bedel otomatik ikiye katlanmaz; instance sayısı kapasite eşiği aşılınca basamaklı artar. **Sık hata:** Kullanıcı başına maliyeti yalnız sunucu sayısına bağlamak. **Karar sorusu:** Trafik 10 katına çıkarsa ilk önce hangi kapasite veya fatura kalemi tasarımı değiştirmeye zorlar?

Terim sözlüğü

Kısaltmayı gördüğünde bu tablodan anlamına bak; ayrıntı ve kullanım koşulları ilgili bilgi maddelerinde bulunur.

Terim	Kısa anlamı
RPS / QPS	Saniyedeki istek / sorgu sayısı; hangi katmanın sayıldığını belirt.
Latency / throughput	Bir işin bekleme süresi / birim zamanda tamamlanan iş miktarı.
p50 / p95 / p99	İsteklerin yüzde 50, 95 veya 99'unun altında kaldığı gecikme eşiği.
SLA / SLO / SLI	Dış taahhüt / iç hizmet hedefi / hedefi ölçen gösterge.
Error budget	SLO'nun izin verdiği başarısızlık payı; sınırsız arıza izni değildir.
RTO / RPO	Hedeflenen azami kesinti süresi / hedeflenen azami veri kaybı süresi.
TTL	Bir cache kaydının geçerli sayılacağı süre; kaynak güncelliğini tek başına garanti etmez.
Cache hit / miss	Verinin cache'de bulunması / bulunmaması.
Invalidation	Değişen veya artık yetkisiz verinin cache'deki kopyasını geçersiz kılma.
CDN	İçeriği kullanıcılara yakın noktalardan sunan dağıtım ağı.
DNS / TLS	Alan adını adrese çözme / bağlantıyı şifreleme ve karşı taraf kimliğini doğrulama.
HTTP / API	İstek-yanıt protokolü / bir yazılımın sunduğu işlem sözleşmesi.
Idempotency	Aynı mantıksal işlemi tekrarlamanın ek iş etkisi oluşturmaması.
Deduplication	Aynı olay veya işlemin tekrarlarını tanıyıp ayıklama.
ACK / NACK	Mesajın işlendiğine dair onay / işlenemediğine dair olumsuz bildirim.
Lease / visibility timeout	İşin sınırlı süre bir işleyiciye ayrılması; süre dolunca tekrar görünmesi mümkün.
DLQ	Deneme bütçesini tüketen veya işlenemeyen mesajların inceleme kuyruğu.
Backoff / jitter	Tekrarlar arasında artan bekleme / istemcilerin aynı anda yüklenmesini önleyen rastgele sapma.
Backpressure	Üreticinin hızını tüketicinin taşıyabildiği düzeye göre azaltma.
Load shedding	Sistem çökmeye yaklaşmadan işin bir kısmını kontrollü reddetme.

Terim	Kısa anlamı
Circuit breaker / bulkhead	Arızalı bağımlılığa çağrıyı kesme / kaynak havuzlarını ayırarak arızayı sınırlama.
Transaction / ACID	Birlikte kesinleşen işlem grubu / atomiklik, tutarlılık, izolasyon ve kalıcılık özellikleri.
Isolation	Eşzamanlı transaction'ların birbirinin ara sonuçlarından ne kadar korunduğu.
Index / N+1	Sorguyu hızlandıran veri yapısı / bir liste için gereksiz çok sayıda ek sorgu üretme.
Replica / shard	Aynı verinin kopyası / veri kümesinin ayrı sunuculara bölünmüş parçası.
Replication lag	Bir kopyanın birincildeki değişikliklerin gerisinde kalması.
Quorum / consensus	Karar için gereken oy/katılımcı eşiği / dağıtık katılımcıların ortak karara varması.
Fencing / split-brain	Eski sahibin yazmasını engelleme / birden fazla düğümün kendini yetkili lider sanması.
2PC / saga	Hazırlık ve ortak kararla dağıtık commit / yerel işlemler ve iş anlamında telafi dizisi.
Outbox / inbox	Yayın niyetini iş kaydıyla atomik yazma / tüketilen olayı yerel etkiyle atomik kaydetme.
CQRS / event sourcing	Komut ve sorgu modellerini ayırma / olayları asıl kayıt kabul etme.
Eventual consistency	Yeni güncellemenin tüm okumalarda hemen görünmeyebilmesi; yakınsama koşulları ayrıca gerekir.
BFF	Belirli bir arayüzün veri ve sözleşme ihtiyacına hizmet eden backend katmanı.
CSR / SSR / SSG	Tarayıcıda üretim / istek anında sunucuda üretim / önceden statik sayfa üretimi.
Hydration	Önceden üretilmiş HTML'e istemci davranışını bağlama.
Polling / SSE / WebSocket	Aralıklı sorgu / sunucudan olay akışı / çift yönlü sürekli mesaj bağlantısı.
Observability / trace	Sistemin davranışını sinyallerden anlayabilme / bir işin servisler boyunca izlediği yol.
Deploy / release / rollback	Sürümü ortama koyma / kullanıcıya açma / önceki çalışan sürüme dönme.
Canary / feature flag	Yeni sürümü trafiğin bir kısmında sınama / davranışı kod dağıtımından ayrı açıp kapatma.

Terim	Kısa anlamı
Runbook / postmortem	Müdahale rehberi / olay sonrası kanıtlı inceleme ve tekrarını önleme kaydı.
Tenant / ACL	Ayrı müşteri alanı / hangi kimliğin hangi veriye erişebileceği listesi.
RAG / embedding	Yanıt öncesi ilgili kaynakları getirip modele bağlama / içeriği sayısal vektörle temsil etme.
Token / context window	Modelin işlediği metin birimi / bir çalışmada kullanabildiği bağlam sınırı.
Rate limit / quota	Belirli sürede istek hızının sınırı / daha geniş kullanım hakkı veya bütçesi.
Cursor / offset	Sıralamada devam anahtarı / başlangıçtan atlanacak kayıt sayısı.

Kaynakça ve önemli bağlantılar

HTTP ve API sözleşmeleri

• MDN HTTP genel bakış - istek, yanıt, başlık ve bağlantı kavramları.

• MDN 202 Accepted - kabul edilen iş ile tamamlanan iş arasındaki fark.

• OpenAPI Specification - HTTP API'lerinin istek, yanıt ve şema sözleşmeleri.

• Stripe idempotent requests - ödeme sağlayıcısında tekrar ve anahtar davranışının somut örneği; garantiler sağlayıcıya ve API sürümüne göre değerlendirilir.

Teşhis araçları ve SQL

• Chrome DevTools Network referansı - istek, yanıt, cache ve zamanlama incelemesi.

• cURL kullanım kılavuzu - HTTP isteklerini komutla kurmak ve çıktıyı incelemek.

• Postman istek ve yanıt rehberi - yöntem, parametre, yetki, ortam ve yanıt incelemesi.

• PostgreSQL JOIN - tabloların ilişkiler üzerinden birleştirilmesi.

• PostgreSQL aggregate functions - sayma, toplama ve gruplama.

• PostgreSQL eşzamanlılık kontrolü - transaction, izolasyon ve eşzamanlı erişim.

Geliştirme ve yayın

• Pro Git değişiklikleri kaydetmek - çalışma alanı, staging ve commit.

• GitHub flow - branch, pull request, inceleme ve merge akışı.

• AWS Well-Architected Framework - mimari kararları güvenilirlik, güvenlik, performans ve maliyetle birlikte değerlendirmek.

Analitik ve güvenilirlik

• Microsoft Research güvenilir deney tasarımı - deney öncesi hipotez, metrik ve veri kalitesi kontrolleri.

• Google SRE hizmet seviyesi hedefleri - SLI, SLO ve kullanıcı odaklı hizmet hedefleri.

• Google SRE kitabı - izleme, aşırı yük, hata ve işletim başlıkları.

• Web Vitals - kullanıcı deneyimini etkileyen web performans ölçütleri.

Dağıtık sistemler ve güvenlik

• Apache Kafka tasarım dokümanı - mesaj teslimi, tüketim ve garantilerin sınırları.

• Raft - dağıtık anlaşma ve çoğunluk yaklaşımı.

• OWASP API Security Top 10 - API güvenliğinde nesne yetkisi ve diğer önemli riskler.

Yapay zekâ sistemleri

• Google Rules of Machine Learning - basit başlangıç çözümü, ölçüm ve ürün hedefleri.

• Anthropic Building effective agents - workflow ve agent ayrımı, araç kullanımı ve karmaşıklık seçimi.

• Google Cloud RAG açıklaması - dış bilgiyi bulup üretimde kullanma yaklaşımı.

• Hugging Face fine-tuning rehberi - önceden eğitilmiş modeli göreve veya alana özgü örneklerle uyarlamak.

• OWASP prompt injection rehberi - güvenilmeyen içerik, araçlar ve yetki sınırları.

Önceki kişisel çalışmadan örnekler

• Teknik Ürün Atlası - önceki ders ve örneklerin sitesi.

• Teknik Ürün Atlası kaynak deposu - içerik ve kaynak envanteri.

• Uçtan uca kurgusal vakalar - giriş, satın alma, kampanya ve kaynaklı AI asistanı.

• Kısa hata senaryoları - belirsiz ödeme, eski fiyat, hesap değişimi ve eski istemci örnekleri.

• Karar soruları ve gözlenebilir kontroller - kavramları ürün davranışına bağlayan örnekler.

Teknik diyagram örnekleri

• ByteByteGo Read Replica Pattern - okuma, yazma ve çoğaltma yollarını ayıran teknik çizim örneği.

• ByteByteGo Types of Message Queues - bileşenleri aynı görsel dille karşılaştırma örneği.

• ByteByteGo Designing a Permission System - yetki kapsamlarını ve kontrol adımlarını görselleştirme örneği.

Bu genişletmenin başvuru bağlantıları

• AWS - Transactional outbox: İş verisi ve olay yayınlama niyetinin birlikte kalıcılaştırılması.

• AWS - Saga orchestration: Yerel işlemler ve telafi.

• PostgreSQL - Two-phase transactions: Hazırlanmış transaction'lar ve koordinasyon.

• Microsoft - CQRS: Komut ve sorgu modelini ayırmanın bedelleri.

• Microsoft - Event Sourcing: Olayları asıl kayıt olarak kullanmak.

• Microsoft - Circuit Breaker: Closed, open ve half-open durumları.

- Microsoft - Bulkhead: Kaynak izolasyonu.
- Google SRE - Handling Overload: Aşırı yükü kabul ve reddetme politikaları.
- web.dev - Rendering on the Web: CSR, SSR ve statik üretim seçenekleri.
- React - hydrateRoot: Hazır HTML'e davranış bağlama ve eşleşme gereksinimi.
- Microsoft - Backends for Frontends: İstemciye özel backend.
- PostgreSQL - Transaction Isolation: Eşzamanlı işlemlerin görünürlüğü ve çakışması.
- MDN - Server-sent events: Sunucudan olay akışı.
- MDN - WebSocket API: Çift yönlü bağlantı.
- Google SRE Workbook - Non-Abstract Large System Design: Sayısal sistem ve kapasite tasarımı.
- Google SRE Workbook - Incident Response: Olay müdahalesinde roller ve koordinasyon.
- PostgreSQL - Failover: Eski birincili durdurma ve devralmayı işletme.
- Debezium - Outbox Event Router: Outbox olaylarını değişiklik akışından yönlendiren uygulama örneği.

İstemci platformları ve AI sözleşmeleri için kaynaklar

- Android uygulama temelleri - platform bileşenleri ve yaşam döngüsü.
- Android App Links - bağlantıdan uygulama içeriğine yönlendirme.
- Android bildirim izni - izin reddi ve bildirim davranışı.
- W3C WCAG 2.2 - erişilebilir arayüz için başarı ölçütleri.
- OpenAI Structured Outputs - şemaya uygun çıktı ile içerik doğruluğunun ayrımı.
- MCP mimarisi - host, client, server ve araç/kaynak sınırları.
- Zheng ve arkadaşları - LLM-as-a-Judge - otomatik hakemlerde konum, uzunluk ve öz tercih yanlılıkları.

Ek kararlar için birincil kaynaklar

- AWS WAF resmî dokümanı - HTTP filtreleme, kurallar ve koruma kapsamı.
- The Twelve-Factor App - uygulama ilkeleri ve ortamlar arasında taşınabilirlik.
- 12 Factor Disposability - hızlı başlatma, kontrollü kapanış ve yeniden işleme.
- Microsoft API Gateway mimarisi - gateway, reverse proxy ve ortak sorumluluklar.
- OWASP MFA rehberi - ikinci faktör, TOTP, phishing ve kurtarma sınırları.
- Redis veri yapıları - sayaç, hash, sorted set, bitmap ve stream ayrımları.
- Redis INCR komutu - atomik sayaç işlemi ve kullanım örnekleri.
- Slack bildirim tercihleri - kanal ve doğrudan mesaj tercihlerinin ürün örneği.
- Slack bildirim duraklatma - rahatsız etmeyin ve bildirim programı örnekleri.
- Kubernetes bileşenleri - kontrol düzlemi, scheduler, kubelet ve node rolleri.

• VisaNet Connect Acceptance - authorization, capture, clearing ve settlement ayrımı.

• Visa ödeme bileşenleri - issuer, acquirer ve işlem sağlayıcısı rolleri.

• OWASP Session Management - tarayıcı oturumu, cookie ve token saklama.

• OWASP Top 10 2025 - web uygulamaları için güncel genel risk çerçevesi.

Üretim kararları için birincil kaynaklar

• Google Cloud veri silme - Aktif veri, yedekler ve silme aşamalarının sağlayıcıya özgü örneği; süreler başka sistemlere genellenmez.

• Confluent şema evrimi - Üretici ve tüketici uyumluluğu; kurallar formata ve seçilen moda göre değişir.

• TigerBeetle finansal kayıt modeli - Borç/alacak ve çift taraflı kayıt yaklaşımı; ürünün teknik modelini açıklar.

• Stripe bakiye hareketleri - Ödeme, iade, bakiye ve sağlayıcı hareketlerinin ayrımı; Stripe dışındaki garantiler ayrıca doğrulanır.

• OWASP Logging - Kayıt içeriği, hassas veri, erişim ve değiştirmeye karşı koruma.

• SLSA 1.2 - Yazılım tedarik zinciri, artifact kökeni ve doğrulama.

• PostgreSQL PITR - Temel yedek, WAL zinciri, hedef ana kurtarma ve zaman çizgileri; PostgreSQL uygulaması.

• OWASP Authorization - Her istekte yetki doğrulama ve güvenli erişim sınırları.

• OWASP File Upload - Dosya türü, içerik, depolama, erişim ve boyut kontrolleri.

• Elastic sıralama değerlendirmesi - Temsilî sorgular, değerlendirilmiş belgeler ve sıralama metrikleri.

• AWS kuyruk birikmesinden toparlanma - Birikmiş iş, önceliklendirme, backpressure ve güvenli toparlanma.

• AWS AppConfig - Kod dağıtımından bağımsız yapılandırma ve kontrollü davranış değişikliği; servis örneği.

• SPDX SBOM kavramları - Yazılım bileşenlerinin ve tedarik zinciri bilgilerinin standart biçimde ifade edilmesi.